

High-throughput Verifiable Distributed OPRF from Gold PRF

Nan Cheng
University of St. Gallen
St. Gallen, Switzerland
nan.cheng@unisg.ch

Yohei Watanabe
The University of
Electro-Communications
Tokyo, Japan
AIST
Japan
watanabe@uec.ac.jp

Yugo Kasashima
The University of
Electro-Communications
Tokyo, Japan
kasashima@uec.ac.jp

Ioannis Katis
University of St. Gallen
St. Gallen, Switzerland
ioannis.katis@unisg.ch

Aikaterini Mitrokotsa
University of St. Gallen
St. Gallen, Switzerland
katerina.mitrokotsa@unisg.ch

Abstract

An oblivious pseudorandom function (OPRF) is a two-party protocol that enables a client to obtain $F_k(x)$ on an input x without learning the server-held key k , while the server learns nothing about x . OPRF is a fundamental building block in a wide range of privacy-preserving applications, including password-authenticated key exchange (PAKE), private set intersection (PSI), and distributed function secret sharing. In this work, we present the first concrete, high-throughput, post-quantum secure, verifiable distributed OPRF (dOPRF) tolerating $t < n/2$ malicious servers and a malicious client over replicated secret sharing, built on the recently introduced Gold OPRF of Yang et al. (IEEE S&P 2025), which supports multi-bit outputs in a single invocation.

We adopt an offline-online paradigm and introduce new protocol designs for both phases. (i) In the offline phase, we develop efficient protocols for batched generation of replicated secret sharing of α^e whose cost is independent of e . We present two complementary approaches, each designed for different parameter regimes: the first leverages a degenerate additive encoding under which exponentiation commutes with secret sharing; the second employs $t+1$ designated dealers that prove dual-share consistency via non-interactive zero-knowledge proofs, instantiated using both VOLE-in-the-Head and Ligerio, yielding post-quantum security based solely on collision-resistant hashing. (ii) In the online phase, we propose a constant-round protocol that tightly integrates secure multiplication-and-opening with distributed zero-knowledge proof (DZKP) verification, reducing both computation and communication compared to naively using the standard DZKP framework. (iii) We give a verifiable input protocol that binds a possibly-malicious client to a single well-defined evaluation point; its consistency check is absorbed into a hash exchange the online protocol already

performs, and it lowers the client's upload from a full replicated share to one field element per server per input.

Our end-to-end benchmarks show that the construction substantially outperforms the state-of-the-art Legendre-PRF dOPRF of Kaluderović et al. (ESORICS 2025) in communication complexity across all evaluated settings, and remains practical up to $(n, t) = (9, 4)$, a regime where prior approaches become bandwidth- or memory-prohibitive.

CCS Concepts

• Security and privacy → Cryptography.

Keywords

distributed OPRF, Gold PRF, distributed ZKP, Replicated Secret Sharing

ACM Reference Format:

Nan Cheng, Yohei Watanabe, Yugo Kasashima, Ioannis Katis, and Aikaterini Mitrokotsa. 2018. High-throughput Verifiable Distributed OPRF from Gold PRF. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 34 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Oblivious pseudorandom functions (OPRFs) [21] let a client compute $F_k(x)$ without revealing x to the key-holding server, and without learning k beyond what the output implies. They are a core building block of many privacy-preserving protocols.

OPRFs enable clients to transform low-entropy secrets, such as passwords, identifiers, or personal attributes, into high-entropy cryptographic material without revealing the underlying secret. This property has led to numerous applications, including password-authenticated key exchange (PAKE), password-protected secret sharing, private set intersection (PSI), single sign-on systems, and function secret sharing. OPRFs also serve as a mechanism for delegating computation to servers, allowing clients to remain lightweight while avoiding the storage of sensitive cryptographic state.

Distributed & Verifiable OPRFs. Relying on a single OPRF server represents a single point of trust and failure. This has motivated the study of *distributed OPRFs* (dOPRFs), in which the PRF key is secret-shared among multiple servers that jointly assist the client

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
<https://doi.org/XXXXXXX.XXXXXXX>

in evaluating $F_k(x)$. As long as a threshold of servers remains honest, the privacy of both the client input and the server-held key is preserved. Beyond secrecy, modern applications also require *verifiability*, ensuring that even malicious servers cannot deviate from the protocol and cause the client to accept an incorrect output. Thus, verifiable dOPRFs are a key building block for adversarial environments and large-scale deployments. Constructing verifiable dOPRFs that tolerate malicious adversaries while maintaining low communication and round complexity remains a significant challenge.

Post-Quantum Security & the Gold OPRF. Nearly all deployed OPRF constructions rely on Diffie–Hellman-type assumptions [26]. Although these schemes are efficient, they are vulnerable to quantum adversaries due to Shor’s algorithm. Thus, there is a growing need for OPRF constructions that offer post-quantum (PQ) security guarantees [1–3, 6, 8, 10, 17, 25, 32]. Although recent years have seen a surge of interest in PQ OPRFs, existing constructions either incur prohibitive computational and communication overheads, rely on semi-honest security assumptions, or do not extend naturally to distributed and verifiable settings. Closing the efficiency gap between classical and PQ-secure OPRFs remains an open problem.

A notable development is the Gold OPRF [34], based on the power-residue (Gold) pseudorandom function. Unlike classical PRFs, Gold naturally supports multi-bit outputs in a single invocation and admits efficient constant-round two-party evaluation based on vector oblivious linear evaluation (VOLE) correlations.

Prior Works & Their Limitations. Our construction follows the paradigm of [18, 24] that evaluates $(x + k)^g$ obliviously: it lets the client C learn $\alpha^e \cdot (x + k)$ for a uniformly random $\alpha \in \mathbb{F}_p^*$ that is *unknown* to C ; then C computes $(\alpha^e \cdot (x + k))^g$. This works because (1) $\alpha^{e \cdot g} \equiv 1 \pmod{p}$ for every $\alpha \in \mathbb{F}_p^*$, ensuring correctness, and (2) $\alpha^e \cdot (x + k)$ is uniformly distributed over the roots of $X^g = (x + k)^g$, which a simulator can reproduce by sampling such a root.

In the MPC context, the paradigm splits power-residue PRF evaluation into an *offline/online* decomposition (Fig. 1): servers precompute $[\alpha^e]$ with no client input in the offline phase. In the online phase, given $[x]$ from the client, they perform one secure multiplication $[\alpha^e] \cdot [x + k]$ and open $[\alpha^e(x + k)]$ to the client, who applies $(\cdot)^g$ locally. Kaluđerović et al. [28] followed this paradigm, they instantiate the technique for the Legendre PRF ($e = 2$) in the distributed setting, leveraging duplicated replicated secret sharing to run the online phase in only two rounds. However, their protocol tolerates only a *dishonest-minority* threshold $t < n/3$, outputs a single bit per invocation, scales unfavorably in total communication with (t, n) , and does not support Gold PRF ($e > 2$).

Yang et al. [34] target Gold PRF directly and achieve strong concrete efficiency in the client-server setting. However, their construction is a *two-party* OPRF that presupposes a VOLE correlation produced by an auxiliary setup, which is hard to obtain in practical deployments and does not generalize to multiple servers. Furthermore, their offline phase prepares α^e by *direct sequential squaring*: starting from a VOLE-committed α , they chain $\log_2 e = \lambda$ squarings and attach a batched line-point zero-knowledge proof to certify each step, incurring an $\tilde{O}(\lambda)$ per-evaluation overhead on top of the VOLE correlations, a cost that scales with the exponent e ; moreover,

each squaring would additionally require a 2PC secure multiplication. When λ is a relatively large integer, the *direct sequential squaring* could be a real efficiency bottleneck.

Challenges & Our Contribution. Existing constructions of distributed verifiable OPRFs fall into two camps, both unsatisfactory for deployment. Classical Diffie–Hellman-based dVOPRFs are concretely efficient but break under quantum adversaries; recent post-quantum candidates close the security gap but pay a heavy concrete cost because their PRF circuits demand many secret-shared multiplications, each of which costs a round of communication and a distributed zero-knowledge verification step. The compounded effect is that no published PQ dVOPRF runs at competitive throughput in WAN deployments under malicious-server adversaries.

We close this gap by exploiting an algebraic property of the recent Gold PRF [34]: its evaluation circuit C_{PRF} consists of a single addition and a single secret-shared multiplication, an unusually low multiplicative complexity that makes Gold uniquely well-matched to honest-majority MPC. Building on this property, we design an *outsourced MPC* scheme (Fig. 1) in which a committee of servers jointly evaluates Gold on behalf of the client, with the verifiability proof amortized into the same round as the multiplication itself. The result is the first concrete high-throughput PQ dVOPRF tolerating $t < n/2$ malicious servers.

Using this insight, we present the first *concrete, high-throughput, PQ-secure, verifiable distributed OPRF* tolerating $t < n/2$ malicious servers and a malicious client. Our construction builds on the Gold OPRF and follows an offline-online paradigm with new protocol designs in both phases.

In the offline phase, we develop efficient protocols for batched generation of secret-shared exponentiations $[\alpha^e]$, with costs independent of the security parameter. We present two approaches: (i) a degenerate additive encoding under which exponentiation commutes with secret sharing; (ii) a construction based on $t+1$ designated dealers that prove dual-share consistency via non-interactive zero-knowledge proofs, using VOLE-in-the-Head [7] and Ligerio [5]. *In the online phase*, we introduce a constant-round protocol that integrates secure multiplication with distributed zero-knowledge proof verification, substantially reducing communication compared to standard DZKP frameworks. We also propose an optimization that significantly decreases the communication sent to the client, further improving practicality in latency-sensitive settings.

On the client’s input, we observe that the usual way of handing a secret-shared input to a committee—the client generates the shares and distributes them—is not verifiable: nothing stops a malicious client from sending mismatched copies of a replicated share, leaving different servers evaluating on different points. We close this with Π_{Input} , a mask-and-broadcast input protocol whose consistency check the online protocol absorbs into a hash exchange it already performs, so it costs no extra round there. Because the client uploads one field element per server per input instead of a full replicated share, Π_{Input} is also *cheaper* than the direct sharing it replaces at every threshold we evaluate beyond $(3, 1)$, where it breaks even.

We implement all protocols in Rust and benchmark them on a simulated 100 ms / 100 Mbps WAN. End-to-end, our protocol suite consistently outperforms the naive PQ baseline $\Pi_{\text{DZKP}}^{\text{Aly}}$: at $(n, t) =$

(3, 1) we run $\approx 1.7\times$ faster on computation and reduce total communication by $\approx 2\times$, and at $(n, t) = (5, 2)$ we run $\approx 1.8\times$ faster. Compared to the closest existing post-quantum dOPRF $\Pi_{\text{L-dOPRF}}$ [28], we outperform it by $\approx 70\times$ on computation and over $7,400\times$ on communication at matched threshold (7, 2). The advantage widens further at larger thresholds and batches: at $(n, t) = (7, 3)$, $m=100$, our Liger-based variant $\Pi_{\text{dVOPRF}}^{\text{Lig}}$ runs $\approx 4.8\times$ faster than the naive baseline while reducing total communication by $\approx 7.5\times$.

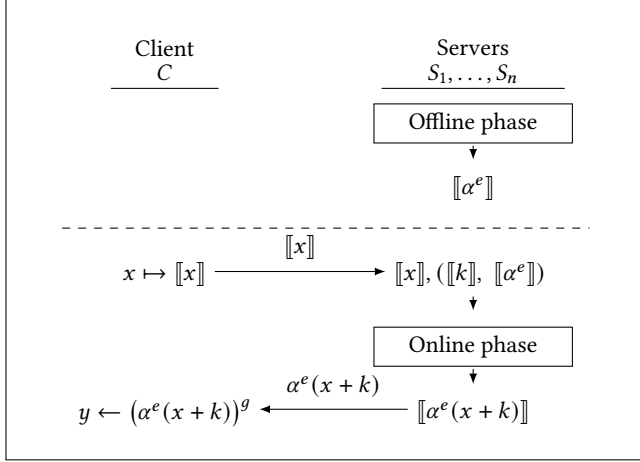


Figure 1: Offline/online Gold PRF $F_k(x) = (x + k)^g \bmod p$.

2 Preliminaries

Notations. Let $[a] := \{1, 2, \dots, a\}$ for any integer a . We denote by $x \xleftarrow{\$} S$ that x is sampled uniformly at random from a finite set S . Throughout, p denotes a prime, $\mathbb{F}_p$ the corresponding finite field, and $\mathbb{F}_p^*$ its multiplicative group. We let $\mathbb{G}$ denote the g -th power-residue subgroup of $\mathbb{F}_p^*$.

2.1 Secret Sharing Schemes

In a t -out-of- n secret sharing scheme, a secret value s is distributed across n shares, denoted by $[s] = \{[s]_1, \dots, [s]_n\}$, and can be reconstructed from any combination of at least $t + 1$ shares, while any t shares leak no information about s . For any shares $[s]$, we implicitly assume that party P_i is holding $[s]_i$, and computing $f([s])$ for any function f means that each party P_i computes $f([s]_i)$. We use the notation $[s]_{\mathcal{I}} := \{[s]_i\}_{i \in \mathcal{I}}$ for a subset of shares corresponding to $\mathcal{I} \subset [n]$. See Appendix D for the detailed definitions.

2.1.1 Local Linear Operations and Share Conversion. In this work, we consider secret sharing schemes that support local linear operations, such as $[a+b] := [a] + [b]$, $[c+a] := c + [a]$, and $[c \cdot a] := c \cdot [a]$. The *local share conversion* [14] allows parties to locally generate shares $[[s]_i]$ of the i -th share for any $i \in [n]$.

2.1.2 Additive Secret Sharing. Additive secret sharing (ASS) is an $(n - 1)$ -out-of- n secret sharing scheme supporting the local linear operations: a secret $s \in \mathbb{F}_p$ is divided into n random field elements, called *additive shares*, denoted by $[s]_1, \dots, [s]_n$. The secret s can be recovered by computing $s = \sum_{i=1}^n [s]_i$ over $\mathbb{F}_p$. A multiplicative variant $[s]^*$ can also be defined as $s = \prod_{i=1}^n [s]_i^*$ over $\mathbb{F}_p^*$.

Players: Servers $S_1, \dots, S_n$, client C , adversary $\mathcal{A}$ corrupting up to t servers and, possibly, C , and $\mathcal{F}_{\text{dVOPRF}}^k$.

Input: Each server S_i inputs its share $[k]_i$ of the PRF key k ; C inputs m evaluation points $(x^{(1)}, \dots, x^{(m)}) \in \mathbb{F}_p^m$. A corrupted C may submit arbitrary points of $\mathcal{A}$'s choice, or the abort signal $\perp$.

Computation: If C 's input is $\perp$, $\mathcal{F}_{\text{dVOPRF}}^k$ halts. Otherwise, it reconstructs k from the honest servers' shares $[k]_{\mathcal{H}}$ and sets $c^{(j)} \leftarrow F_k(x^{(j)})$ for every $j \in [m]$; delivers the corrupted parties' shares to $\mathcal{A}$, which replies with continue or abort.

Output: On continue, send $(c^{(1)}, \dots, c^{(m)})$ to C ; on abort, or if $\mathcal{F}_{\text{dVOPRF}}^k$ halted, send $\perp$ to C . Servers receive no output.

Figure 2: $\mathcal{F}_{\text{dVOPRF}}^k$ – dVOPRF for PRF F

2.1.3 Replicated Secret Sharing. Replicated secret sharing (RSS) generalizes ASS to handle an arbitrary threshold t by providing the same additive shares of s to multiple parties. Consider an ASS scheme that produces $N := \binom{n}{t}$ shares $[s]_1, \dots, [s]_N$ indexed by a set $\mathcal{T} := \{T \subset [n] \mid \#T = t\} = \{T_1, \dots, T_N\}$. A secret s is shared as *RSS shares* $[s] = \{[s]_1, \dots, [s]_N\}$, defined by $[s]_i := \{[s]_j \mid i \notin T_j \in \mathcal{T}\}$ for all $i \in [n]$. Therefore, each RSS share consists of $N' := \binom{n-1}{t-1}$ field elements. Any $t + 1$ RSS shares $\{[s]_{i_1}, \dots, [s]_{i_{t+1}}\}$ cover all additive shares, thereby recovering s , whereas any set of at most t RSS shares does not cover at least one additive share. RSS supports the local linear operations and the local share conversion.

A multiplicative RSS $[s]^*$ can also be defined by replacing the underlying additive shares $[s]$ with their multiplicative variant $[s]^*$.

2.2 Basic Ideal Functionalities

Our constructions invoke the standard ideal functionalities $\mathcal{F}_{\text{Rand}}$, $\mathcal{F}_{\text{Coin}}$, $\mathcal{F}_{\text{RSS.Share}}$, $\mathcal{F}_{\text{RSS.Open}}$, $\mathcal{F}_{\text{A2R}}$, $\mathcal{F}_{\text{RSS.Mul}}$, $\mathcal{F}_{\text{DoubleRand}}$, and $\mathcal{F}_{\text{Zero}}$; full definitions are deferred to Appendix E.

2.3 Distributed Zero-Knowledge Proofs

Distributed zero-knowledge proof (DZKP) systems (over secret sharing) provide proofs of a statement on an input that is distributed among several verifiers. In this work, we adapt the functionality $\mathcal{F}_{\text{provDeg2Rel}}^{\text{abort}}$ for proving degree-2 relations, which proves that given $([c], \{[a_j], [b_j]\}_{j \in L})$, the prover knows $(c, \{a_j, b_j\}_{j \in L})$ such that $c = \sum_{u=1}^L \sum_{v=1}^L a_u \cdot b_v$ (see Appendix F for the definition of $\mathcal{F}_{\text{provDeg2Rel}}^{\text{abort}}$).

3 Verifiable Distributed Oblivious PRF

This section formalizes the dVOPRF primitive informally introduced in §1. We write $F_k : \mathcal{D} \rightarrow \mathcal{R}$ for a pseudorandom function indexed by a key $k \xleftarrow{\$} \mathcal{K}$, and capture the dVOPRF setting [2, 21, 27, 28] as the ideal functionality $\mathcal{F}_{\text{dVOPRF}}^k$.

$\mathcal{F}_{\text{dVOPRF}}^k$ (Fig. 2) reconstructs k from the honest servers' RSS shares and returns $(F_k(x^{(j)}))_{j \in [m]}$ to the client, modulo a continue/abort decision by the adversary $\mathcal{A}$ that captures *security with abort* in the malicious honest-majority setting. **In this work, we focus on how to efficiently instantiate $\mathcal{F}_{\text{dVOPRF}}^k$.**

Note that $\mathcal{F}_{\text{dVOPRF}}^{F_k}$ accepts *any* $(x^{(1)}, \dots, x^{(m)}) \in \mathbb{F}_p^m$, and a corrupted C may submit points of $\mathcal{A}$'s choosing. This is deliberate: input *well-formedness* is an obligation of the protocol, not a restriction of the functionality. Guaranteeing that the servers end up holding a consistent sharing of a single, well-defined input—even when C is malicious—is exactly what Π_{Input} (§E.1) provides, and what lets us drop any honesty assumption on the client in Theorem 5.1.

3.1 PRF Candidate

We use *Gold PRF* [34], currently the most efficient post-quantum PRF in terms of the balanced trade-off between computation and communication, as the PRF in our protocol construction. Gold PRF is an instantiation of the *power-residue PRFs*, which work over a *power-residue subgroup* of $\mathbb{F}_p^*$, where p is a sufficiently large prime of $O(\lambda)$ bits such that $p := eg + 1$ for public integers e and g . Define the g -th *power-residue subgroup* $\mathbb{G} := \{a^g \bmod p \mid a \in \mathbb{F}_p^*\}$, which is the unique subgroup of $\mathbb{F}_p^*$ of order e . A power-residue PRF $F_k: \mathbb{F}_p \rightarrow \mathbb{G}$ is defined as $F_k(x) := (x + k)^g \bmod p$.¹ Gold PRF sets $e = 2^\lambda$ (so $|\mathbb{G}| = 2^\lambda$), and thus a single evaluation yields an output carrying λ bits of entropy. Yang et al. [34] analyzed the post-quantum security of Gold PRF, which is based on the underlying computational problem called the *Decisional Shifted Power-Residue Symbol* (DSPRS) problem. We refer interested readers to Appendix G for details on the power-residue and Gold PRFs.

3.2 High-level Overview of Our Solution

We present the first dVOPRF for Gold PRF that is simultaneously *multi-server*, *honest-majority* ($t < n/2$), *maliciously-secure*, and *practical*. Like prior constructions, our protocol follows the offline/online pipeline of Fig. 1: the offline phase realizes $\mathcal{F}_{\text{RandExp}}$ (Fig. 3); the online phase performs a verifiable multiply-and-open.

3.2.1 Secret Sharing Choice. We work over *replicated secret sharing* (RSS) rather than Shamir secret sharing because, in the honest-majority setting $t < n/2$ for small n , RSS offers two advantages in terms of concrete efficiency. *First*, RSS admits a more efficient semi-honest multiplication protocol than Shamir secret sharing, which our offline generators and online multiply-and-open exploit directly. *Second*, the fully-linear-PCP-based (FLPCP-based) DZKP of [9]—which we use to verify semi-honest multiplications in batch—applies to any linear secret sharing in principle, but matches RSS particularly well: a semi-honest RSS multiplication already produces an *additive* sharing of the product, which is precisely the witness shape the FLPCP requires. Over Shamir the same framework applies, but the degree- $2t$ output of a Shamir multiplication must first be reinterpolated back to degree t —adding an extra interactive round and $\Theta(n)$ communication per multiplication layer—or else verified through a degree- $2t$ FLPCP variant whose witness lives over $2t+1$ evaluation points rather than $t+1$ additive shares, roughly doubling proof size and verifier work. Either way, Shamir pays a concrete overhead per multiplication that RSS avoids.

¹Practical instantiations fix an injective encoding from bitstrings to $\mathbb{F}_p$ and explicitly handle the negligible-probability event $x + k \equiv 0 \pmod{p}$, in which $F_k(x) = 0 \notin \mathbb{G}$.

Players: Servers $S_1, \dots, S_n$, adversary $\mathcal{A}$ corrupting up to t servers, and $\mathcal{F}_{\text{RandExp}}$.

Input: A public exponent $e \in \mathbb{Z}^+$ from all servers.

Computation: $\mathcal{F}_{\text{RandExp}}$ samples $\alpha \xleftarrow{\$} \mathbb{F}_p$ and generates an RSS sharing $\llbracket \alpha^e \rrbracket$ of α^e ; delivers the corrupted parties' shares to $\mathcal{A}$, which replies with continue or abort.

Output: On continue, send $\llbracket \alpha^e \rrbracket_i$ to each honest S_i ; on abort, send $\perp$ to all honest servers.

Figure 3: $\mathcal{F}_{\text{RandExp}}$ – Random Power RSS Generation

The cost RSS pays for this efficiency is a per-party complexity of $N' = \binom{n-1}{t}$ shares—versus $O(1)$ for Shamir—which grows combinatorially in n and would indeed favor Shamir in the large- n regime. In practice, however, threshold deployments consistently operate in the small- (t, n) regime, e.g., $n \leq 9$, and in this regime the per-party share count stays small (at most $\binom{9}{4} = 126$), and the complexity of dealing with RSS's local shares remains modest.

3.2.2 Offline phase. We propose two novel methods for batched generation of $\llbracket \alpha^e \rrbracket$; both avoid the $O(\log e)$ sequential modular exponentiations and concretely outperform the state-of-the-art protocol of Aly et al. [4, 30] for computing $\llbracket \alpha^e \rrbracket \leftarrow \exp(\llbracket \alpha \rrbracket, e)$. Our first method works over what we call the *degenerate additive encoding*, which exists in RSS and enables efficient secure multiplication by converting locally computed multiplicative RSS to additive RSS at low cost. Our second approach reduces the multiplication count to t using an additional tailored post-quantum ZKP. We give two instantiations, Π_{VITH} (VOLE-in-the-Head) and Π_{Lig} (Ligero). Both are adapted to a non-standard multi-prover/multi-verifier setting in which $t+1$ servers prove dual-share consistency to verifiers that each hold only a partial witness—an interesting use case of ZKP in MPC. Together these give three instantiations; we provide all three precisely because no single one dominates across all deployment settings—the trade-offs among (t, n) , batch parameter m , bandwidth, and latency are intricate, and we lay them out precisely in §6.

3.2.3 Online phase. Given $(\llbracket k \rrbracket, \llbracket \alpha^e \rrbracket)$ and $\llbracket x \rrbracket$, servers compute $\llbracket \alpha^e(x+k) \rrbracket$ via a *verifiable inner product* protocol, introduced in this paper, that fuses the secret-shared multiplication with a batched DZKP optimized for the designated-verifier setting. The client reconstructs $\alpha^e(x+k)$ and locally raises it to g to recover $F_k(x)$. Our online protocol is concretely efficient in terms of communication and rounds. In particular, our protocol batches across both servers and parallel queries, and incurs $O(3L \log L \cdot (n-1))$ server-to-server and $O(n)$ server-to-client communication, where $L = N^2/n$.

3.2.4 Concrete performance. In a 100 ms / 100 Mbps WAN setting, our benchmarks show that all three of our novel offline protocols outperform the state-of-the-art baseline Π_{AlyGen} [4, 30] on both computation and communication across every threshold profile we test; the online-phase benchmark compares our novel Π_{dVOPRF}^m against the state-of-the-art naive baseline $\Pi_{\text{naive-dVOPRF}}^m$ [9], with up to $\approx 30\times$ less communication; **[Implementation rework pending: benchmark-derived, predates Π_{Input}]** and combining the two

phases, our end-to-end benchmarks demonstrate consistent dominance over the composed naive baseline $\Pi_{\text{DZKP}}^{\text{Aly}}$. Specifically, at the most commonly deployed $(n, t) = (3, 1)$, our $\Pi_{\text{dVOPRF}}^{\text{DegGen}}$ runs end-to-end in ≈ 1.91 s versus 3.23 s for the baseline while incurring $\approx 2\times$ less total communication; at the modest $(n, t) = (5, 2)$, our $\Pi_{\text{dVOPRF}}^{\text{VitH}}$ completes in ≈ 2.07 s versus 3.72 s. Against the closest external dOPRF $\Pi_{\text{L-dOPRF}}$ [28], at matched threshold $(7, 2)$ we outperform it by $\approx 70\times$ on computation time and by over $7400\times$ on communication. Taken together, our design yields the first high-throughput post-quantum dVOPRF for Gold PRF in the honest-majority malicious setting under small-to-modest (n, t) threshold deployments, making it a strong candidate for production use today and for boosting the efficiency of higher-level cryptographic primitives that build on OPRFs.

4 Offline Phase

The offline phase prepares $\llbracket \alpha^e \rrbracket$ for $\alpha \xleftarrow{\$} \mathbb{F}_p$, secure against $t < n/2$ malicious servers. Since every online dVOPRF evaluation consumes one fresh instance of $\llbracket \alpha^e \rrbracket$, achieving high throughput hinges on generating this correlated randomness in bulk efficiently.

State-of-the-art over general additive secret sharing. Given a secret base $[\alpha]$ and a public exponent e , Aly et al.’s protocol [4, Protocol 7], stated over $\mathcal{F}_{\text{ABB}}$, is the state-of-the-art for generating $[\alpha^e]$: it replaces $\log e$ sequential multiplications by two public-base exponentiations, one secure multiplication, and one public opening, via $[\bar{r}] \leftarrow \exp(g, [r'])$, $[c] \leftarrow [\bar{r}] \cdot [\alpha]$, $c \leftarrow \text{Open}([c])$, $[\alpha^e] \leftarrow c \cdot \exp(g, -e[r'])$. Round and communication complexity are thus independent of the public exponent, dominated by two calls to $\Pi_{\exp(b, [\cdot])}$ [4, Protocol 6]. Mennink [30] re-analysed these protocols and reported the malicious cost as 10 rounds, $4n+11$ multiplications, and $8n+19$ openings per invocation – the baseline for additively-shared instantiations under $\mathcal{F}_{\text{ABB}}$.

Our RSS adaptation Π_{AlyGen} . We port Aly et al.’s protocol to the honest-majority RSS setting and refer to the resulting protocol as Π_{AlyGen} ; the detailed instantiation is deferred to Protocol 13 in Appendix H. The high-level structure is unchanged compared to [4, Protocol 7], but two RSS-specific factors increase communication complexity. First, exponentiation by a public scalar is *local* in RSS, but each public-base exponentiation $[\bar{r}] \leftarrow \exp(g, [r'])$ and $[\rho] \leftarrow \exp(g, [\bar{e}])$ produces $N = \binom{n}{t}$ locally-computed field elements (one per replicated component of the secret exponent) that must be folded into a standard RSS sharing via $N-1$ sequential $\Pi_{\text{RSS.Mul}}$ steps. Second, the malicious upgrade is realised by a single batched $\mathcal{F}_{\text{provDeg2Rel}}^{\text{abort}}$ DZKP over all multiplications, deferred to the end of the protocol. A single invocation of Π_{AlyGen} therefore costs $2(N-1)+1 = 2N-1$ $\Pi_{\text{RSS.Mul}}$ invocations, 1 public opening, 1 batched DZKP, and a per-generation $\mathbb{F}_p \leftrightarrow \mathbb{Z}_{p-1}$ share conversion. In the RSS setting, Π_{AlyGen} thus presents two structural disadvantages: (i) the multiplication count is $O(N)$ rather than $O(n)$; and (ii) it requires a per-instance $\mathbb{F}_p \leftrightarrow \mathbb{Z}_{p-1}$ share conversion, which further complicates protocol instantiation.

To overcome these disadvantages, we present two novel protocols for generating $\llbracket \alpha^e \rrbracket$, denoted Π_{DegGen} and Π_{ZKPGen} , both tailored to the RSS structure: they eliminate the share conversion

and avoid high communication volume. Π_{ZKPGen} contains two concrete ZKP-dependent instantiations: $\Pi_{\text{ZKPGen}}^{\text{VitH}}$ (VOLEitH-based) and $\Pi_{\text{ZKPGen}}^{\text{Lig}}$ (Ligero-based). Tab. 1 summarises the resulting three constructions in terms of primitives, security assumption, and asymptotic cost.

Table 1: Offline Phase Protocol Overview

Protocol	Primitives	Sec.	#Mul.*	Extra Comm.†
Π_{AlyGen} [4, 30]	$\Pi_{\exp(b, [\cdot])} + \text{DZKP}$	IT	$2N-1$	$O(1)$
Our Π_{DegGen}	DegMul + DZKP	IT	$(N-1)^{\ddagger}$	0
Our $\Pi_{\text{ZKPGen}}^{\text{VitH}}$	RSS.Mul + VitH + DZKP	CRH	t	$O(t\kappa N)$
Our $\Pi_{\text{ZKPGen}}^{\text{Lig}}$	RSS.Mul + Lig + DZKP	CRH	t	$O(t\kappa \log N)$

$n \geq 2t+1$, $N = \binom{n}{t}$, κ : security parameter. IT: information-theoretic; CRH: collision-resistant hashing. *Number of $\Pi_{\text{RSS.Mul}}$ or Π_{DegMul} invocations. ‡Counts Π_{DegMul} invocations. †Extra communication beyond multiplication overhead; Π_{AlyGen} additionally requires $\mathbb{F}_p \leftrightarrow \mathbb{Z}_{p-1}$ share conversion.

In brief, Π_{DegGen} roughly halves the multiplication count of Π_{AlyGen} and removes its opening and $\mathbb{F}_p \leftrightarrow \mathbb{Z}_{p-1}$ conversion while remaining IT secure; Π_{ZKPGen} further drops the count to t and gains post-quantum security from collision-resistant hashing, at the cost of extra ZKP communication. We present Π_{DegGen} and Π_{ZKPGen} in turn below, with concrete benchmarks deferred to §6.1.

4.1 Protocol Π_{DegGen}

Rather than computing $\llbracket \alpha^e \rrbracket$ from an additive sharing $[\alpha]$, we start from a multiplicative secret sharing $[\alpha]^*$. We observe that two structural properties of $[\alpha]^*$ make this advantageous:

- (1) **Commutativity with exponentiation.** If $\alpha = \prod_{T \in \mathcal{T}} m_T$ is the secret reconstructed from the multiplicative sharing $[\alpha]^*$, then

$$\alpha^e = \left(\prod_{T \in \mathcal{T}} m_T \right)^e = \prod_{T \in \mathcal{T}} m_T^e.$$

Each server can therefore *locally* raise its multiplicative shares to the power e , obtaining a multiplicative sharing $[\alpha^e]^*$ without any interaction.

- (2) **Degenerate structure.** Each component m_T of $[\alpha]^*$ is held by exactly the $n-t$ servers in $[n] \setminus T$, forming a special type of additive replicated secret sharing that we call a *degenerate additive encoding* (Definition 1).

It remains to convert $[\alpha^e]^*$ into its standard additive form $\llbracket \alpha^e \rrbracket$. We accomplish this by sequentially multiplying all N degenerate encodings via a dedicated protocol Π_{DegMul} , and then verifying all multiplication results in a single batched invocation of $\mathcal{F}_{\text{provDeg2Rel}}$. Below, we first define the degenerate additive encoding and the multiplication protocol, and lastly present the complete protocol Π_{DegGen} .

DEFINITION 1 (DEGENERATE ADDITIVE ENCODING). Let $v \in \mathbb{F}_p$ and $T \in \mathcal{T}$. We say that a standard RSS $\llbracket v \rrbracket$ is a degenerate additive encoding with respect to T , denoted $\langle v \rangle_T$, if its components $\{s_{T'}\}_{T' \in \mathcal{T}}$ satisfy

$$s_{T'} = \begin{cases} v & \text{if } T' = T, \\ 0 & \text{otherwise.} \end{cases}$$

With parties holding $\langle v \rangle_T$, under the standard RSS distribution rule, the $n - t$ parties in $[n] \setminus T$ all hold the identical value v , while the remaining t parties in T hold only zeros. Hence, we observe that any standard RSS $\llbracket s \rrbracket$ with components $\{s_T\}_{T \in \mathcal{T}}$ is composed of $N = |\mathcal{T}| = \binom{n}{t}$ degenerate additive encodings, i.e., $\{\langle s_T \rangle_T\}_{T \in \mathcal{T}}$.

4.1.1 Multiplication of standard RSS by degenerate encoding. Π_{DegMul} (Protocol 1) realizes secure multiplication of a standard RSS $\llbracket a \rrbracket$ by a degenerate additive encoding $\langle v \rangle_{T'}$. It follows the same structure as the DN protocol—masking the cross-product with correlated randomness and publicly reconstructing the masked difference—but exploits the sparse structure of degenerate encodings to reduce both computation and communication. As in the DN protocol, the required correlated randomness (a dual sharing $([r], \llbracket r \rrbracket)$) is generated by an adapted protocol $\Pi_{\text{DoubleRand}}^{\text{deg}}$ (Protocol 14 in Appendix I), which restricts the additive share $[r]$ to the target subset $\mathcal{A} = [n] \setminus T'$ (with $[r]_i = 0$ for $i \in T'$) so that only the $t + 1$ parties in $\mathcal{A}$ contribute to reconstructing r , in contrast to the standard $\Pi_{\text{DoubleRand}}$ which involves all n parties.

This sparsity carries through to the multiplication itself. On the computation side, only parties in $\mathcal{A}$ hold the non-zero component of $\langle v \rangle_{T'}$, so the cross-multiplication involves $|\mathcal{T}| + 1$ non-zero terms instead of $|\mathcal{T}|^2$ in general RSS multiplication, while the t parties in T' hold zero shares and skip cross-multiplication entirely. On the communication side, the target-matched double sharing ensures that parties in T' have $d_i = 0$ and remain silent: only the $t + 1$ parties in $\mathcal{A}$ communicate, each sending one field element to S_{agg} , which broadcasts δ back to $\mathcal{A}$, with the output remaining a standard RSS.

Protocol 1 Π_{DegMul} – Deg. Encoding Multiplication

Players: Servers $S_1, \dots, S_n$.

Input: $\llbracket a \rrbracket$; degenerate encoding $\langle v \rangle_{T'}$ for some $T' \in \mathcal{T}$; target subset $\mathcal{A} = [n] \setminus T'$ with a designated aggregator $S_{\text{agg}} \in \mathcal{A}$; target-matched double sharing $([r], \llbracket r \rrbracket)$ from $\Pi_{\text{DoubleRand}}^{\text{deg}}$ (Protocol 14) with target $\mathcal{A}$.

- 1: **foreach** $S_i \in \mathcal{A}$ **do**
- 2: Cross-multiply: $[av]_i \leftarrow \llbracket a \rrbracket_i \cdot \langle v \rangle_{T'}$.
- 3: Send $d_i \leftarrow [av]_i - [r]_i$ to S_{agg} .
- 4: **end**
- 5: S_{agg} reconstructs $\delta \leftarrow \sum_{i \in \mathcal{A}} d_i = av - r$ and broadcasts δ to $\mathcal{A}$, so that each server in $\mathcal{A}$ computes $\llbracket av \rrbracket = \delta + \llbracket r \rrbracket$.

Output: $\llbracket av \rrbracket$.

4.1.2 Full Protocol. We combine all sub-protocols into the full generation protocol Π_{DegGen} , which proceeds in three steps. First, in a *local exponentiation* step, each server S_i raises its multiplicative shares m_T to the power e , obtaining $M_T = m_T^e$ for every T with $i \notin T$; since $\prod_{T \in \mathcal{T}} M_T = \alpha^e$, this yields an RSS multiplicative sharing of α^e , requiring each server to derive $\binom{n-1}{t}$ shares from PRG seeds and compute $\binom{n-1}{t}$ modular exponentiations. Next, in the *sequential multiplication* step, servers convert the multiplicative sharing into an additive RSS by sequentially accumulating the degenerate encodings $\langle M_{T_j} \rangle_{T_j}$ via Π_{DegMul} : fixing an ordering $T_1, \dots, T_N$ of $\mathcal{T}$, let

$$\llbracket q_1 \rrbracket = \langle M_{T_1} \rangle_{T_1}, \quad \llbracket q_j \rrbracket = \llbracket q_{j-1} \rrbracket + \langle M_{T_j} \rangle_{T_j} \text{ for } 2 \leq j \leq N,$$

so that after $N - 1$ multiplications, $\llbracket q_N \rrbracket = \llbracket \alpha^e \rrbracket$. In each invocation, only the $t + 1$ parties in $\mathcal{A}_j$ participate, each sending 1 field element to S_{agg} , which broadcasts δ back, for $2(t + 1)$ field elements per multiplication. Since the cross-multiplication involves $|\mathcal{T}| + 1$ non-zero terms instead of $|\mathcal{T}|^2$ as in general RSS multiplication, the total communication over the $N - 1$ rounds is $2(t + 1)(N - 1)$ field elements. Finally, in the *DZKP verification* step, all $N - 1$ multiplication relations are verified in a single batched invocation of $\mathcal{F}_{\text{provDeg2Rel}}^{\text{abort}}$ with sublinear communication complexity $O(n \log N)$.

Protocol 2 Π_{DegGen} – Offline Generation via Π_{DegMul}

Players: Servers $S_1, \dots, S_n$.

Input: Public exponent e ; multiplicative RSS $\llbracket \alpha \rrbracket^* = \{m_T\}_{T \in \mathcal{T}}$ with $\alpha = \prod_{T \in \mathcal{T}} m_T$; ordering $T_1, \dots, T_N$ of $\mathcal{T}$.

- 1: **foreach** $S_i \in [n]$, $T \in \mathcal{T}$ with $i \notin T$ **do**
- 2: $M_T \leftarrow m_T^e$; form $\langle M_T \rangle_T$.
- 3: **end**
- 4: Set $\llbracket q_1 \rrbracket \leftarrow \langle M_{T_1} \rangle_{T_1}$.
- 5: **for** $j = 2, \dots, N$ **do**
- 6: Let $\mathcal{A}_j \leftarrow [n] \setminus T_j$ and $([r], \llbracket r \rrbracket) \leftarrow \Pi_{\text{DoubleRand}}^{\text{deg}}(\mathcal{A}_j)$.
- 7: $\llbracket q_j \rrbracket \leftarrow \Pi_{\text{DegMul}}(\llbracket q_{j-1} \rrbracket, \langle M_{T_j} \rangle_{T_j}; ([r], \llbracket r \rrbracket))$.
- 8: **end**
- 9: Verify $\{q_j = q_{j-1} \cdot M_{T_j}\}_{j=2}^N$ via $\mathcal{F}_{\text{provDeg2Rel}}^{\text{abort}}$; abort on failure.

Output: $\llbracket q_N \rrbracket$ as $\llbracket \alpha^e \rrbracket$.

4.2 Protocol Π_{ZKPGen}

Π_{DegGen} requires $N - 1$ invocations of Π_{DegMul} . Here we present Π_{ZKPGen} (Protocol 3) that reduces the number of multiplication invocations to t , at some additional ZKP costs.

The idea is to select $t + 1$ designated dealers $\mathcal{A} \subset [n]$. Each dealer $S_i \in \mathcal{A}$ distributes an additive secret sharing $\llbracket M^i \rrbracket$, where M^i admits an e -th root in $\mathbb{F}_p$. The servers then accumulate the product $\llbracket M \rrbracket = \llbracket \prod_{i \in \mathcal{A}} M^i \rrbracket$ using t invocations of $\Pi_{\text{RSS.Mul}}$, thus reducing the multiplication invocations from $N - 1$ to t . The challenge in this alternative is to prove that each M^i indeed admits an e -th root in $\mathbb{F}_p$; we address this via a post-quantum dual-share consistency ZKP, for which we provide two instantiations: Π_{Vith} (Protocols 15–16 in Appendix J.2), based on VOLE-in-the-Head with communication complexity $O(t\kappa N)$, and Π_{Lig} (Protocols 17–18 in Appendix J.3), based on the Liger ZKP system with communication complexity $O(t\kappa \log N)$. Both require only collision-resistant hashing and no trusted setup. Concretely, Π_{ZKPGen} works as follows:

- (1) Each of $t + 1$ designated servers $S_i \in \mathcal{A}$ uses PRG seeds shared with every other server to locally derive multiplicative shares $\{m_j\}$ and additive shares $\{a_j\}$ with $A^i = \sum_j a_j$. Each server then locally raises its multiplicative shares to the e -th power, yielding $M^i = \prod_j (m_j)^e$, which is an e -th power by construction.
- (2) The designated server S_i reconstructs both M^i and A^i , computes a correction $\delta \leftarrow M^i - A^i$, and broadcasts δ to all servers. Accordingly, each server updates their local share of M^i .
- (3) S_i invokes $\Pi_{\text{DualShareZKP}}$ to prove that δ was correctly computed, i.e., that the additive sharing is consistent with the multiplicative shares and that M^i is indeed an e -th power.

- (4) After all $t+1$ designated servers have contributed and proven their factors, servers multiply the $t+1$ additive sharings $\llbracket M^1 \rrbracket, \dots, \llbracket M^{t+1} \rrbracket$ using t invocations of $\Pi_{\text{RSS}, \text{Mul}}$, yielding $\llbracket \alpha^e \rrbracket$ where $\alpha^e = \prod_{i \in \mathcal{A}} M^i$.

Only $t+1$ parties need to contribute a factor, since at least one honest party's contribution suffices to randomize the product; with $t+1$ factors, even if t parties are corrupt, the product remains uniformly random from the adversary's perspective.

The remaining challenge is how to efficiently realize $\Pi_{\text{DualShareZKP}}$, which operates under a non-standard ZKP setting: multiple distributed verifiers each hold a *subset* of the prover's witness (the PRG-derived shares). We present two concrete instantiations of $\Pi_{\text{DualShareZKP}}$ —one based on VOLE-in-the-Head and one based on Liger, both requiring only collision-resistant hashing (CRH) and no trusted setup. Below, we first define the extended witness and the relation circuit that both instantiations prove, then summarize the key ideas and properties of each protocol (full protocol descriptions are deferred to Appendices J.2 and J.3).

Protocol 3 Π_{ZKPGen} – Offline Generation of $\llbracket \alpha^e \rrbracket$

Players: Servers $S_1, \dots, S_n$.

Input: A designated dealer set $\mathcal{A} \subset [n]$ with $|\mathcal{A}| = t+1$; pairwise PRG seeds $\{\text{seed}_{i,j}\}$ shared between each pair of servers.

```

1: for  $i \in \mathcal{A}$  do
2:   Let  $\mathcal{P} = S_i$  be the current dealer.
   // Dual-share derivation
3:   From the PRG seeds shared between  $\mathcal{P}$  and each other
   server, all servers locally derive pseudorandom values
    $\{m_T\}_{T \in \mathcal{T}}$  and  $\{a_T\}_{T \in \mathcal{T}}$  over  $\mathbb{F}_p$ .
   These implicitly define  $M^i \leftarrow \prod_{T \in \mathcal{T}} (m_T)^e$  and
    $A^i \leftarrow \sum_{T \in \mathcal{T}} a_T$ , both uniform in  $\mathbb{F}_p$ .
   // Correction broadcast
4:    $\mathcal{P}$  locally reconstructs  $M^i$  and  $A^i$ , computes  $\delta \leftarrow M^i - A^i$ ,
   and broadcasts  $\delta$  to all servers.
   // Share conversion
5:   Each server  $S_j$  updates  $\llbracket M^i \rrbracket_j \leftarrow a_j + \delta \cdot \mathbf{1}_{[S_j = \mathcal{P}]}$ .
   // Consistency proof
6:    $\mathcal{P}$  invokes  $\Pi_{\text{DualShareZKP}}(\delta, \{a_T, m_T\}_{T \in \mathcal{T}})$  to prove that
    $\delta = M^i - A^i = \prod_T (m_T)^e - \sum_T a_T$ ; all other servers verify
   the proof and abort on failure.
7: end
   // Accumulation via multiplication tree
8:   Order  $\mathcal{A}$  as  $i_1, \dots, i_{t+1}$ , set  $\llbracket q_1 \rrbracket \leftarrow \llbracket M^{i_1} \rrbracket$ , and for  $j = 2, \dots, t+1$ 
   compute  $\llbracket q_j \rrbracket \leftarrow \Pi_{\text{RSS}, \text{Mul}}(\llbracket q_{j-1} \rrbracket, \llbracket M^{i_j} \rrbracket)$ , yielding  $\llbracket \alpha^e \rrbracket = \llbracket q_{t+1} \rrbracket$  after  $t$  invocations.
   // Multiplication verification
9:   Verify  $\{q_j = q_{j-1} \cdot M^{i_j}\}_{j=2}^{t+1}$  via  $\mathcal{F}_{\text{provDeg2Rel}}^{\text{abort}}$ ; abort on failure.
```

Output: Each server S_j holds $\llbracket \alpha^e \rrbracket_j$, where $\alpha^e = \prod_{i \in \mathcal{A}} M^i$.

4.2.1 The Dual-Share Circuit C_{dual} . Both ZKP instantiations prove correctness of the same arithmetic circuit C_{dual} over $\mathbb{F}_p$ that, on extended witness $\mathbf{w} = (M_1, a_1, \dots, M_N, a_N, w_2, \dots, w_N) \in \mathbb{F}_p^L$ with $L = 3N-1$, checks the chain $w_{t+1} = w_t \cdot M_{t+1}$ together with the output relation $w_N - \sum_{T \in \mathcal{T}} a_T - \delta = 0$. The full circuit construction (input wires, $N-1$ multiplication gates, linear addition gates, and the extended witness) is given in Appendix J.1.

4.2.2 Π_{VITH} : VOLEitH-based Dual-Share ZKP. We first present an instantiation based on VOLE-in-the-Head (VOLEitH) [7]. VOLEitH achieves post-quantum security from symmetric-key primitives alone (collision-resistant hashing and PRGs), requires no trusted setup, and proves the relation $\delta = \prod_T M_T - \sum_T a_T$. Its proof sizes scale as $O(M \log |\mathbb{F}| + \kappa^2 / \log \tau)$ (where $M = N-1$ is the number of multiplication gates and τ is a tree-arity parameter), making it particularly efficient for small-to-moderate N . At a high level, VOLEitH combines three ingredients:

• **QuickSilver commit–challenge–verify pipeline** [33].

The prover holds the extended witness $\mathbf{w}$ and a VOLE correlation: for every wire j , values (u_j, v_j) satisfy $q_j = u_j \cdot \Delta + v_j$ for a global secret verifier key Δ . The prover publishes masked wire values $\tilde{w}_j = w_j + u_j$. The verifier checks each multiplication gate by computing a degree-2 expression in Δ from the VOLE-committed wires; when the gate is satisfied, the Δ^2 terms cancel and the check reduces to a linear identity (see Appendix J.2 for the full derivation). All M gate checks are batched via a random Fiat–Shamir challenge into a single verification equation.

• **Non-interactive VOLE from a GGM tree.** Rather than running an interactive OT-based VOLE between the prover and the verifier, VOLEitH [7] generates the correlation locally. A GGM tree of arity τ expands a single root seed into τ leaf seeds via a length-doubling PRG; each leaf i determines a virtual party P_i 's random shares $\{s_j^{(i)}\}$ for all wires j , with $u_j = \sum_i s_j^{(i)}$ and $v_j = -\sum_i i \cdot s_j^{(i)}$. The prover commits to the GGM tree (Merkle hash of the leaf nodes), then after a Fiat–Shamir challenge $\Delta \xleftarrow{\$} [\tau]$, opens every leaf *except* leaf Δ by revealing the $\lceil \log_2 \tau \rceil$ GGM sibling seeds along the path to Δ . The verifier reconstructs all sd_i for $i \neq \Delta$, expands them to shares, and computes the VOLE tags $q_j = \sum_{i \neq \Delta} s_j^{(i)} \cdot (\Delta - i)$. Since Δ is uniformly sampled from a set of only τ elements, the soundness of a single instance is limited to $1/\tau$.

• **Parallel repetitions for soundness amplification.** To achieve κ -bit soundness, the protocol runs $R = \lceil \kappa / \log_2 \tau \rceil$ independent repetitions, each with its own GGM tree, fresh VOLE masking, and separate QuickSilver check. Over a large prime field $\mathbb{F}_p$ (with $p \gg \tau$), the shares $s_j^{(i)}$ live directly in $\mathbb{F}_p$, so each repetition produces a full VOLE over $\mathbb{F}_p$ —no extension-field or subspace-VOLE machinery is needed. The resulting R independent GGM trees can be merged into a single *Batch All-But-One Vector Commitment* (BAVC) to reduce commitment overhead [7].

Input binding via two-level GGM commitment (our contribution).

The standard two-party VOLEitH proof by itself establishes only that *some* witness satisfies C_{dual} ; it does not bind the witness to the partial witness held by a verifier as in our setting. To close this gap, we use a *two-level Merkle commitment* for the GGM tree. In the standard VOLEitH construction, each leaf commitment is simply $h_i = H(\text{sd}_i)$, so rt commits only to the leaf *seeds*, not to the share vectors expanded from them. After all-but-one opening, the verifier uses the co-path sibling hashes to check each opened h_i against rt , but since h_Δ commits only to the seed sd_Δ —not to its expanded shares—the verifier cannot verify any individual share

$s_k^{(\Delta)}$ against h_Δ —precisely the check that our distributed-verifier setting requires.

Our variant instead builds, for each GGM leaf $i \in [\tau]$, a *sub-Merkle-tree* over the expanded share vector $s^{(i)} = (s_0^{(i)}, \dots, s_L^{(i)})$ derived from sd_i , giving $h_i = \text{MT}(s^{(i)})$. The outer Merkle tree over $\{h_i\}_{i \in [\tau]}$ yields the global commitment rt . After the Fiat–Shamir challenge selects Δ , the prover opens all leaves except leaf Δ via the co-path. For the hidden leaf Δ , each verifier S_j recovers individual shares at its known input-wire positions k (those with $j \in \bar{T}_k$) as follows:

- (1) Compute the VOLE mask $u_k = \tilde{w}_k - w_k^{\text{local}}$, using the masked value $\tilde{w}_k$ from the proof and the locally held share w_k^{local} .
- (2) Recover the hidden leaf’s share: $s_k^{(\Delta)} = u_k - \sum_{i \neq \Delta} s_k^{(i)}$, where the sum is over all opened leaves.
- (3) Verify $s_k^{(\Delta)}$ against the sub-Merkle-tree commitment h_Δ using the authentication path provided by the prover.

Since $|\bar{T}_k| = t+1 \geq 2$, at least one honest verifier checks each input wire. Because the commitment rt is included in the Fiat–Shamir hash, any alteration of input-wire shares changes the challenge (χ, Δ) and invalidates the entire proof. This two-level structure increases the prover’s hashing cost from $O(\tau)$ to $O(\tau L)$, but the per-verifier Merkle cost is only $O(L + \log \tau)$ rather than $O(\tau L)$, keeping verification efficient. A single invocation of Π_{Vith} has broadcast cost $O(\kappa N)$ field elements and per-verifier P2P cost $O(\kappa \log N)$ hashes, so the $t+1$ invocations inside Π_{ZKPGen} total $O(t\kappa N)$ broadcast and $O(t\kappa \log N)$ per-verifier P2P, entirely from symmetric-key primitives. The full prove and verify protocols $\Pi_{\text{Vith.Prove}}$ and $\Pi_{\text{Vith.Verify}}$, together with the detailed proof-size and complexity breakdown, are given in Appendix J.2.

4.2.3 Π_{Lig} : Liger-based Dual-Share ZKP. As an alternative to the VOLEitH-based proof above, we present a second instantiation of $\Pi_{\text{DualShareZKP}}$ based on Liger [5]. Like Π_{Vith} , it achieves post-quantum security from collision-resistant hashing alone, with no trusted setup, and proves the relation $\delta = \prod_T M_T - \sum_T a_T$ over C_{dual} . A key advantage of the Liger approach is its natural support for *multi-instance batching*: a single proof can simultaneously establish $B \geq 1$ independent dual-share consistency relations $\delta^{(b)} = \prod_k m_k^{(b)} - \sum_k a_k^{(b)}$ for $b = 1, \dots, B$, by stacking the per-instance witness rows into a single matrix. The Merkle commitment, interleaved consistency codeword, and column sampling are shared across all instances, so the amortised per-instance cost decreases with scaling B . At a high level, Liger proceeds in four stages:

(1) *Encode and commit.* For each instance $b \in [B]$, the prover computes the running products and sums from $(m_k^{(b)}, a_k^{(b)})_{k=1}^N$. All B instances are arranged into a single witness matrix W (with $4B$ rows) and encoded row-by-row via a Reed–Solomon (RS) code, extending N columns to $n_c \gg N$ codeword columns. The columns of the extended matrix $\hat{W}$ are Merkle-hashed into a single root $rt_{\hat{W}}$.

(2) *Compose and commit the consistency codeword.* All $5B$ constraints (five per instance) are batched into a single composition polynomial $C(x) = \mathcal{N}(x)/Z_W(x)$ using a Fiat–Shamir challenge, RS-encoded, and Merkle-committed. An *interleaved consistency codeword* u (a random linear combination of all rows) is committed before the column-sampling challenge is derived.

(3) *Column sampling: constraint and proximity checks.* The verifier samples random column pairs from the extension part and checks that the composition polynomial evaluations are consistent (constraint verification) and that rows are valid RS codewords (proximity test via interleaved consistency).

(4) *Partial opening of witness positions (our contribution).* Each server S_j already holds a subset of the original shares for every instance. We design a variant of Merkle commitment using a two-level column hash that separates *share rows* from *private rows* (running products, sums, blinding values), allowing each verifier to check the committed shares at positions it knows against $rt_{\hat{W}}$ without seeing the computation rows.

Stages 1–3 together guarantee soundness, with zero knowledge secured within stage 3: the column sampling is restricted to the extension columns and the witness rows are padded with random blinding entries, so the opened views reveal nothing about the underlying shares. Stage 4 then binds the proof to the actual shares held by the distributed verifiers. Parameter sizing (including the regime where Π_{Lig} becomes practical), the witness encoding, constraint verification via quotient polynomials, prove/verify protocols, and complexity analysis are deferred to Appendix J.3.

5 Our dVOPRF Protocol over Gold PRF

In the online phase, we assume servers hold $\llbracket k \rrbracket$ and $\llbracket \alpha^e \rrbracket$, obtained from $\mathcal{F}_{\text{RandExp}}$, and take in $\llbracket x \rrbracket$ from the client C . The goal is for C to receive $\alpha^e(x + k)$ in the presence of $t < n/2$ corrupted servers.

Naive Idea. A straightforward way to compute $\llbracket ab \rrbracket$ is to employ a semi-honest secure multiplication protocol, such as the DN protocol [16], together with Boyle et al.’s DZKP protocol that proves the correctness of multiplication [11]; we denote this baseline by $\Pi_{\text{naive-dVOPRF}}^m$. However, it requires 11 rounds and blows up the communication cost as the number of client inputs increases. We propose a more efficient solution.

Our Approach and Challenges. Consider two additive sharings of $a = \sum_{j=1}^N [a]_j$ and $b = \sum_{j=1}^N [b]_j$, where $N = \binom{n}{t}$. Expanding the product gives $ab = ([a]_1 + \dots + [a]_N)([b]_1 + \dots + [b]_N) = \sum_{u=1}^N \sum_{v=1}^N [a]_u \cdot [b]_v$. Another straightforward way to compute $\llbracket ab \rrbracket$ would be to convert each additive component, $[a]_u, [b]_v$, into RSS form and then invoke an interactive secure multiplication protocol [16, 23] to compute RSS shares of each term $[a]_u \cdot [b]_v$ of ab . However, this approach requires N^2 multiplications and is obviously extremely inefficient. Instead, we partition the N^2 product terms into n disjoint collections, one per server, so that each server can evaluate its assigned terms *locally* from its RSS view. Each server S_i computes a local partial sum c_i of the form $c_i = \sum_{(u,v) \in \Lambda_i} [a]_u \cdot [b]_v$ for an assignment $\{\Lambda_i\}_{i \in [n]}$ that we show in the next section, and then reshapes c_i into an RSS sharing. The desired RSS shares $\llbracket ab \rrbracket$ can be locally computed by $\llbracket c_1 \rrbracket + \dots + \llbracket c_n \rrbracket$. This eliminates interactive multiplication altogether, while still producing the correct multiplication result in shared form. Therefore, the challenges for this approach are: (1) *how we assign RSS shares to servers accordingly*; and (2) *how we efficiently make a protocol secure against up to $t/2$ malicious servers*.

5.1 Share Assignments

Our goal is to obtain an additive sharing of the product ab using local multiplications only, i.e., without interactive multiplication protocols. In this section, we propose a (balanced) share-assignment protocol for RSS multiplications that partitions the N^2 product terms among n servers.

By definition of RSS, server S_i holds $\llbracket a \rrbracket_i = \{[a]_u \mid i \notin T_u \in \mathcal{T}\}$ and $\llbracket b \rrbracket_i = \{[b]_v \mid i \notin T_v \in \mathcal{T}\}$. Hence, S_i can locally compute the term $[a]_u \cdot [b]_v$ if and only if $i \notin T_u \cup T_v$. Define the set of *eligible servers* for term (u, v) as $\mathcal{E}_{u,v} := [n] \setminus (T_u \cup T_v)$. Assuming $n = 2t + 1$, we have $\#\mathcal{E}_{u,v} = n - \#(T_u \cup T_v) = n - (2t - \#(T_u \cap T_v)) = \#(T_u \cap T_v) + 1$, and in particular $1 \leq \#\mathcal{E}_{u,v} \leq t + 1$ for all $u, v \in [N]$, so every product term is computable by at least one server.

We now compute a balanced assignment $\{\Lambda_i\}_{i \in [n]}$ such that (i) $(u, v) \in \Lambda_i$ implies $i \in \mathcal{E}_{u,v}$ for local computability, and (ii) each server computes the same number of terms. Since there are N^2 terms in total and N^2 is divisible by n in our setting, each server is assigned exactly $L := N^2/n$ terms. We compute such an allocation by reducing it to a (weighted) bipartite b -matching and max-flow instance. First, consider a bipartite graph in which left vertices represent pair indices $(u, v) \in [N]^2$, right vertices represent servers $i \in [n]$, and each left vertex (u, v) has edges to right vertex i with weight 1 if and only if $i \in \mathcal{E}_{u,v}$. Therefore, each left vertex (u, v) has exactly $b((u, v)) = 1$ edge and each right vertex i has exactly $b(i) = L$ edges. Then, construct a directed flow network based on the bipartite graph as follows: a source node n_s has edges to each left node (u, v) with capacity 1, a sink node n_t has incoming edges with capacity L from each right node. Any integral max flow of value $N^2 = nL$ yields a balanced assignment: term (u, v) is assigned to the unique server i with one unit of flow on edge $(u, v) \rightarrow i$. This guarantees that each server S_i can compute its assigned terms locally since $i \in \mathcal{E}_{u,v}$, and that each server receives exactly L terms. The seminal Ford–Fulkerson algorithm [20] (or more efficient algorithms such as [22]) computes the max flow in polynomial time in N .

5.2 Our Solution

5.2.1 Basic dVOPRF Construction. Given the above assignment, our basic construction, which works in a semi-honest security setting, is as follows. Each server S_i locally computes its partial sum $c_i := \sum_{(u,v) \in \Lambda_i} [a]_u \cdot [b]_v$ using only its local RSS components $\llbracket a \rrbracket_i$ and $\llbracket b \rrbracket_i$. The servers then distribute RSS sharings of these values, so that each server obtains $\llbracket c_1 \rrbracket_i, \dots, \llbracket c_n \rrbracket_i$ and computes $\llbracket c \rrbracket_i := \sum_{j=1}^n \llbracket c_j \rrbracket_i$, where $c = \sum_{j=1}^n c_j = ab = \alpha^e(x + k)$. Finally, the client reconstructs c from the received RSS shares and obtains a PRF value $c^g \bmod p = (x + k)^g \bmod p$.

5.2.2 Towards Malicious Security. To upgrade the above protocol to malicious security, we build on the DZKP techniques for degree-2 relations of Boyle et al. [11]. In particular, we introduce a core sub-protocol, which we call the *verifiable inner product* (VIP) protocol.² For each server S_i , the VIP protocol outputs not only an RSS sharing $\llbracket c_i \rrbracket$, but also RSS sharings of four auxiliary elements

²This protocol can be viewed as computing a shared inner product $c_i = \langle \vec{a}_i, \vec{b}_i \rangle$ for $\vec{a}_i = (a_i^{(1)}, \dots, a_i^{(L)})$ and $\vec{b}_i = (b_i^{(1)}, \dots, b_i^{(L)})$, together with additional values used to (later) check the validity of the computation; therefore we name it the VIP protocol.

Protocol 4 Π_{VIP} – Verifiable Computation for Inner Products over RSS in the $\mathcal{F}_{Coin}$ -Hybrid Model

Players: Servers $S_1, \dots, S_n$, where S_i is a dealer.

Input: Servers hold $\{\llbracket a_i^{(\ell)} \rrbracket, \llbracket b_i^{(\ell)} \rrbracket\}_{\ell \in [L]}$ where S_i knows $\{a_i^{(\ell)}, b_i^{(\ell)}\}_{\ell \in [L]}$.

```

1: Let  $\gamma \leftarrow \lceil \log L \rceil$ .
2: for  $k = 1, \dots, \gamma$  do
3:   Let  $L \leftarrow L + 1$  if  $L$  is odd; let  $L \leftarrow L$  otherwise.
4:   Servers define (or update) polynomials  $f_1, f_2, \dots, f_L$ , and
      $q^{(k)}$  as defined in Eqs. (1) and (2).
5:    $S_i$  shares  $q^{(k)}(1), q^{(k)}(2), q^{(k)}(3)$  to all servers using VSS.
6:   if  $k = 1$  then
7:     Servers locally compute  $\llbracket c_i \rrbracket \leftarrow \llbracket q^{(k)}(1) \rrbracket + \llbracket q^{(k)}(2) \rrbracket$ .
8:   end
9:   Servers call  $\mathcal{F}_{Coin}$  to obtain  $r_k \xleftarrow{\$} \mathbb{F}_p \setminus \{1, 2, 3\}$ .
10:   $S_i$  sets  $\{a_i^{(\ell)}, b_i^{(\ell)}\} \leftarrow \{f_{2\ell-1}(r_k), f_{2\ell}(r_k)\}$  for all  $\ell \in [L/2]$ , and  $d_i^{(k)} \leftarrow q^{(k)}(r_k)$ .
11:  Servers use Lagrange interpolation to locally compute
      $\{\llbracket a_i^{(\ell)} \rrbracket, \llbracket b_i^{(\ell)} \rrbracket\} \leftarrow \{\llbracket f_{2\ell-1}(r_k) \rrbracket, \llbracket f_{2\ell}(r_k) \rrbracket\}$  for all  $\ell \in [L/2]$  and  $\llbracket d_i^{(k)} \rrbracket \leftarrow \llbracket q^{(k)}(r_k) \rrbracket$ , and set  $L \leftarrow L/2$ .
12: end
13: Servers call  $\mathcal{F}_{Coin}$  to obtain  $\epsilon_2, \epsilon_3, \dots, \epsilon_\gamma \xleftarrow{\$} \mathbb{F}_p$  and locally
     update  $\llbracket \sigma_i \rrbracket \leftarrow \sum_{k=2}^\gamma \epsilon_k (\llbracket d_i^{(k-1)} \rrbracket - \llbracket q^{(1)}(1) \rrbracket - \llbracket q^{(1)}(2) \rrbracket)$ .
Output: Servers record  $(\llbracket d_i^{(\gamma)} \rrbracket, \llbracket a_i^{(1)} \rrbracket, \llbracket b_i^{(1)} \rrbracket, \llbracket \sigma_i \rrbracket, \llbracket c_i \rrbracket)$ .
```

d_i, a_i, b_i, σ_i used to certify the correctness of $\llbracket c_i \rrbracket$. At a high level, the L multiplication terms contributing to c_i are *compressed* into three values (d_i, a_i, b_i) that are required to satisfy the multiplicative relation $d_i = a_i b_i$. Intuitively, this compression binds S_i to a single well-defined computation: if S_i attempts to inconsistently encode the underlying L terms while still claiming a valid relation, then the equality $d_i = a_i b_i$ fails with high probability by the Schwartz–Zippel lemma. The additional check value σ_i enforces that the claimed inner product value c_i exactly equals $\sum_{(u,v) \in \Lambda_i} [a]_u \cdot [b]_v$; in particular, $\sigma_i = 0$ if and only if c_i is consistent with the L terms. Consequently, if the protocol is executed honestly, then both checks hold, i.e., $d_i = a_i b_i$ and $\sigma_i = 0$, whereas if up to t servers (possibly including S_i) deviate arbitrarily, then at least one of these conditions fails.

Thus, our maliciously secure dVOPRF protocol proceeds as follows. Each server S_i runs the VIP protocols to share S_i 's terms c_i with other servers, and obtains $\llbracket c \rrbracket_i = \llbracket c_1 \rrbracket_i + \dots + \llbracket c_n \rrbracket_i$ and their proof elements. Servers batch those proofs and send them along with $\llbracket c \rrbracket$ to the client C , thereby allowing C to securely open $\llbracket c \rrbracket$ by checking the validity of the computation with the proofs.

5.3 Protocol Description

5.3.1 Verifiable Inner Product (VIP) Protocol. We begin with the formal description of the VIP protocol Π_{VIP} in Protocol 4, which computes RSS sharing of $c_i := \sum_{\ell=1}^L a_i^{(\ell)} b_i^{(\ell)}$, where $\{(a_i^{(\ell)}, b_i^{(\ell)})\}_{\ell \in [L]} := \{([a]_u, [a]_v) \in \llbracket a \rrbracket_i \times \llbracket b \rrbracket_i\}_{(u,v) \in \Lambda_i}$ (namely, c_i consists only of additive shares that S_i holds), together with RSS sharings of the corresponding auxiliary proof elements $(d_i^{(\gamma)}, a_i^{(1)}, b_i^{(1)}, \sigma_i)$.

This protocol requires all servers to hold RSS shares of all $a_i^{(\ell)}$ and all $b_i^{(\ell)}$, which can be locally computed by the local share conversion [14], since thanks to the RSS property of $\llbracket a \rrbracket$ and $\llbracket b \rrbracket$, each of $a_i^{(\ell)} (= [a]_u)$ and $b_i^{(\ell)} (= [b]_v)$ is shared among $t+1$ servers.

Suppose L is even.³ For $j \in [L]$ and $\ell \in [2]$, define linear polynomials $f_1, \dots, f_L, q^{(1)}$ as:

$$f_j(\ell) = \begin{cases} a_i^{(\frac{\ell}{2} \cdot (\ell \bmod 2) + \lceil \frac{\ell}{2} \rceil)} & \text{if } j \text{ is odd, i.e., } j \bmod 2 = 1, \\ b_i^{(\frac{\ell}{2} \cdot (\ell \bmod 2) + \frac{\ell}{2})} & \text{if } j \text{ is even, i.e., } j \bmod 2 = 0, \end{cases} \quad (1)$$

$$q^{(1)}(x) = \sum_{\ell=1}^{L/2} f_{2\ell-1}(x) \cdot f_{2\ell}(x). \quad (2)$$

That is, each $f_j(x)$ contains every 2 elements of $2L$ inputs $\{(a_i^{(\ell)}, b_i^{(\ell)})\}_{\ell \in [L]}$, and therefore the polynomial $q^{(1)}(x)$ of degree L can be seen as a folded one of $q^{(0)}(x)$, which is a polynomial of degree $2L$ such that $q^{(0)}(1) = c_i$ (and $q^{(0)}(2) = 0$). S_i then evaluates random points $f_1(r_1), \dots, f_L(r_1), q^{(1)}(r_1)$ with randomness $r_1 \in \mathbb{F}_p$; the Schwartz-Zippel lemma ensures that every 2 elements are embedded into each polynomial $f_1, \dots, f_L$ and the folded relation between $q^{(1)}$ and $q^{(0)}$ is committed. In such a way, S_i recursively folds polynomials $\gamma := \lceil \log L \rceil$ times in total. In the k -th iteration, S_i sets $a_i^{(\ell)} \leftarrow f_{2\ell-1}(r_{k-1})$ and $b_i^{(\ell)} \leftarrow f_{2\ell}(r_{k-1})$ for $\ell \in [L/2^{k-1}]$, redefines $f_1, \dots, f_{L/2^{k-1}}$, and folds the previous polynomial $q^{(k-1)}(x)$ of degree $L/2^{k-2}$ to get a folded polynomial $q^{(k)}(x)$ of degree $L/2^{k-1}$. After the γ iterations, S_i only holds $a_i^{(1)} (= f_1(r_\gamma)), b_i^{(1)} (= f_2(r_\gamma))$, and $d_i^{(\gamma)} (= q^{(\gamma)}(r_\gamma))$, which still ensures that $d_i^{(\gamma)} = a_i^{(1)} \cdot b_i^{(1)}$ holds if S_i proceeds correctly; $d_i^{(\gamma)} \neq a_i^{(1)} \cdot b_i^{(1)}$ holds otherwise.

In addition, we must verify the relation between adjacent iterations, i.e., $q^{(k-1)}(r_{k-1}) \stackrel{?}{=} q^{(k)}(1) + q^{(k)}(2)$ for every $k \in [\gamma]$. To this end, S_i computes $\sigma_i := \sum_{k=2}^{\gamma} \epsilon_k (q^{(k-1)}(r_{k-1}) - q^{(k)}(1) - q^{(k)}(2))$, where $\epsilon_2, \dots, \epsilon_\gamma$ are random field elements; $\sigma_i = 0$ holds if and only if S_i honestly connects all adjacent iterations.⁴ All servers except for S_i can follow the same procedures by using their RSS shares $\{(\llbracket a_i^{(\ell)} \rrbracket, \llbracket b_i^{(\ell)} \rrbracket)\}_{\ell \in [L]}$, which is derivable by the local conversion, and $\{\llbracket q^{(k)} \rrbracket\}_{k \in [\gamma]}$ sent by S_i , and obtain $\llbracket d_i^{(\gamma)} \rrbracket, \llbracket a_i^{(1)} \rrbracket, \llbracket b_i^{(1)} \rrbracket, \llbracket \sigma_i \rrbracket$, and $\llbracket c_i \rrbracket (= \llbracket q^{(1)}(1) \rrbracket + \llbracket q^{(1)}(2) \rrbracket)$.

Unlike in Boyle et al.'s setting, neither the client nor the servers open the output, particularly $a_i^{(1)}$ and $b_i^{(1)}$, in the current form during the dVOPRF protocol; consequently, we do not require the additional output-randomization step used in Boyle et al.'s protocol [11], where openings would otherwise reveal linear relations among inputs. Note that the output-randomization step will be required later in the dVOPRF protocol.

5.3.2 dVOPRF Protocol with Single Input. At a high level, our dVOPRF protocol proceeds as follows. First, the servers execute n instances of the VIP protocol Π_{VIP} in parallel (one instance per server) in order to obtain RSS sharings of the partial products $\{\llbracket c_i \rrbracket\}_{i \in [n]}$, together with the auxiliary proofs $(\llbracket d_i \rrbracket, \llbracket a_i \rrbracket, \llbracket b_i \rrbracket, \llbracket \sigma_i \rrbracket)_{i \in [n]}$ required for verification. Second, the servers batch the auxiliary values into a single tuple of aggregated proofs of correctness while applying the output-randomization step described above. Third, the client

³If L is odd, one may just set $L \leftarrow L+1$ and $a_i^{(L)} = b_i^{(L)} = 0$.

⁴ c_i itself ensures that $c_i - q^{(1)}(1) - q^{(1)}(2) = 0$ so we can ignore the case of $k = 1$.

Protocol 5 Π_{VIP}^{Prf} – Batched Verifiable Computation for Inner Products in the $(\mathcal{F}_{\text{Coin}}, \mathcal{F}_{\text{Rand}}, \mathcal{F}_{\text{RSS.Mul}})$ -Hybrid Model

Players: Servers $S_1, \dots, S_n$.

Input: Each Server S_i holds $\{a_i^{(\ell)}, b_i^{(\ell)}\}_{\ell \in [L]}$ and $\{\llbracket a_j^{(\ell)} \rrbracket, \llbracket b_j^{(\ell)} \rrbracket\}_{\ell \in [L]}$ for all $j \in [n]$.

- 1: Run n instances of Π_{VIP} with the servers' shares in parallel, where S_i acts as the dealer in the i -th instance. All instances share the same outputs $(r_k, \epsilon_k)_{k \in [\log L]}$ of $\mathcal{F}_{\text{Coin}}$.
- 2: Each instance i produces $(\llbracket d_i \rrbracket, \llbracket a_i \rrbracket, \llbracket b_i \rrbracket, \llbracket \sigma_i \rrbracket, \llbracket c_i \rrbracket)$.
- 3: Servers locally compute $\llbracket c \rrbracket = \sum_{i=1}^n \llbracket c_i \rrbracket$.
- 4: Servers call $\mathcal{F}_{\text{Coin}}$ to obtain $\{\epsilon'_i\}_{i \in [n]}$, and then locally compute $\llbracket \Sigma \rrbracket = \sum_{i=1}^n \epsilon'_i \cdot \llbracket \sigma_i \rrbracket$.
- 5: Servers call $\mathcal{F}_{\text{Rand}}$ to obtain $\llbracket a_0 \rrbracket, \llbracket b_0 \rrbracket$.
- 6: Servers locally compute $\llbracket U(x) \rrbracket$ and $\llbracket V(x) \rrbracket$ of degree n by interpolating through $\{(0, \llbracket a_0 \rrbracket)\} \cup \{(i, \llbracket a_i \rrbracket)\}_{i \in [n]}$ and $\{(0, \llbracket b_0 \rrbracket)\} \cup \{(i, \llbracket b_i \rrbracket)\}_{i \in [n]}$, respectively.
- 7: **for** $i \in \{0, n+1, \dots, 2n\}$ **do**
- 8: Servers call $\mathcal{F}_{\text{RSS.Mul}}$ to get $\llbracket W(i) \rrbracket = \llbracket U(i) \cdot V(i) \rrbracket$.
- 9: **end**
- 10: Servers locally compute $\llbracket W(x) \rrbracket$ of degree $2n$ by interpolating via $\{(i, \llbracket W(i) \rrbracket)\}_{i \in \{0, \dots, 2n\}}$, where $\llbracket W(i) \rrbracket \leftarrow d_i$ for $i \in [n]$.
- 11: Servers call $\mathcal{F}_{\text{Coin}}$ to obtain $\rho \xleftarrow{\$} \mathbb{F}_p \setminus \{0, 1, \dots, 2n\}$, then locally compute $\llbracket U(\rho) \rrbracket, \llbracket V(\rho) \rrbracket, \llbracket W(\rho) \rrbracket$.

Output: Servers hold $(\llbracket W(\rho) \rrbracket, \llbracket U(\rho) \rrbracket, \llbracket V(\rho) \rrbracket, \llbracket \Sigma \rrbracket, \llbracket c \rrbracket)$.

(robustly) reconstructs the product c from $\llbracket c \rrbracket (= \sum_{i=1}^n \llbracket c_i \rrbracket)$ if verification with aggregated proofs succeeds; the client then computes $c^g \bmod p = (x+k)^g \bmod p$.

We encapsulate the first two steps into a single *batched VIP* protocol, denoted Π_{VIP}^{Prf} (Protocol 5). Here, we elaborate on the second step: the batching process. With the proof values $(\llbracket d_i \rrbracket, \llbracket a_i \rrbracket, \llbracket b_i \rrbracket)_{i \in [n]}$ for proving the folding relation, servers also compress them into single polynomials as follows: $U(x)$ and $V(x)$ are polynomials such that $U(i) = a_i$ and $V(i) = b_i$ for all $i \in \{0, 1, \dots, n\}$, where a_0 and b_0 are random values to randomize the polynomials, which we call the output-randomization step mentioned above. This randomization is necessary to prevent $U(x)$ and $V(x)$ from leaking information on the servers' inputs. Therefore, servers can interpolate $\llbracket U(x) \rrbracket$ and $\llbracket V(x) \rrbracket$ from $\{\llbracket a_i \rrbracket, \llbracket b_i \rrbracket\}_{i \in [n]}$ and the random shares $\llbracket a_0 \rrbracket, \llbracket b_0 \rrbracket$. Let $W(x)$ be a degree- $2n$ polynomial such that $W(x) = U(x) \cdot V(x)$, i.e., $W(i) = d_i$ for all $i \in [n]$. Since servers already have (shares of) n evaluations $\{\llbracket d_i \rrbracket\}_{i \in [n]}$, they additionally execute secure multiplication protocols to obtain $\llbracket W(i) \rrbracket$ from $\llbracket U(i) \rrbracket$ and $\llbracket V(i) \rrbracket$ for $i = 0, n+1, \dots, 2n$, enabling them to interpolate $\llbracket W(x) \rrbracket$.

Together with the last step, i.e., the client-side robust opening procedure, Π_{VIP}^{Prf} yields a dVOPRF protocol for a *single* input that is maliciously secure against up to $t < n/2$ corrupted servers.

5.3.3 Batching m Inputs Together. We are ready to describe our final protocol Π_{dVOPRF}^m (Protocol 6), a more efficient variant that supports m client inputs $(x^{(1)}, \dots, x^{(m)})$ and amortizes the verification and communication costs across multiple evaluations. Briefly speaking, servers run the batched VIP protocol Π_{VIP}^{Prf} once for *batched* product $c = \sum_{j=1}^m \epsilon_j c^{(j)}$, where $c^{(j)} = (\alpha^{(j)})^e \cdot (x^{(j)} + k)$ and ϵ_j is

Protocol 6 Π_{dVOPRF}^m – Our dVOPRF Protocol in the $(\mathcal{F}_{\text{RandExp}}, \mathcal{F}_{\text{Zero}}, \mathcal{F}_{\text{Coin}}, \mathcal{F}_{\text{Rand}}, \mathcal{F}_{\text{RSS,Open}})$ -Hybrid Model

Players: Client C and Servers $S_1, \dots, S_n$.

Input: Client C inputs $(x^{(1)}, \dots, x^{(m)})$, and servers input $\llbracket k \rrbracket$.

- 1: Servers call $\mathcal{F}_{\text{RandExp}}(e)$, $\mathcal{F}_{\text{Zero}}$, and $\mathcal{F}_{\text{Rand}}$ m times each to obtain $(\llbracket (\alpha^{(j)})^e \rrbracket)_{j \in [m]}$, $(\llbracket z^{(j)} \rrbracket)_{j \in [m]}$, and the input masks $(\llbracket r^{(j)} \rrbracket)_{j \in [m]}$, respectively.
- 2: Client C and the servers run Π_{Input} (Protocol 9) on $(x^{(j)})_{j \in [m]}$ with the masks $(\llbracket r^{(j)} \rrbracket)_{j \in [m]}$, deferring its echo step to Step 6; the servers obtain $(\llbracket x^{(j)} \rrbracket)_{j \in [m]}$ and C aborts if the mask opening fails.
- 3: For every $j \in [m]$, let $\llbracket a^{(j)} \rrbracket \leftarrow \llbracket x^{(j)} + k \rrbracket = (\llbracket x^{(j)} \rrbracket + \llbracket k \rrbracket)$ and $\llbracket b^{(j)} \rrbracket \leftarrow \llbracket (\alpha^{(j)})^e \rrbracket$.
- 4: For every $i \in [n]$ and $j \in [m]$, let $a_i^{(j,\ell)} \leftarrow [a^{(j)}]_{u_\ell}$ and $b_i^{(j,\ell)} \leftarrow [b^{(j)}]_{v_\ell}$ for every $(u_\ell, v_\ell) \in \Lambda_i$, where $\llbracket a^{(j)} \rrbracket_i = \{[a^{(j)}]_\ell \mid i \notin T_\ell\}$, $\llbracket b^{(j)} \rrbracket_i = \{[b^{(j)}]_\ell \mid i \notin T_\ell\}$, and $\{\Lambda_i\}_{i \in [n]}$ is a balanced assignment (defined in §5.1).
- 5: Each server S_i locally computes $v_i^{(j)} = \sum_{\ell=1}^L a_i^{(j,\ell)} \cdot b_i^{(j,\ell)} + [z^{(j)}]_i$ for all $j \in [m]$.
- 6: Each server S_i computes $h_i \leftarrow H(v_i^{(1)} \parallel \dots \parallel v_i^{(m)})$ and the deferred echo value $\chi_i \leftarrow H(u^{(1)} \parallel \dots \parallel u^{(m)})$ of Π_{Input} , and sends (h_i, χ_i) to all other servers; S_i aborts unless $\chi_{i'} = \chi_i$ for all $i' \in [n]$.
- 7: Servers derive $\rho_\epsilon \leftarrow H(h_1 \parallel \dots \parallel h_n)$, and then compute $\epsilon_j \leftarrow H(\rho_\epsilon \parallel j)$ for every $j \in [m]$.
- 8: Servers locally obtain $\llbracket a_i^{(j,\ell)} \rrbracket$ and $\llbracket b_i^{(j,\ell)} \rrbracket$ via local share conversion for all $i \in [n]$, $j \in [m]$, and $\ell \in [L]$.
- 9: Servers locally compute scaled inputs $\llbracket \hat{a}_i^{(j,\ell)} \rrbracket = \epsilon_j \cdot \llbracket a_i^{(j,\ell)} \rrbracket$ and $\llbracket \hat{b}_i^{(j,\ell)} \rrbracket = \llbracket b_i^{(j,\ell)} \rrbracket$ for all $i \in [n]$, $j \in [m]$, and $\ell \in [L]$.
- 10: Servers invoke $\Pi_{\text{VIP}}^{\text{PrI}}$ with the scaled inputs to obtain proof $(\llbracket W \rrbracket, \llbracket U \rrbracket, \llbracket V \rrbracket, \llbracket \Sigma \rrbracket, \llbracket c \rrbracket)$.
- 11: Client C calls $\mathcal{F}_{\text{RSS,Open}}$ to robustly open $\llbracket W \rrbracket, \llbracket U \rrbracket, \llbracket V \rrbracket, \llbracket \Sigma \rrbracket$, and $\llbracket c \rrbracket$. If at least one opening fails, C aborts the protocol.
- 12: Client C computes $v^{(j)} = \sum_{i \in [n]} v_i^{(j)}$ and $\{\epsilon_j\}_{j \in [m]}$ using $\{v_i^{(j)}\}_{j \in [m]}$ for all $j \in [m]$.
- 13: Client C aborts the protocol if at least one of the following equations does not hold: $W = U \cdot V$; $\Sigma = 0$; and $c = \sum_{j=1}^m (\epsilon_j \sum_{i=1}^n v_i^{(j)})$.

Output: Client C outputs $((v^{(1)})^g, \dots, (v^{(m)})^g)$.

randomness synchronously and securely generated by using random oracles. Then, c turns into a part of the proofs of batched computation, not the computation results of dVOPRF, together with other outputs of $\Pi_{\text{VIP}}^{\text{PrI}}$. Thus, each server S_i additionally prepares their local partial sums $c_i^{(1)}, \dots, c_i^{(m)}$ to have the client reconstruct the products $c^{(j)} = \sum_{i=1}^n c_i^{(j)}$. Although the partial sums $\{c_i^{(j)}\}_{i \in [n], j \in [m]}$ can be seen as additive shares and hence have no robustness, the batched proofs $(\llbracket W \rrbracket, \llbracket U \rrbracket, \llbracket V \rrbracket, \llbracket \Sigma \rrbracket, \llbracket c \rrbracket)$ generated by $\Pi_{\text{VIP}}^{\text{PrI}}$ guarantee the correctness of computations for all $c^{(j)}$.

A remaining concern is that each partial sum $c_i^{(j)}$ would leak information about S_i 's inputs. To resolve this issue, we employ random additive sharings $[z^{(j)}]$ of zero; S_i sends $(v_i^{(j)} := c_i^{(j)} + [z^{(j)}]_i)_{j=1}^m$, instead of $(c_i^{(j)})_{j=1}^m$, to the client. Since $\sum_{i=1}^n v_i^{(j)} =$

Table 2: Online-phase efficiency comparison; Comm. is per-server communication in field elements.

(n, t)	Protocol (online)	$m = 1$		$m = 100$	
		Comm.	Rnds	Comm.	Rnds
(3, 1)	$\Pi_{\text{naive-dVOPRF}}^m$	109	13	1 224.5	13
(3, 1)	Our Π_{dVOPRF}^m	72	9	544	9
(5, 2)	$\Pi_{\text{naive-dVOPRF}}^m$	840	13	4 957.5	13
(5, 2)	Our Π_{dVOPRF}^m	633	9	1 731	9
(7, 3)	$\Pi_{\text{naive-dVOPRF}}^m$	5 485	13	23 888.5	13
(7, 3)	Our Π_{dVOPRF}^m	4 470	9	8 964	9
(9, 4)	$\Pi_{\text{naive-dVOPRF}}^m$	32 056	13	113 230.5	13
(9, 4)	Our Π_{dVOPRF}^m	27 373	9	46 128	9

$\sum_{i=1}^n c_i^{(j)} = c^{(j)}$ whenever $\sum_{i=1}^n [z^{(j)}]_i = 0$, this masking does not affect the above procedures while hiding the crucial information of $c_i^{(j)}$.

A final concern is the client's own input. The batched proof certifies the computation the servers actually performed, so if a malicious client could hand different servers inconsistent shares of $x^{(j)}$, the servers would evaluate on a point the client never committed to and the proof would certify it regardless. Step 2 therefore invokes Π_{Input} (Protocol 9 in Appendix E.1): the servers open a pre-computed mask $\llbracket r^{(j)} \rrbracket$ to C with abort, and C broadcasts $u^{(j)} := r^{(j)} + x^{(j)}$, from which every server derives $\llbracket x^{(j)} \rrbracket := u^{(j)} - \llbracket r^{(j)} \rrbracket$ locally. The servers confirm that they all received the same $u^{(j)}$ by exchanging the hash χ_i , which we fold into the hash exchange of Step 6 so that it costs no additional round. By Lemma E.1, this fixes a unique input that the simulator can extract, keeps it hidden from any t servers, and—because the mask is opened with abort rather than reconstructed—prevents corrupted servers from shifting an honest client's input.

We prove that Π_{dVOPRF}^m securely realizes $\mathcal{F}_{\text{dVOPRF}}^{F_k}$ for the Gold PRF F . The full proof is given in Appendix K.

THEOREM 5.1. *Let $p = eg + 1$ be a prime of $O(\lambda)$ bits, where p, e, g are public, and $F_k(x)$ be Gold PRF over $\mathbb{F}_p^*$. Then, assuming random oracles, Π_{dVOPRF}^m in Protocol 6 securely realizes $\mathcal{F}_{\text{dVOPRF}}^{F_k}$ over $\mathbb{F}_p$, with statistical error at most ϵ_{dVOPRF} in the $(\mathcal{F}_{\text{RandExp}}, \mathcal{F}_{\text{Zero}}, \mathcal{F}_{\text{Coin}}, \mathcal{F}_{\text{Rand}}, \mathcal{F}_{\text{RSS,Open}})$ -hybrid model, against a malicious adversary corrupting up to $t < n/2$ servers and, possibly, the client C , where $\epsilon_{\text{dVOPRF}} = n(2\lceil \log(mL) \rceil + 1)/(|\mathbb{F}_p| - 3) + 2/|\mathbb{F}_p| + 2n/(|\mathbb{F}_p| - 2n - 1)$.⁵*

5.4 Efficiency Analysis

We analyze the communication costs of the online computations in our dVOPRF protocol Π_{dVOPRF}^m with m inputs and compare them with those of the naive protocol $\Pi_{\text{naive-dVOPRF}}^m$.⁶ Tab. 2 shows an efficiency comparison between them for concrete values of n, t , and

⁵If $\mathcal{F}_{\text{RandExp}}$ is instantiated by sampling $\alpha \leftarrow \mathbb{F}_p$ instead of $\alpha \leftarrow \mathbb{F}_p^*$, then the probability that some $\alpha^{(j)} = 0$ is at most $m/|\mathbb{F}_p|$. In that case, this term should be added to the statistical error.

⁶We assume the FS transform [19] is applied to both schemes; the FS variant of Π_{dVOPRF}^m is given in Appendix L.

m . Our protocol reduces per-server communication in every (n, t) regime—by $\approx 1.2\text{--}1.5\times$ at $m = 1$ and up to $\approx 2.9\times$ at $m = 100$ —and saves four rounds ($13 \rightarrow 9$). Moreover, if we allow a slight increase in rounds and use more efficient building blocks (Protocols 7 and 8 in Appendix E), the efficiency gap between them widens significantly ($4.8\times$ smaller for $(n, t, m) = (9, 4, 100)$). A detailed analysis and comparison are given in Appendix M.

6 Evaluation

For the offline phase, we implement our protocols Π_{DegGen} , $\Pi_{\text{ZKPGen}}^{\text{Vith}}$, $\Pi_{\text{ZKPGen}}^{\text{Lig}}$ alongside the baseline Π_{AlyGen} [4, 30]. For the online phase, we implement Π_{dVOPRF}^m and the baseline $\Pi_{\text{naive-dVOPRF}}^m$ [9]. **[Implementation rework pending: neither implementation yet includes Π_{Input} (Protocol 9), which both protocols now use for the client's input. All online and end-to-end measurements must be regenerated once it is added; the analytical tables (Tab. 2 and Appendix M) have already been updated.]** For end-to-end comparison, we compare with the Legendre dOPRF $\Pi_{\text{L-dOPRF}}$ [28], the closest available dOPRF, benchmarked on $(4, 1)$ and $(7, 2)$ since theirs requires $t < n/3$.

Setup. All protocols are implemented in Rust. Protocols are executed in a simulated network adopting a WAN model with $\text{RTT} = 100 \text{ ms}$ and bandwidth $\text{BW} = 100 \text{ Mbps}$ as $T_{\text{WAN}} = T_{\text{comp}} + \text{Rounds} \cdot \text{RTT} + \frac{8 \cdot \text{TotalBytes}}{\text{BW}}$, where T_{comp} is the mean local computation time over 10 runs. The same setup applies to all our benchmarks, each averaged over 10 runs, executed on a machine running Ubuntu 22.04 with a 12th-gen Intel Core i7-12700K and 32 GB RAM.

Parameters. We set $\kappa = 40$ (per MP-SPDZ [29]); Π_{Vith} uses GGM tree arity $\tau = 16$, giving $R = \lceil 40/4 \rceil = 10$ repetitions. Benchmarks span $(n, t) \in \{(3, 1), (5, 2), (7, 3), (9, 4)\}$ for Gold PRF and $(4, 1), (7, 2)$ for Legendre OPRF (which requires $t < n/3$). For matched post-quantum security at $\lambda = 128$, both PRFs are configured at $|p| = 3\lambda = 384 \text{ bits}$ —ours via DSPRS [34], Legendre via DSLSP [28] after its classical 2λ bound is inflated by the $3/2$ post-quantum factor—so all benchmarks share the same $\mathbb{F}_p$.

6.1 Offline Phase

Fig. 4 compares the offline protocols on computation overhead and communication volume at $m = 1$ (left) and $m = 100$ (right). $\Pi_{\text{ZKPGen}}^{\text{Lig}}$ is plotted only over $(7, 3) \rightarrow (9, 4)$, since its Liger RS-code requires a code length n_c that is too large to instantiate at small (n, t) , and Π_{AlyGen} is omitted from the right panel as it exceeds our memory budget. The full numerical breakdown is given in Tab. 5 of Appendix N.

Scaling. As (n, t) grows from $(3, 1)$ to $(9, 4)$, Π_{AlyGen} scales worst on both axes, since the number of RSS multiplications it requires grows fastest. Π_{DegGen} is the cheapest on both axes at small (n, t) , but its sequential per-share check inflates rounds super-linearly in (n, t) (cf. Tab. 5), yielding the steepest computation time growth. In contrast, both Π_{ZKPGen} variants are constant-round and thus achieve the flattest computation time growth; on the communication side, $\Pi_{\text{ZKPGen}}^{\text{Lig}}$ amortizes broadcast via RS-code batching while $\Pi_{\text{ZKPGen}}^{\text{Vith}}$'s communication grows linearly in m .

Per-setting winners. For the most commonly deployed setting $(n, t) = (3, 1)$, Π_{DegGen} performs best on both axes. At $(n, t) = (5, 2)$,

$\Pi_{\text{ZKPGen}}^{\text{Vith}}$ outperforms all others on computation time when $m=1$, though it offers only a computation-communication trade-off relative to Π_{DegGen} at $(3, 1)$. From $(7, 3)$ onward, the two Π_{ZKPGen} variants perform comparably at $m=1$, while $\Pi_{\text{ZKPGen}}^{\text{Lig}}$ clearly outperforms $\Pi_{\text{ZKPGen}}^{\text{Vith}}$ at large batch sizes (i.e., $m=100$).

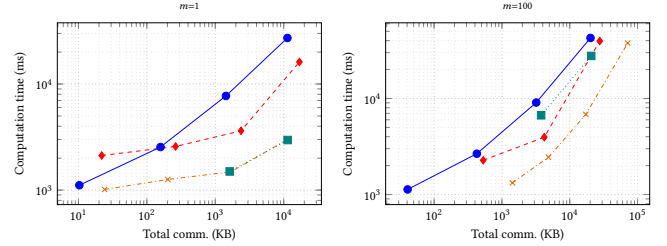


Figure 4: Offline phase: total communication vs. computation time. Each line traces $(n, t) \in \{(3, 1), (5, 2), (7, 3), (9, 4)\}$ left-to-right; $\Pi_{\text{ZKPGen}}^{\text{Lig}}$ traces $(7, 3) \rightarrow (9, 4)$. **Legend:** Π_{Aly} [4, 30] (dashed, $\diamond$); Π_{DegGen} (solid, $\bullet$); $\Pi_{\text{ZKPGen}}^{\text{Vith}}$ (dash-dot, $\times$); $\Pi_{\text{ZKPGen}}^{\text{Lig}}$ (dotted, $\blacksquare$).

6.2 Online Phase

[Implementation rework pending: every measured number in this subsection predates Π_{Input} and is expected to shift; see the note in Appendix N for the predicted deltas.]

Fig. 5 compares Π_{dVOPRF}^m against $\Pi_{\text{naive-dVOPRF}}^m$ at $m=1$ (left) and $m=100$ (right). Detailed numbers are given in Tab. 6 of Appendix N.

Scaling. On the communication axis, Π_{dVOPRF}^m scales much more gracefully: its advantage grows from $\sim 2\times$ at $(3, 1)$ to over $25\times$ at $(9, 4)$, since the naive RSS share/open routine sends $O(N)$ shares per opening while our fused $\Pi_{\text{VIP}}^{\text{Prf}}$ closes over a single opened term, where $N = \binom{n}{t}$. On the computation axis, however, our protocol's local work grows super-linearly in (n, t) and linearly in m because fusing multiplication with verification forces each server to evaluate $O(Lm)$ cross-products, whereas the naive baseline pays only a linear-in- m DZKP, where $L = N^2/n$.

Per-setting winners. Π_{dVOPRF}^m outperforms the baseline on both axes at every $(n, t, m=1)$ profile and at $(3, 1) - (7, 3)$ for $m=100$. The exception is $(9, 4), m=100$, where the naive baseline overtakes our protocol on computation while still losing the communication axis by $\approx 30\times$. **[Implementation rework pending: this ratio is read off Tab. 6, which predates Π_{Input} ; recheck after re-measuring.]**

6.3 End-to-end Performance

We construct a full deployment by composing one offline protocol with one online protocol. For our three end-to-end constructions, we write Π_{dVOPRF}^X for the pairing of our online Π_{dVOPRF}^m with offline preprocessing $X \in \{\text{DegGen}, \text{Vith}, \text{Lig}\}$. The naive baseline $\Pi_{\text{DZKP}}^{\text{Aly}}$ pairs the Aly offline phase Π_{AlyGen} with the naive online protocol $\Pi_{\text{naive-dVOPRF}}^m$. Fig. 6 compares the full deployment cost (offline+online) of our three constructions against $\Pi_{\text{DZKP}}^{\text{Aly}}$ and $\Pi_{\text{L-dOPRF}}$ at $m=1$ (left) and $m=100$ (right). Detailed numbers are reported in Tab. 7 of Appendix N.

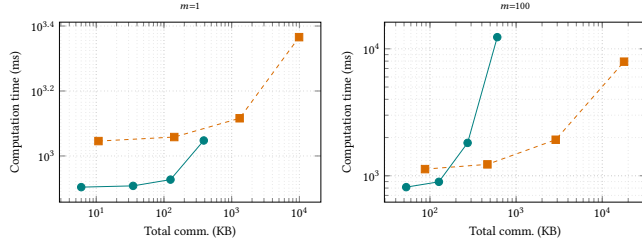


Figure 5: Online phase: total communication vs. computation time. Each line traces $(n, t) \in \{(3, 1), (5, 2), (7, 3), (9, 4)\}$. **Legend:** $\Pi_{\text{naive-dVOPRF}}^m$ [9] (dashed, $\blacksquare$); our Π_{dVOPRF}^m (solid, $\bullet$).

The naive $\Pi_{\text{DZKP}}^{\text{Aly}}$ baseline is uniformly dominated by all three of our protocols on both axes. Among our constructions, the end-to-end picture mirrors the offline phase: $\Pi_{\text{dVOPRF}}^{\text{DegGen}}$ has the cheapest communication at every profile but its round count grows super-linearly in (n, t) , yielding the steepest computation-time growth, while both $\Pi_{\text{dVOPRF}}^{\text{ZKPGen}}$ variants are constant-round and scale gracefully on computation time. $\Pi_{\text{L-dOPRF}}$ has very low computation at $(4, 1)$ but its per-query communication scales catastrophically, reaching ≈ 1.76 GB at $t=2$ and over 176 GB at $m=100$.

Specifically, $\Pi_{\text{dVOPRF}}^{\text{DegGen}}$ wins on both axes from $(3, 1)$ through $(7, 3)$ at $m=1$ and at small (n, t) for $m=100$. $\Pi_{\text{L-dOPRF}}$ outperforms our protocols only at $(4, 1)$, $m=1$, beating $\Pi_{\text{dVOPRF}}^{\text{Vith}}$ by $\approx 2.6\times$ on computation time but at the cost of $\approx 147\times$ more traffic; everywhere else our protocols dominate by orders of magnitude (e.g., $\approx 4,300\times$ on computation time at $t=2$, $m=100$). At $(9, 4)$, $\Pi_{\text{dVOPRF}}^{\text{Lig}}$ takes over thanks to its constant-round transcript, while $\Pi_{\text{dVOPRF}}^{\text{Vith}}$ remains competitive in the $(3, 1)$ – $(5, 2)$ regime where Ligerio is unavailable. Unlike $\Pi_{\text{L-dOPRF}}$, our protocols do not provide malicious-client security: we assume the client supplies well-formed RSS shares to the servers, and we do not address whether a malicious client could compromise the privacy of the servers' key. We leave this to future work.

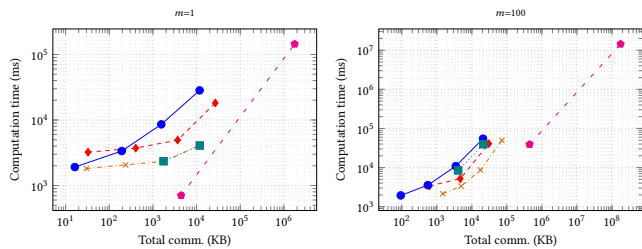


Figure 6: End-to-end phase: total communication vs. computation time. Our protocols and the Aly baseline trace $(n, t) \in \{(3, 1), (5, 2), (7, 3), (9, 4)\}$ left-to-right; $\Pi_{\text{L-dOPRF}}$ traces $(4, 1) \rightarrow (7, 2)$; $\Pi_{\text{dVOPRF}}^{\text{Lig}}$ traces $(7, 3) \rightarrow (9, 4)$. **Legend:** $\Pi_{\text{DZKP}}^{\text{Aly}}$ (dashed, $\blacklozenge$); $\Pi_{\text{dVOPRF}}^{\text{DegGen}}$ (solid, $\bullet$); $\Pi_{\text{dVOPRF}}^{\text{Vith}}$ (dash-dot, $\times$); $\Pi_{\text{dVOPRF}}^{\text{Lig}}$ (dotted, $\blacksquare$); $\Pi_{\text{L-dOPRF}}$ [28] (loosely dashed, pentagon).

7 Further Improvements

We briefly discuss further improvements for our dVOPRF protocol (see Appendix O for detailed discussions).

Saving Terms by Applying Local Share Conversion. In Π_{dVOPRF}^m , each server is assigned L terms of the product c . Using local share conversion, S_i can non-interactively share N/n of its terms, leaving only $L - N/n$ terms to run through Π_{VIP} .

Faster Online Protocol. If the servers' input, namely the shared PRF key $\llbracket k \rrbracket$, may be generated in the offline phase, a faster dVOPRF protocol can be realized based on Π_{dVOPRF}^m . For simplicity, fix $m = 1$. Servers run Π_{dVOPRF}^1 to compute $\llbracket c \rrbracket := \llbracket \alpha^e(r + k) \rrbracket$ in the offline phase, where $\llbracket \alpha^e \rrbracket$ is obtained from $\mathcal{F}_{\text{RandExp}}$, r is a random value, and the servers only hold $\llbracket r \rrbracket$. After receiving $\llbracket x \rrbracket$ from the client, servers compute $\llbracket \delta \rrbracket := \llbracket x \rrbracket + \llbracket r \rrbracket$ and open it. Servers locally compute $\llbracket v \rrbracket := \llbracket c \rrbracket + \delta \cdot \llbracket \alpha^e \rrbracket = \llbracket \alpha^e(x + k) \rrbracket$, and the client reconstructs v from $\llbracket v \rrbracket$ and computes $v^g \bmod p$ to obtain the desired output.

References

- [1] Martin R. Albrecht, Alex Davidson, Amit Deo, and Daniel Gardham. 2024. Crypto Dark Matter on the Torus - Oblivious PRFs from Shallow PRFs and TFHE. In *Advances in Cryptology - EUROCRYPT 2024 - 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques*, Zurich, Switzerland, May 26–30, 2024, *Proceedings, Part VI (Lecture Notes in Computer Science)*, Marc Joye and Gregor Leander (Eds.). Springer, 447–476. doi:10.1007/978-3-031-58751-1_16
- [2] Martin R. Albrecht, Alex Davidson, Amit Deo, and Nigel P. Smart. 2021. Round-Optimal Verifiable Oblivious Pseudorandom Functions from Ideal Lattices. In *Public-Key Cryptography - PKC 2021 - 24th IACR International Conference on Practice and Theory of Public Key Cryptography, Virtual Event, May 10–13, 2021, Proceedings, Part II (Lecture Notes in Computer Science)*, Juan A. Garay (Ed.). Springer, 261–289. doi:10.1007/978-3-030-75248-4_10
- [3] Martin R. Albrecht and Kamil Doruk Gür. 2024. Verifiable Oblivious Pseudorandom Functions from Lattices: Practical-Ish and Thresholdisable. In *Advances in Cryptology - ASIACRYPT 2024 - 30th International Conference on the Theory and Application of Cryptology and Information Security, Kolkata, India, December 9–13, 2024, Proceedings, Part IV (Lecture Notes in Computer Science)*, Kai-Min Chung and Yu Sasaki (Eds.). Springer, 205–237. doi:10.1007/978-981-96-0894-2_7
- [4] Abdelrahman Aly, Aysajan Abidin, and Svetla Nikova. 2018. Practically Efficient Secure Distributed Exponentiation Without Bit-Decomposition. In *Financial Cryptography and Data Security - 22nd International Conference, FC 2018, Nieuwpoort, Curaçao, February 26 - March 2, 2018, Revised Selected Papers (Lecture Notes in Computer Science)*, Sarah Meiklejohn and Kazuo Sako (Eds.). Springer, 291–309. doi:10.1007/978-3-662-58387-6_16
- [5] Scott Ames, Carmit Hazay, Yuval Ishai, and Muthuramakrishnan Venkatasubramanian. 2017. Ligerio: Lightweight Sublinear Arguments Without a Trusted Setup. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS 2017, Dallas, TX, USA, October 30 - November 03, 2017*, Bhavani Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu (Eds.). ACM, 2087–2104. doi:10.1145/3133956.3134104
- [6] Andrea Basso. 2023. A Post-Quantum Round-Optimal Oblivious PRF from Isogenies. In *Selected Areas in Cryptography - SAC 2023 - 30th International Conference, Fredericton, Canada, August 14–18, 2023, Revised Selected Papers (Lecture Notes in Computer Science)*, Claude Carlet, Kalikinkar Mandal, and Vincent Rijmen (Eds.). Springer, 147–168. doi:10.1007/978-3-031-53368-6_8
- [7] Carsten Baum, Lennart Braun, Cyprien Delpéch de Saint Guilhem, Michael Klooß, Emmanuela Orsini, Lawrence Roy, and Peter Scholl. 2023. Publicly Verifiable Zero-Knowledge and Post-Quantum Signatures from VOLE-in-the-Head. In *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20–24, 2023, Proceedings, Part V (Lecture Notes in Computer Science)*, Helena Handschuh and Anna Lysyanskaya (Eds.). Springer, 581–615. doi:10.1007/978-3-031-38554-4_19
- [8] Ward Beullens, Lucas Dodgson, Sebastian H. Faller, and Julia Hesse. 2025. The 2Hash OPRF Framework and Efficient Post-quantum Instantiations. In *Advances in Cryptology - EUROCRYPT 2025 - 44th Annual International Conference on the Theory and Applications of Cryptographic Techniques Madrid, Spain, May 4–8, 2025, Proceedings, Part VIII (Lecture Notes in Computer Science)*, Serge Fehr and Pierre-Alain Fouque (Eds.). Springer, 332–362. doi:10.1007/978-3-031-91101-9_12
- [9] Dan Boneh, Elette Boyle, Henry Corrigan-Gibbs, Niv Gilboa, and Yuval Ishai. 2019. Zero-Knowledge Proofs on Secret-Shared Data via Fully Linear PCPs. In

- Advances in Cryptology - CRYPTO 2019 - 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18–22, 2019, Proceedings, Part III (Lecture Notes in Computer Science)*, Alexandra Boldyreva and Daniele Micciancio (Eds.), Springer, 67–97. doi:10.1007/978-3-030-26954-8_3
- [10] Dan Boneh, Dmitry Kogan, and Katharine Woo. 2020. Oblivious Pseudorandom Functions from Isogenies. In *Advances in Cryptology - ASIACRYPT 2020 - 26th International Conference on the Theory and Application of Cryptology and Information Security, Daejeon, South Korea, December 7–11, 2020, Proceedings, Part II (Lecture Notes in Computer Science)*, Shihō Moriai and Huaxiong Wang (Eds.). Springer, 520–550. doi:10.1007/978-3-030-64834-3_18
- [11] Elette Boyle, Niv Gilboa, Yuval Ishai, and Ariel Nof. 2020. Efficient Fully Secure Computation via Distributed Zero-Knowledge Proofs. In *Advances in Cryptology - ASIACRYPT 2020 - 26th International Conference on the Theory and Application of Cryptology and Information Security, Daejeon, South Korea, December 7–11, 2020, Proceedings, Part III (Lecture Notes in Computer Science)*, Shihō Moriai and Huaxiong Wang (Eds.). Springer, 244–276. doi:10.1007/978-3-030-64840-4_9
- [12] Elette Boyle, Niv Gilboa, Yuval Ishai, and Ariel Nof. 2020. Efficient Fully Secure Computation via Distributed Zero-Knowledge Proofs. *IACR Cryptol. ePrint Arch.* 2020 (2020), 1451. <https://eprint.iacr.org/2020/1451>
- [13] Koji Chida, Koki Hamada, Dai Ikarashi, Ryo Kikuchi, Daniel Genkin, Yehuda Lindell, and Ariel Nof. 2023. Fast large-scale honest-majority MPC for malicious adversaries. *Journal of Cryptology* 36, 3 (2023), 15.
- [14] Ronald Cramer, Ivan Damgård, and Yuval Ishai. 2005. Share Conversion, Pseudorandom Secret-Sharing and Applications to Secure Computation. In *Theory of Cryptography, Second Theory of Cryptography Conference, TCC 2005, Cambridge, MA, USA, February 10–12, 2005, Proceedings (Lecture Notes in Computer Science)*, Joe Kilian (Ed.). Springer, 342–362. doi:10.1007/978-3-540-30576-7_19
- [15] Ivan Damgård. 1990. On the Randomness of Legendre and Jacobi Sequences. In *Proceedings of the 8th Annual International Cryptology Conference on Advances in Cryptology (CRYPTO '88)*. Springer-Verlag, London, UK, 163–172. <http://dl.acm.org/citation.cfm?id=646753.704912>
- [16] Ivan Damgård and Jesper Buus Nielsen. 2007. Scalable and unconditionally secure multiparty computation. In *Annual International Cryptology Conference*. Springer, 572–590.
- [17] Sebastian H. Faller, Astrid Ottenhues, and Johannes Ottenhues. 2023. Composable Oblivious Pseudo-Random Functions via Garbled Circuits. In *Progress in Cryptology - LATINCRYPT 2023 - 8th International Conference on Cryptology and Information Security in Latin America, LATINCRYPT 2023, Quito, Ecuador, October 3–6, 2023, Proceedings (Lecture Notes in Computer Science)*, Abdelrahman Aly and Mehdi Tibouchi (Eds.). Springer, 249–270. doi:10.1007/978-3-031-44469-2_13
- [18] Uriel Feige, Joe Kilian, and Moni Naor. 1994. A minimal model for secure computation (extended abstract). In *Proceedings of the Twenty-Sixth Annual ACM Symposium on Theory of Computing, 23–25 May 1994, Montréal, Québec, Canada*, Frank Thomson Leighton and Michael T. Goodrich (Eds.). ACM, 554–563. doi:10.1145/195058.195408
- [19] Amos Fiat and Adi Shamir. 1986. How to Prove Yourself: Practical Solutions to Identification and Signature Problems. In *Advances in Cryptology - CRYPTO '86, Santa Barbara, California, USA, 1986, Proceedings (Lecture Notes in Computer Science)*, Andrew M. Odlyzko (Ed.). Springer, 186–194. doi:10.1007/3-540-47721-7_12
- [20] L. R. Ford and D. R. Fulkerson. 1956. Maximal Flow Through a Network. *Canadian Journal of Mathematics* 8 (1956), 399–404. doi:10.4153/CJM-1956-045-5
- [21] Michael J. Freedman, Yuval Ishai, Benny Pinkas, and Omer Reingold. 2005. Keyword Search and Oblivious Pseudorandom Functions. In *Theory of Cryptography*, Joe Kilian (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 303–324.
- [22] Andrew V. Goldberg and Satish Rao. 1998. Beyond the Flow Decomposition Barrier. *J. ACM* 45, 5 (1998), 783–797. doi:10.1145/290179.290181
- [23] Vipul Goyal, Hanjun Li, Rafail Ostrovsky, Antigoni Polychroniadou, and Yifan Song. 2021. ATLAS: Efficient and Scalable MPC in the Honest Majority Setting. In *Advances in Cryptology - CRYPTO 2021 (Lecture Notes in Computer Science)*, Tal Malkin and Chris Peikert (Eds.). Springer, 244–274. doi:10.1007/978-3-030-84245-1_9
- [24] Lorenzo Grassi, Christian Rechberger, Dragos Rotaru, Peter Scholl, and Nigel P. Smart. 2016. MPC-Friendly Symmetric Key Primitives. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, Vienna, Austria, October 24–28, 2016*, Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi (Eds.). ACM, 430–443. doi:10.1145/2976749.2978332
- [25] Lena Heimberger, Tobias Hennerbichler, Fredrik Meisingseth, Sebastian Ramacher, and Christian Rechberger. 2024. OPRFs from Isogenies: Designs and Analysis. In *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security, ASIA CCS 2024, Singapore, July 1–5, 2024*, Jianying Zhou, Tony Q. S. Quek, Debin Gao, and Alvaro A. Cárdenas (Eds.). ACM. doi:10.1145/3634737.3645010
- [26] Stanislaw Jarecki, Aggelos Kiayias, and Hugo Krawczyk. 2014. Round-Optimal Password-Protected Secret Sharing and T-PAKE in the Password-Only Model. In *Advances in Cryptology - ASIACRYPT 2014 - 20th International Conference on the Theory and Application of Cryptology and Information Security, Kaoshiung, Taiwan, R.O.C., December 7–11, 2014, Proceedings, Part II (Lecture Notes in Computer Science)*, Palash Sarkar and Tetsu Iwata (Eds.). Springer, 233–253. doi:10.1007/978-3-662-45608-8_13
- [27] Stanislaw Jarecki, Aggelos Kiayias, Hugo Krawczyk, and Jiayu Xu. 2017. TOPPSS: Cost-Minimal Password-Protected Secret Sharing Based on Threshold OPRF. In *Applied Cryptography and Network Security - 15th International Conference, ACNS 2017, Kanazawa, Japan, July 10–12, 2017, Proceedings (Lecture Notes in Computer Science)*, Dieter Gollmann, Atsuko Miyaji, and Hiroaki Kikuchi (Eds.). Springer, 39–58. doi:10.1007/978-3-319-61204-1_3
- [28] Novak Kaluderović, Nan Cheng, and Katerina Mitrokovska. 2025. A post-quantum Distributed OPRF from the Legendre PRF. In *Computer Security - ESORICS 2025*. Springer. Preprint available as Cryptology ePrint Archive, Report 2024/544, <https://eprint.iacr.org/2024/544>.
- [29] Marcel Keller. 2020. MP-SPDZ: A Versatile Framework for Multi-Party Computation. In *CCS '20: 2020 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, USA, November 9–13, 2020*, Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna (Eds.). ACM, 1575–1590. doi:10.1145/3372297.3417872
- [30] Bart Mennink. 2023. Secure Distributed Modular Exponentiation: Systematic Analysis and New Results. *IEEE Transactions on Information Forensics and Security* 18 (2023), 4188–4204. doi:10.1109/TIFS.2023.3293396
- [31] Stephen C. Pohlig and Martin E. Hellman. 1978. An improved algorithm for computing logarithms over GF(p) and its cryptographic significance (Corresp.). *IEEE Trans. Inf. Theory* 24, 1 (1978), 106–110. doi:10.1109/TIT.1978.1055817
- [32] István András Seres, Máté Horváth, and Péter Burcsi. 2025. The Legendre pseudorandom function as a multivariate quadratic cryptosystem: security and applications. *Appl. Algebra Eng. Commun. Comput.* 36, 2 (2025), 223–253. doi:10.1007/S00200-023-00599-2
- [33] Kang Yang, Pratik Sarkar, Chenkai Weng, and Xiao Wang. 2021. QuickSilver: Efficient and Affordable Zero-Knowledge Proofs for Circuits and Polynomials over Any Field. In *CCS '21: 2021 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, Republic of Korea, November 15 – 19, 2021*, Yongdae Kim, Jong Kim, Giovanni Vigna, and Elaine Shi (Eds.). ACM, 2986–3001. doi:10.1145/3460120.3484556
- [34] Yibin Yang, Fabrice Benhamouda, Shai Halevi, Hugo Krawczyk, and Tal Rabin. 2025. Gold OPRF: Post-Quantum Oblivious Power-Residue PRF. In *IEEE Symposium on Security and Privacy, SP 2025, San Francisco, CA, USA, May 12–15, 2025*, Marina Blanton, William Enck, and Cristina Nita-Rotaru (Eds.). IEEE, 259–278. doi:10.1109/SP61157.2025.00116

A Open Science

In line with the CCS open-science policy, we describe below all artifacts needed to evaluate the core contributions of this paper and how they can be accessed.

Artifacts. The sole artifact accompanying this paper is a source-code repository containing:

- Rust implementations of the offline-phase protocols (degenerate additive encoding and the $t+1$ designated-dealer NIZK approaches, instantiated with both VOLE-in-the-Head and Liger), and of the online-phase constant-round protocol with integrated DZKP verification.
- Microbenchmarks reproducing the end-to-end communication and runtime numbers reported in Section 6, together with the analytical byte-counting harness used as a cross-check.
- Build and run instructions, and the configuration files required to reproduce all parameter regimes $(n, t) \in \{(3, 1), (5, 2), (7, 3), (9, 4)\}$ evaluated in the paper.

Access. The repository is publicly accessible; no credentials are required.⁷

Restricted or unavailable artifacts. None. The submission introduces no datasets, models, or proprietary components, and uses no

⁷[De-anonymization pending: replace the anonymized review link <https://anonymous.4open.science/r/Verifiable-Distributed-OPRF-43C0/README.md> with the permanent public repository URL (and archival DOI, if minted).]

human-subject or otherwise sensitive data. All software artifacts required to evaluate the paper’s contributions are released openly under a permissive license in the repository above.

B Ethical Considerations

B.1 Stakeholder Identification and Impact

Research team. Our work is purely theoretical and cryptographic: we use no human subjects, conduct no physical experiments, and collect no real-world private data. All evaluation is performed through complexity analysis and microbenchmarks on synthetic inputs.

Direct and indirect stakeholders. The primary stakeholders are companies and researchers active in cryptography and cybersecurity who can leverage our advancements to design and improve privacy-preserving implementations (e.g., PAKE, PSI, oblivious key management). Indirect stakeholders are end-users of such deployed systems, who benefit from stronger, post-quantum, distributed-trust guarantees.

B.2 Cryptography Community and Society

Beneficence. We contribute a high-throughput, post-quantum verifiable distributed OPRF, batched offline protocols for RSS exponentiation, and a tightly integrated online DZKP. These directly benefit the research community and industry by enabling more efficient privacy-preserving application deployments.

Respect for persons. We have taken care to give proper attribution to prior and concurrent work throughout, citing all foundational and related results that informed our constructions.

Justice. We pursue open research: nothing in this work is proprietary, and we strive to explain every construction, assumption, and analytical step in sufficient detail to allow independent scrutiny and reuse.

Respect for law and public interest. Our research complies with established academic and cryptographic publication policies, and its goals of improved efficiency, verifiability, and post-quantum security of distributed OPRFs align with the public interest in robust privacy-preserving infrastructure.

Publication impacts. We anticipate the impact of this work to manifest both in concrete implementations of distributed OPRFs and in further theoretical advancements building on our offline RSS exponentiation and online DZKP techniques.

B.3 Potential Harm Analysis

We acknowledge that, like any cryptographic primitive offering confidentiality and obliviousness, our protocols could in principle be misused by malicious actors to secretly compute functions that conceal illicit activity or harm society. Honest users additionally face risk if a sufficiently large fraction of servers (beyond the assumed threshold $t < n/2$) is controlled by an adversary. We urge that deployments of our protocols proceed in accordance with applicable legal and regulatory frameworks; to this end, we have explicitly documented all security assumptions, threat models, and corruption thresholds underlying our constructions.

C Generative AI Usage

We primarily used three AI tools during this project: Claude Sonnet (4.6), Claude Opus (4.6, 4.7), and Microsoft Copilot. Per regulation, we describe below the two phases of the project in which these tools played a non-trivial role.

C.1 Implementation

Our first substantial use was for code generation. The initial code design was produced manually to remain faithful to our theoretical constructions. Claude was then used to generate the initial code filling out this structure. To guard against hallucinated APIs or protocol steps, and to ensure the code actually implemented our protocols, we took the following three steps in order:

- (1) **Unit tests** were written for each protocol and sub-routine, covering input/output behavior as well as intermediate computation states.
- (2) **Manual debugging** of the codebase was carried out iteratively over several days. All bug identification and fixes were performed exclusively by humans, supported by unit tests, peer code review, and rubber-duck debugging (debugging with a colleague).
- (3) **Line-by-line comparison against the theoretical specification** served as the final check for all of our protocols, as well as for the naive baseline that we implemented ourselves. The code was rigorously compared against our theoretical contributions and challenged at every step until it was deemed faithful to the theory.

C.2 Text editing

AI tools were used in a more limited manner during writing: to generate placeholder text in the early stages of drafting, and to assist with grammar checks and tightening content in the final stages of editing. In the first case, all such text was subsequently revised and reviewed by several team members through an iterative process. No AI-generated text remains in the final submitted version. In the second case, AI assistance was applied strictly to verified, human-written text.

As a team, we do not believe current AI tools can reliably present our contributions with the precision and concision that this work requires. Our use of AI in writing was therefore confined to repetitive and standardized tasks, so that we could (1) clearly and effectively communicate our contributions and (2) make explicit the constructions, assumptions, and techniques on which they rest.

D Secret Sharing Schemes

A t -out-of- n secret sharing scheme, where a secret value s is distributed across n shares, denoted by $\llbracket s \rrbracket = \{\llbracket s \rrbracket_1, \dots, \llbracket s \rrbracket_n\}$, and can be reconstructed from any combination of at least $t + 1$ shares, while any t shares leak no information about s . For any shares $\llbracket s \rrbracket$, we implicitly assume that party P_i is holding $\llbracket s \rrbracket_i$, and computing $f(\llbracket s \rrbracket)$ for any function f means that each party P_i computes $f(\llbracket s \rrbracket_i)$. We use the notation $\llbracket s \rrbracket_I := \{\llbracket s \rrbracket_i\}_{i \in I}$ for a subset of shares corresponding to $I \subset [n]$.

Formally, there are two interactive protocols: $\text{Share}(s)$, which takes the secret s as input and outputs n shares $\llbracket s \rrbracket$ for all parties, and $\text{Reconstruct}(\llbracket s \rrbracket_Q, i)$, which takes $\llbracket s \rrbracket_Q$ and reveals the secret

to party P_i by sending their shares to P_i . The latter one outputs the secret s only if $\#Q \geq t + 1$. We say that shares are *consistent* if $\text{Reconstruct}(\llbracket s \rrbracket_{Q_0}, i) = \text{Reconstruct}(\llbracket s \rrbracket_{Q_1}, i)$ for any two sets of honest parties $Q_0, Q_1 \subset [n]$ such that $\#Q_b \geq t + 1$ for all $b \in \{0, 1\}$. Plus, an interactive algorithm $\text{Open}(\llbracket s \rrbracket)$ takes consistent shares $\llbracket s \rrbracket$ and outputs the secret s to all parties.

D.1 Local Linear Operations and Share Conversion

In this work, we consider the secret sharing scheme that supports local linear operations, such as $\llbracket a + b \rrbracket := \llbracket a \rrbracket + \llbracket b \rrbracket$, $\llbracket c + a \rrbracket := c + \llbracket a \rrbracket$, and $\llbracket c \cdot a \rrbracket := c \cdot \llbracket a \rrbracket$. The *local share conversion* [14] allows parties to locally generate shares $\llbracket [s]_i \rrbracket$ of the i -th share for any $i \in [n]$.

D.2 Additive Secret Sharing

Additive secret sharing (ASS) is an $(n - 1)$ -out-of- n secret sharing scheme supporting the local linear operations: a secret $s \in \mathbb{F}_p$ is divided into n random field elements, called *additive shares*, denoted by $[s]_1, \dots, [s]_n$. The secret s can be recovered by computing $s = \sum_{i=1}^n [s]_i$ over $\mathbb{F}_p$. A multiplicative variant $[s]^*$ can also be defined as $s = \prod_{i=1}^n [s]_i^*$ over $\mathbb{F}_p^*$.

D.3 Replicated Secret Sharing

Replicated secret sharing (RSS) generalizes ASS to handle an arbitrary threshold t by providing the same additive shares of s to multiple parties. Consider an ASS scheme that produces $N = \binom{n}{t}$ shares $[s]_1, \dots, [s]_N$ indexed by a set $\mathcal{T} := \{T \subset [n] \mid \#T = t\} = \{T_1, \dots, T_N\}$. A secret s is shared as *RSS shares* $\llbracket s \rrbracket = \{\llbracket s \rrbracket_1, \dots, \llbracket s \rrbracket_N\}$, defined by $\llbracket s \rrbracket_i := \{[s]_j \mid i \notin T_j \in \mathcal{T}\}$ for all $i \in [n]$. Therefore, each RSS share consists of $N' = \binom{n-1}{t}$ field elements. Any $t + 1$ RSS shares $\{\llbracket s \rrbracket_{i_1}, \dots, \llbracket s \rrbracket_{i_{t+1}}\}$ cover all additive shares, resulting in recovering s , whereas any set of at most t RSS shares does not cover at least one additive share.

A note on notation. We write $[s]_\ell$ with $\ell \in [N]$ for the additive component of an RSS sharing indexed by $T_\ell \in \mathcal{T}$, and $[s]_i$ with $i \in [n]$ for the i -th share of a standalone $(n - 1)$ -out-of- n additive sharing (as produced by $\mathcal{F}_{\text{Zero}}$, or as the first component of a double sharing $([r], \llbracket r \rrbracket)$). The two are distinct objects—there are $N = \binom{n}{t}$ of the former and n of the latter—and we reserve ℓ for component indices and i for party indices throughout.

The local linear operations are obviously allowed in RSS. Besides, RSS also supports the local share conversion as follows. Party P_i is arbitrarily fixed. For every $T \in \mathcal{T}$ such that $i \notin T$, let $\llbracket [s]_T \rrbracket_j := [s]_T$ for all $j \notin T$ and $\llbracket [s]_T \rrbracket_j := 0$ for all $j \in T$. Therefore, every party P_j can locally compute $\llbracket [s]_i \rrbracket_j := \{\llbracket [s]_T \rrbracket_j \mid j \notin T \in \mathcal{T}\}$.

A multiplicative RSS $\llbracket s \rrbracket^*$ can also be defined by replacing the underlying additive shares $[s]$ with their multiplicative variant $[s]^*$.

D.4 Verifiable Secret Sharing

Verifiable secret sharing (VSS) guarantees that at the end of the Share protocol, either the parties hold consistent shares of a secret or the honest parties abort the protocol. This can be achieved by an additional consistency check after each party receives their share from the dealer.

E Building Block Protocols for RSS

This appendix gives the concrete realizations of the RSS building-block functionalities introduced in Section 2.2. We first recall the ideal functionalities themselves:

- $\mathcal{F}_{\text{Rand}}$: hands parties shares of a random secret value, while allowing the adversary to choose the corrupted parties' shares.
- $\mathcal{F}_{\text{Coin}}$: hands parties fresh random coins.
- $\mathcal{F}_{\text{RSS.Share}}$: on input a secret $s \in \mathbb{F}_p$ from a dealer, outputs $\llbracket s \rrbracket$ to parties.
- $\mathcal{F}_{\text{RSS.Open}}$: on input $\llbracket s \rrbracket$ from parties, outputs s to all parties, or $\perp$ if the shares are inconsistent (security with abort).
- $\mathcal{F}_{\text{A2R}}$: on input an additive sharing $[s]$ from parties, outputs a replicated sharing $\llbracket s \rrbracket$ of the same value.
- $\mathcal{F}_{\text{RSS.Mul}}$: on input $\llbracket a \rrbracket, \llbracket b \rrbracket$ from parties, outputs $\llbracket ab \rrbracket$.
- $\mathcal{F}_{\text{DoubleRand}}$: samples a uniformly random $r \in \mathbb{F}_p$ and outputs a double sharing $([r], \llbracket r \rrbracket)$ to parties.
- $\mathcal{F}_{\text{Zero}}$: hands parties additive shares of zero.

Next, we present building blocks used throughout our work one by one as follows.

Protocol 7 $\Pi_{\text{RSS.Share}} - (t, n)$ -RSS Sharing

Players: A dealer D and servers $S_1, \dots, S_n$.

Input: Dealer D holds a secret $s \in \mathbb{F}_p$; servers hold a pre-computed double sharing $([r], \llbracket r \rrbracket)$ of a random $r \in \mathbb{F}_p$.

- 1: Each server S_i sends $[r]_i$ to D .
- 2: D reconstructs $r \leftarrow \sum_{i=1}^n [r]_i$ and sends $\delta \leftarrow s - r$ to each server S_j for $j \in Q$, where $Q \subset [n]$ with $\#Q = t + 1$ is a fixed public subset.
- 3: Each server S_j for $j \in Q$ locally updates $\llbracket s \rrbracket_j$ by adding δ to the appropriate share component of $\llbracket r \rrbracket_j$. Each server S_i for $i \notin Q$ sets $\llbracket s \rrbracket_i \leftarrow \llbracket r \rrbracket_i$.

Output: Each server S_i holds a share $\llbracket s \rrbracket_i$.

Protocol 7 requires 2 rounds of communication: one round where each server sends 1 field element (its additive share $[r]_i$) to the dealer D , and one round where D sends 1 field element δ to the $t + 1$ servers in Q .

Protocol 8 $\Pi_{\text{RSS.Open}} - (t, n)$ -RSS Opening

Players: Servers $S_1, \dots, S_n$.

Input: A public access policy $\mathcal{A}$. Each server S_i holds $\llbracket s \rrbracket_{\mathcal{T}_i}$ which is an ordered list of S_i 's local shares.

- 1: $\forall i \in [n]$, server S_i computes $h_i \leftarrow H(\llbracket s \rrbracket_{\mathcal{T}_i})$, where $H: \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$.
- 2: Each server S_i sends h_i and $\llbracket s \rrbracket_{\mathcal{A}_i}$ (a list of shares defined by the public access policy $\mathcal{A}$) to each S_j for $j \in [n] \setminus \{i\}$.
- 3: Each server S_j reconstructs s from the received shares together with its own local shares.
- 4: Each server S_j verifies $h_i \stackrel{?}{=} H(\llbracket s \rrbracket_{\mathcal{T}_i})$ for all $i \in [n]$. // abort if any check fails

Output: All servers learn s , or $\perp$.

Protocol 8 requires 1 round of communication. For RSS, each server S_j receives $\binom{n-1}{t-1}$ share components from other servers, plus

$n - 1$ hash values. The hash verification enables security with abort: after reconstruction, each server S_j can determine every other server's full local share from the complete share information and check it against the received hash, detecting any inconsistency.

E.1 Verifiable Client Input Sharing

The online protocol of §5 requires the client to hand the servers an RSS sharing $\llbracket x \rrbracket$ of its evaluation point. A *direct* sharing—the client generates $\llbracket x \rrbracket$ itself and sends each S_i its list $\llbracket x \rrbracket_i$ —is not verifiable: a malicious client can send mismatched copies of a replicated component $\llbracket x \rrbracket_\ell$ to two servers that both hold it, and nothing detects the inconsistency. We therefore give Π_{Input} (Protocol 9), which realizes $\mathcal{F}_{\text{RSS,Share}}$ with abort for a *client* dealer, at strictly lower communication cost than the direct sharing.

Π_{Input} follows the mask-and-broadcast pattern: the servers hold a pre-computed RSS sharing $\llbracket r \rrbracket$ of a uniform r , open r to the client, and the client broadcasts $u := r + x$, from which every server derives $\llbracket x \rrbracket := u - \llbracket r \rrbracket$ locally. Two details make it verifiable. *First*, r is opened *with abort* rather than by plain reconstruction: each server also sends a hash of its full local list, and the client—having assembled every additive component—recomputes each server's list and checks it, exactly as in $\Pi_{\text{RSS,Open}}$. Without this check a single corrupted server could send a wrong component, the client would reconstruct some $r' \neq r$, and the servers would end up holding $\llbracket x + (r - r') \rrbracket$: an honest client's input silently shifted by the adversary. *Second*, u must reach *all* n servers identically; the servers therefore echo a hash of u and abort on disagreement, which is what forces a malicious client to commit to a single input. When Π_{Input} is composed into Π_{dVOPRF}^m , this echo rides the hash exchange that protocol already performs, and so costs no additional round.

Each $\llbracket r^{(j)} \rrbracket$ must be consumed by *exactly one* input: reusing a mask across $x^{(1)}$ and $x^{(2)}$ reveals $u^{(1)} - u^{(2)} = x^{(1)} - x^{(2)}$ to every server.

Protocol 9 Π_{Input} – Verifiable (t, n) -RSS Sharing of a Client Input

Players: Client C and servers $S_1, \dots, S_n$.

Input: C holds $(x^{(1)}, \dots, x^{(m)}) \in \mathbb{F}_p^m$; servers hold pre-computed RSS sharings $\llbracket r^{(1)} \rrbracket, \dots, \llbracket r^{(m)} \rrbracket$ of independent uniform $r^{(j)} \in \mathbb{F}_p$, each used exactly once. A public assignment $P: [N] \rightarrow [n]$ with $P(\ell) \notin T_\ell$ for every $\ell \in [N]$.

- 1: Each server S_i sends $\{\llbracket r^{(j)} \rrbracket_\ell \mid j \in [m], \ell \in [N], P(\ell) = i\}$ and $\psi_i \leftarrow H(\llbracket r^{(1)} \rrbracket_i \parallel \dots \parallel \llbracket r^{(m)} \rrbracket_i)$ to C .
- 2: C assembles $\llbracket r^{(j)} \rrbracket_\ell$ for all $j \in [m], \ell \in [N]$, recomputes each server's local list $\llbracket r^{(j)} \rrbracket_i = \{\llbracket r^{(j)} \rrbracket_\ell \mid i \notin T_\ell\}$, and verifies $\psi_i \stackrel{?}{=} H(\llbracket r^{(1)} \rrbracket_i \parallel \dots \parallel \llbracket r^{(m)} \rrbracket_i)$ for all $i \in [n]$. // abort and output $\perp$ if any check fails
- 3: C computes $r^{(j)} \leftarrow \sum_{\ell=1}^N \llbracket r^{(j)} \rrbracket_\ell$ and $u^{(j)} \leftarrow r^{(j)} + x^{(j)}$ for every $j \in [m]$, and sends $(u^{(1)}, \dots, u^{(m)})$ to all n servers.
- 4: Each server S_i computes $\chi_i \leftarrow H(u^{(1)} \parallel \dots \parallel u^{(m)})$ from the values it received and sends χ_i to each $S_{i'}$ for $i' \in [n] \setminus \{i\}$. // abort if $\chi_{i'} \neq \chi_i$ for some i'
- 5: Each server S_i locally sets $\llbracket x^{(j)} \rrbracket_i \leftarrow u^{(j)} - \llbracket r^{(j)} \rrbracket_i$ for every $j \in [m]$.

Output: Each server S_i holds $\llbracket x^{(j)} \rrbracket_i$ for every $j \in [m]$; or the honest parties output $\perp$.

Standalone, Protocol 9 requires 3 rounds; composed into Π_{dVOPRF}^m the echo of Step 4 is merged into an existing hash exchange, leaving 2. Because every additive component is sent by exactly one designated server, C receives mN field elements and n hash values in total, and sends m field elements to each server. A direct sharing instead costs mnN' field elements from C ; since $N' = \binom{n-1}{t}$ grows combinatorially in n while the client now uploads a single element per server per input, Π_{Input} is cheaper for every (n, t) we consider and increasingly so as n grows.

LEMMA E.1. *Let $t < n/2$ and let H be collision resistant. In the presence of a static adversary corrupting up to t servers and, possibly, the client C , Protocol 9 satisfies the following.*

- (1) (Consistency) *If the honest servers do not abort, then for every $j \in [m]$ there is a unique $x^{(j)} \in \mathbb{F}_p$ such that the honest servers' outputs are consistent components of a single RSS sharing $\llbracket x^{(j)} \rrbracket$ of $x^{(j)}$.*
- (2) (Input integrity) *If moreover C is honest and does not abort, then $x^{(j)}$ is exactly the input C supplied.*
- (3) (Extractability) *There is an efficient extractor that, given the honest servers' views, outputs the $(x^{(1)}, \dots, x^{(m)})$ of item 1.*
- (4) (Privacy) *If C is honest, the adversary's view is independent of $(x^{(1)}, \dots, x^{(m)})$.*

PROOF. *Consistency.* Suppose no honest server aborts at Step 4. Every honest server compares the full vector $(\chi_1, \dots, \chi_n)$ against its own value, so non-abort means all these hashes agree; by collision resistance of H , all honest servers therefore hold the same $(u^{(1)}, \dots, u^{(m)})$. By assumption the servers entered the protocol holding consistent components of a single RSS sharing $\llbracket r^{(j)} \rrbracket$. Step 5 applies the same public affine map $z \mapsto u^{(j)} - z$ at every honest server, so the resulting shares are consistent components of the RSS sharing of $x^{(j)} := u^{(j)} - r^{(j)}$, which is uniquely determined.

Input integrity. Suppose C is honest and passes Step 2, and suppose for contradiction that some corrupted designated sender $S_{P(\ell)}$ sent $[r^{(j)}]_\ell \neq [r^{(j)}]_\ell$. The component indexed by ℓ is held by the $n - t \geq t + 1$ servers in $[n] \setminus T_\ell$, of which at most t are corrupted; let S_{i^*} be an honest one. Then S_{i^*} sent ψ_{i^*} computed over its true local lists, whereas the list C recomputes for S_{i^*} contains $[r^{(j)}]_\ell$ and hence differs. By collision resistance the check for i^* fails and C aborts, a contradiction. Hence C reconstructs the true $r^{(j)}$, and $u^{(j)} - r^{(j)}$ is the input C supplied.

Extractability. The honest servers hold $[r^{(j)}]_i$ for $n - t \geq t + 1$ distinct i , which together cover all N additive components; the extractor reconstructs $r^{(j)}$ from them, reads $u^{(j)}$ off any honest server's Step-3 transcript, and outputs $u^{(j)} - r^{(j)}$.

Privacy. The adversary holds at most t RSS shares of $r^{(j)}$, which by the RSS access structure omit at least one additive component $[r^{(j)}]_\ell$. That component is uniform and independent of the adversary's view, so $r^{(j)}$ is uniform in that view and $u^{(j)} = r^{(j)} + x^{(j)}$ is uniform and independent of $x^{(j)}$. As each mask is consumed by exactly one input, this holds jointly over $j \in [m]$. The remaining messages the adversary observes, $\{\psi_i\}$ and $\{\chi_i\}$, are functions of $[r^{(j)}]$ and $(u^{(j)})_{j \in [m]}$ alone. $\square$

Protocol 10 Π_{A2R} – Additive-to-Replicated Sharing Conversion

Players: Servers $S_1, \dots, S_n$.

Input: Servers hold an additive sharing $[s]$; a pre-computed double sharing $([r], [r])$ of a random $r \in \mathbb{F}_p$.

- 1: Each server S_i locally computes $[s - r]_i \leftarrow [s]_i - [r]_i$.
- 2: Each server S_i sends $[s - r]_i$ to a designated aggregator S_q , where $q \in Q$ and $Q \subset [n]$ with $\#Q = t + 1$ is a fixed public subset.
- 3: S_q reconstructs $\delta \leftarrow \sum_{i=1}^n [s - r]_i$ and sends δ to each server S_j for $j \in Q \setminus \{q\}$.
- 4: Each server S_j for $j \in Q$ locally updates $[s]_j$ by adding δ to the appropriate share component of $[r]_j$. Each server S_i for $i \notin Q$ sets $[s]_i \leftarrow [r]_i$.

Output: Each server S_i holds $[s]_i$.

Protocol 10 requires 2 rounds of communication: in the first round, each server sends 1 field element to the aggregator S_q ; in the second round, S_q sends 1 field element δ to each of the t other servers in Q . Only the $t + 1$ servers in Q update their shares; the remaining $n - t - 1$ servers keep their shares of r unchanged. The double sharing $([r], [r])$ is assumed to be pre-computed in the offline phase.

Protocol 11 $\Pi_{RSS, \text{Mul}}$ – Multiplication over (t, n) -Replicated Secret Sharing

Players: Servers $S_1, \dots, S_n$.

Input: Servers hold $[a]$ and $[b]$; a pre-computed double sharing $([r], [r])$ of a random $r \in \mathbb{F}_p$.

- 1: Each server S_i locally computes an additive share $[ab]_i$ from its local shares $[a]_i$ and $[b]_i$ via cross-multiplication.
- 2: Servers invoke Π_{A2R} over $[ab]$ to obtain $[ab]$.

Output: Each server S_i holds $[ab]_i$.

Protocol 11 requires 2 rounds of communication (from the invocation of Π_{A2R}): each server sends 1 field element to the aggregator S_q , who then forwards δ to the t other servers in Q . The overall communication overhead is $\frac{3t+1}{2t+1} \approx 1.5$ elements per party.

Generating Double Sharings. Both $\Pi_{RSS, \text{Share}}$ and $\Pi_{RSS, \text{Mul}}$ consume a pre-computed double sharing $([r], [r])$ of a random value $r \in \mathbb{F}_p$. We describe a non-interactive offline protocol for generating such pairs, following the approach of Cramer et. al [14].

Protocol 12 $\Pi_{\text{DoubleRand}}$ – Non-Interactive Double Sharing Generation

Players: Servers $S_1, \dots, S_n$.

Pre-shared material: Replicated PRF keys $\{k_T\}_{T \in \mathcal{T}}$, where each key k_T is held by all servers S_i with $i \notin T$; $\forall i, j \in [n]$, pairwise PRG seeds $\{s_{i,j}\}$ shared between every pair of servers S_i, S_j .

Input: A public counter ctr.

// Step 1 – RSS component

- 1: For each $T \in \mathcal{T}$ with $i \notin T$, server S_i locally computes $[r]_T \leftarrow \text{PRF}_{k_T}(\text{ctr})$.
- 2: Each server S_i sets its threshold share $[r]_i \leftarrow \{[r]_T \mid i \notin T\}$.

// Step 2 – Re-randomized additive share

- 3: Let $P: \mathcal{T} \rightarrow [n]$ be a public covering policy satisfying $P(T) \notin T$ for every $T \in \mathcal{T}$. Each server S_i locally computes:

$$[r]_i \leftarrow \sum_{T \in \mathcal{T}: P(T)=i} [r]_T + \sum_{\substack{j \in [n] \\ j \neq i}} (-1)^{1[j < i]} \text{PRG}(s_{i,j}, \text{ctr}).$$

Output: Servers hold $([r], [r])$ where $r = \sum_{T \in \mathcal{T}} [r]_T$.

Protocol 12 is non-interactive: all computations are local. Each server S_i evaluates the PRF for each $T \in \mathcal{T}$ with $i \notin T$, yielding $\binom{n-1}{t}$ PRF evaluations per server. The base additive share b_i is obtained by summing the RSS components assigned to S_i via the covering policy P : component $[r]_T$ is assigned to $S_{P(T)}$, which holds key k_T since $P(T) \notin T$, and each component is counted exactly once since P is a function. Since the PRF keys are replicated (all parties outside T hold k_T), the base shares b_i are deterministic and could be recomputed by other parties holding the same keys. The zero-sharing $\{z_i\}$ from pairwise PRG seeds re-randomizes the additive shares: $\sum_i z_i = 0$ by construction (each pairwise PRG term appears once with $+$ and once with $-$), so correctness is preserved ($\sum_i [r]_i = \sum_i b_i = r$), while individual shares $[r]_i = b_i + z_i$ are computationally indistinguishable from uniform under $t < n/2$ corruptions, since at least one pairwise PRG term in each corrupt party's z_i involves an honest party's seed.

F Distributed Zero-Knowledge Proofs

We use distributed zero-knowledge proofs to verify degree-2 relations over RSS-shared values. In particular, our offline generation protocols invoke an ideal functionality $\mathcal{F}_{\text{provDeg2Rel}}^{\text{abort}}$ to certify that a batch of multiplication relations was computed correctly. This functionality is adapted from the distributed zero-knowledge framework of Boyle et al. [11], and is formalized in Fig. 7.

At a high level, the functionality receives RSS sharings $[c]$, $\{[a_k]\}_{k \in [L]}$, and $\{[b_k]\}_{k \in [L]}$, reconstructs the corresponding values, and checks whether $c = \sum_{k=1}^L a_k \cdot b_k$ holds. If the relation holds,

Players: Servers $S_1, \dots, S_n$ and adversary $\mathcal{A}$ corrupting up to $t < n/2$ servers.

Input: Each server S_i inputs its local shares $\llbracket c \rrbracket_i$, $\{\llbracket a_k \rrbracket_i\}_{k \in [L]}$, and $\{\llbracket b_k \rrbracket_i\}_{k \in [L]}$.

Computation: Reconstruct c , $\{a_k\}_{k \in [L]}$, and $\{b_k\}_{k \in [L]}$ from the honest servers' shares. Check whether $c = \sum_{k=1}^L a_k \cdot b_k$ holds. If the check holds, deliver accept to $\mathcal{A}$, which replies with continue or abort. If the check does not hold, set the adversary's reply to abort.

Output: On continue, send accept to all servers; on abort, send reject to all servers.

Figure 7: $\mathcal{F}_{\text{provDeg2Rel}}^{\text{abort}}$ – Proving a degree-2 relation over RSS

the adversary is allowed to choose whether the servers continue or abort, capturing security with abort. If the relation does not hold, the functionality outputs reject to all servers. Thus, the adversary may prevent progress, but cannot make the servers accept an invalid degree-2 relation.

This functionality is used in our offline protocols to verify multiplication chains. For example, Π_{DegGen} uses it to check the relations $q_j = q_{j-1} \cdot M_{T_j}$ arising when converting multiplicative RSS into additive RSS, and Π_{ZKPGen} uses it to verify the multiplication tree that accumulates the designated dealers' contributions. We also use it in the analysis of the naive dVOPRF baseline, which relies on the standard DZKP-based multiplication-verification framework.

G Power-Residue PRFs

G.1 Power-Residue Subgroups

Let $p := eg + 1$ be a sufficiently large prime of $O(\lambda)$ bits, where p, e, g are public, and let $\mathbb{F}_p$ be a finite field of order p . The g -th power-residue subgroup $\mathbb{G}$ of $\mathbb{F}_p^*$ is defined as:

$$\mathbb{G} := \{x^g \bmod p \mid x \in \mathbb{F}_p^*\}.$$

Here, $\mathbb{G}$ is the unique subgroup of $\mathbb{F}_p^*$ of order e .

G.2 Definition of Power-Residue PRFs

With the above power-residue subgroup $\mathbb{G}$, a PRF family $\{F_k : \mathbb{F}_p^* \rightarrow \mathbb{G}\}_{k \in \mathbb{F}_p}$ is called the family of the Power-Residue PRFs [15], where for any input $x \in \mathbb{F}_p^*$, each function F_k is defined as:

$$F_k(x) := (k + x)^{\frac{p-1}{e}} = (k + x)^g \pmod{p}.$$

Note that since the Power-Residue PRF can yield a $\lfloor \log e \rfloor$ -bit output. In particular, we consider two variants of the Power-Residue PRF that outputs a $\lfloor \log e \rfloor$ -bit string, i.e., $F_k : \mathbb{F}_p^* \rightarrow \{0, 1\}^{\lfloor \log e \rfloor}$: the *Legendre PRF* [15] and the *Gold PRF* [34].

- Legendre PRFs: When $e = 2$, the Legendre symbol $\left(\frac{a}{p}\right)$ is exactly the map that connects $\mathbb{F}_p^*$ to $\mathbb{G}$. Since $g = (p-1)/2$, the Power-Residue PRF with $e = 2$ can be expressed as $F_k(x) := (k + x)^g = \left(\frac{k+x}{p}\right) \in \{1, -1\}$, which is commonly referred to the Legendre PRF. Therefore, a single evaluation of Legendre PRF can produce 1-bit randomness.
- Gold PRFs: Power-Residue PRF with $e = 2^\lambda$ is called the Gold PRF. A single evaluation of Gold PRF outputs a λ -bit element

of $\mathbb{G}$. In particular, if h is a generator of $\mathbb{G}$, then any output $y \in \mathbb{G}$ can be written uniquely as $y = h^a$ for some exponent $a \in [2^\lambda]$, and this exponent corresponds to a λ -bit string. The exponent a can be recovered by solving the discrete logarithm of y to the base h ; since $e = 2^\lambda$ is smooth, the Pohlig-Hellman algorithm [31] recovers a in $O(\lambda^2)$ group operations.

G.3 Post-Quantum Security of the Variants

The security of the Legendre PRF relies on the *Decisional Shifted Legendre Symbol* (DSLS) problem. Kalderová et al. [28] analyzed the post-quantum security of the Legendre PRF under the DSLS assumption. On the other hand, Yang et al. [34] analyzed the post-quantum security of the Gold PRF that relies on the underlying computational problem called the *Decisional Shifted Power-Residue Symbol* (DSPRS) problem.

H Aly et al. Protocol 7 in the RSS Setting

Protocol 13 gives the full pseudocode for Π_{AlyGen} , the honest-majority RSS instantiation of Aly et al.'s public-exponent construction [4, Protocol 7] (with the corrected complexity analysis of [30]) used as the state-of-the-art baseline in Section 4.

Protocol 13 Π_{AlyGen} – Baseline offline generation via the public-exponent protocol of [4].

Players: Servers $S_1, \dots, S_n$.

Parameters: Public exponent e with $\gcd(e, p-1) = 1$; fixed generator g of $\mathbb{F}_p^*$.

- 1: Servers invoke $\mathcal{F}_{\text{Rand}}$ to obtain $[r']_{p-1} \in \mathbb{Z}_{p-1}^*$ and $\llbracket \alpha \rrbracket_p \in \mathbb{F}_p$.
- 2: Servers invoke $\Pi_{\text{exp}(b, [\cdot])}$ [4, Prot. 6] to compute $\llbracket \tilde{r} \rrbracket_p \leftarrow \exp(g, [r']_{p-1})$, yielding $\tilde{r} = g^{r'}$.
- 3: Servers invoke $\Pi_{\text{RSS.Mul}}$ to compute $\llbracket c \rrbracket_p \leftarrow \llbracket \tilde{r} \rrbracket_p \cdot \llbracket \alpha \rrbracket_p$; the batched DZKP verification $\mathcal{F}_{\text{provDeg2Rel}}^{\text{abort}}$ is deferred to the end.
- 4: $c \leftarrow \text{Open}(\llbracket c \rrbracket_p)$; if $c = 0$, abort.
- 5: Each server locally computes $c' \leftarrow c^e$, $[\tilde{e}]_{p-1} \leftarrow -e \cdot [r']_{p-1}$.
- 6: Servers invoke $\Pi_{\text{exp}(b, [\cdot])}$ to compute $\llbracket \rho \rrbracket_p \leftarrow \exp(g, [\tilde{e}]_{p-1})$, yielding $\rho = g^{-er'}$.
- 7: Each server locally computes $\llbracket \alpha^e \rrbracket_p \leftarrow c' \cdot \llbracket \rho \rrbracket_p$.

Output: $\llbracket \alpha^e \rrbracket_p$, satisfying $\alpha^e = c^e \cdot g^{-er'} = (g^{r'} \alpha)^e \cdot g^{-er'}$.

I Target-Matched Double Randomness Generation

We adapt $\Pi_{\text{DoubleRand}}$ (Protocol 12) to produce a double sharing $([r], \llbracket r \rrbracket)$ of the same random r , where $\llbracket r \rrbracket$ is a standard RSS (held by all parties) and $[r]$ is an additive sharing restricted to a target subset $\mathcal{A} \subseteq [n]$ with $|\mathcal{A}| \geq t + 1$ (parties outside $\mathcal{A}$ hold zero).

The construction follows the same approach as $\Pi_{\text{DoubleRand}}$: a public covering policy $P_{\mathcal{A}} : \mathcal{T} \rightarrow \mathcal{A}$ with $P_{\mathcal{A}}(T) \notin T$ assigns each RSS component r_T to exactly one party in $\mathcal{A}$ that holds key k_T . Since the resulting base shares are deterministic (any party holding k_T can recompute them), we add a locally generated *random secret sharing of zero* among the parties in $\mathcal{A}$, derived from pairwise PRG seeds, to re-randomize the additive shares.

Protocol 14 $\Pi_{\text{DoubleRand}}^{\text{deg}}$ – Target-Matched Double Randomness for Degenerate Encodings

Players: Servers $S_1, \dots, S_n$ with $n = 2t + 1$.

Pre-shared material: Replicated PRF keys $\{k_T\}_{T \in \mathcal{T}}$ (key k_T held by all S_i with $i \notin T$); $\forall i, j \in [n]$, pairwise PRG seeds $\{s_{i,j}\}$ shared between every pair of servers S_i, S_j .

Input: Target subset of parties $\mathcal{A} \subseteq [n]$ where $|\mathcal{A}| \geq t + 1$; a public counter ctr.

// Step 1 – RSS component (identical to $\Pi_{\text{DoubleRand}}$)

1: For each $T \in \mathcal{T}$ with $i \notin T$, server S_i locally computes $r_T \leftarrow \text{PRF}_{k_T}(\text{ctr})$.

2: Each server S_i sets its replicated share $\llbracket r \rrbracket_i \leftarrow \{r_T \mid i \notin T\}$, which defines the standard RSS $\llbracket r \rrbracket$ with $r = \sum_{T \in \mathcal{T}} r_T$.

// Step 2 – Re-randomized subset additive share

3: Let $P_{\mathcal{A}}: \mathcal{T} \rightarrow \mathcal{A}$ be a public covering policy satisfying $P_{\mathcal{A}}(T) \notin T$ for every $T \in \mathcal{T}$ (such a map exists since $|\mathcal{A}| \geq t + 1$ and $|T| = t$). Each server $S_i \in \mathcal{A}$ locally computes:

$$\llbracket r \rrbracket_i \leftarrow \sum_{T \in \mathcal{T}: P_{\mathcal{A}}(T)=i} r_T + \sum_{\substack{\ell \in \mathcal{A} \\ \ell \neq i}} (-1)^{1[\ell < i]} \text{PRG}(s_{i,\ell}, \text{ctr}).$$

Output: $(\llbracket r \rrbracket, \llbracket r \rrbracket)$, where $\llbracket r \rrbracket_i = 0$ for $i \notin \mathcal{A}$.

Correctness. The pairwise PRG terms form a secret sharing of zero among $\mathcal{A}$: each term $\text{PRG}(s_{i,\ell}, \text{ctr})$ appears with $+1$ for one party and -1 for the other, cancelling in the sum. Hence $\sum_{i \in \mathcal{A}} \llbracket r \rrbracket_i = \sum_{T \in \mathcal{T}} r_T = r$.

Non-interactivity. All computations are local: Step 1 uses replicated PRF keys; Step 2 uses the deterministic inclusion rule (from locally held PRF values) and pairwise PRG seeds. Although multiple parties hold the same key k_T and can compute the same r_T , the covering policy $P_{\mathcal{A}}$ ensures each r_T is counted exactly once.

Privacy. The pairwise PRG terms re-randomize the deterministic base shares. Under $t < n/2$ corruptions, at least one party in $\mathcal{A}$ is honest. Each honest party $h \in \mathcal{A}$ contributes at least one unknown PRG term $\text{PRG}(s_{h,h'}, \text{ctr})$ with another honest $h' \in \mathcal{A}$, derived from a seed the adversary does not hold. Hence individual additive shares are computationally indistinguishable from uniform, even though the adversary can recompute many r_T values from replicated keys.

J Full Dual-Share ZKP Protocols

This appendix presents the full protocol descriptions for the two dual-share consistency ZKP instantiations introduced in Section 4.2.1: the VOLEitH-based Π_{VitH} and the Ligero-based Π_{Lig} .

J.1 Dual-Share Circuit C_{dual} in detail

Both ZKP instantiations prove correctness of the same arithmetic circuit C_{dual} over $\mathbb{F}_p$. Recall that for each dealer $S_i \in \mathcal{A}$, the PRG-based derivation in Π_{ZKPGen} produces a pair $(m_T, a_T) \in \mathbb{F}_p^2$ for every subset $T \in \mathcal{T}$. The share m_T (resp. a_T) is held by every server in $\bar{T} = [n] \setminus T$, since it is derived from a PRG seed shared among all servers in $\bar{T}$. In particular, each server S_j knows (m_T, a_T) for every T with $j \in \bar{T}$, while only the dealer S_i knows all N pairs. The correction δ is a public broadcast issued by the dealer. Once an arbitrary ordering $T_1, T_2, \dots, T_N$ of $\mathcal{T}$ is fixed, we write $m_k := m_{T_k}$

and $a_k := a_{T_k}$ for $k = 1, \dots, N$. We define C_{dual} as an arithmetic circuit over $\mathbb{F}_p$ in detail as follows.

Input wires. The circuit has $2N$ input wires, partitioned into multiplicative and additive shares:

$$\mathbf{x} = (x_1, x_2, \dots, x_{2N-1}, x_{2N}) = (m_1, a_1, \dots, m_N, a_N).$$

Each verifier S_j can independently check the input wires corresponding to subsets T with $j \in \bar{T}$, since it holds those shares locally.

Multiplication gates and wires. The product $\prod_{T \in \mathcal{T}} (m_T)^e$ is computed by a chain of $N - 1$ multiplication gates $\{G_\ell^\times\}_{\ell \in [N-1]}$, with running-product wires $w_1, \dots, w_N$ defined by

$$w_1 = M_1, \quad w_k = w_{k-1} \cdot M_k \quad \text{for } k = 2, \dots, N.$$

where $M_i = m_i^e$ for any $i \in [N]$. Gate $G_\ell^\times$ takes left input w_ℓ , right input $(m_{\ell+1})^e$, and produces $w_{\ell+1}$. The intermediate wires $w_2, \dots, w_N$ are unknown to the verifier and form part of the extended witness. The final wire $w_N = \prod_{T \in \mathcal{T}} (m_T)^e$ feeds into the output computation.

Addition gates (linear). The sum $\sum_{T \in \mathcal{T}} a_T$ and the final subtraction of δ are computed by addition (and constant-subtraction) gates, which are purely linear and thus “free” in the arithmetic circuit model — they require no multiplicative witness and contribute no soundness cost.

Output wire. The circuit has a single output wire computing:

$$C_{\text{dual}}(\mathbf{w}) = w_N - \sum_{T \in \mathcal{T}} a_T - \delta. \quad (3)$$

The prover must demonstrate that this output equals zero.

Extended witness. The full witness for the ZKP consists of the $2N$ input wire values $\mathbf{x}$ together with the $N - 1$ intermediate multiplication output wires $(w_2, \dots, w_N)$, for a total of $L = 3N - 1$ witness elements. We denote the extended witness vector by

$$\mathbf{w} = (M_1, a_1, \dots, M_N, a_N, w_2, \dots, w_N) \in \mathbb{F}_p^L,$$

where the first $2N$ entries are the input wires $\mathbf{x}$ and the remaining $M = N - 1$ entries are the intermediate multiplication output wires. In both Π_{VitH} and Π_{Lig} , the prover commits to $\mathbf{w}$ and proves that (i) each multiplication gate is satisfied, i.e., $w_{\ell+1} = w_\ell \cdot M_{\ell+1}$ for $\ell = 1, \dots, N - 1$, and (ii) the output wire equals zero.

J.2 Full VOLEitH Protocol

We now give the protocol descriptions and supporting derivations for Π_{VitH} (cf. Section 4.2.2).

Prove workflow. In Protocol 15, the prover expands a random seed into τ virtual VOLE shares via a GGM tree, uses them to mask the extended witness, and commits to all shares through a two-level Merkle tree. A Fiat–Shamir challenge then selects one leaf to hide; the prover opens $\text{cp}(\Delta)$ for all other leaves and performs a batched multiplication check (QuickSilver) over the VOLE correlation. Finally, the prover broadcasts the commitment, masked witness, and check values to all verifiers, and sends each verifier the sub-tree authentication paths for the input-wire shares it holds locally.

Verify workflow. In Protocol 16, each verifier re-derives the Fiat-Shamir challenge from the broadcast data, then reconstructs and checks all opened GGM leaves against the Merkle root. For the single hidden leaf, each verifier recovers the missing shares at the input-wire positions it knows, using its local shares and the masked witness, and verifies them against the sub-tree authentication paths received from the prover. Finally, all verifiers perform the QuickSilver multiplication check to confirm that the circuit relation holds.

QuickSilver multiplication-gate check. For each multiplication gate ℓ with left input $a^{(\ell)}$, right input $b^{(\ell)}$, and output $c^{(\ell)}$ (so the circuit enforces $a^{(\ell)} \cdot b^{(\ell)} = c^{(\ell)}$), denote the corresponding VOLE masks by $v_a^{(\ell)}, v_b^{(\ell)}, v_c^{(\ell)}$. The verifier computes:

$$\begin{aligned} B_\ell &= \text{Commit}(a^{(\ell)}) \text{Commit}(b^{(\ell)}) - \text{Commit}(c^{(\ell)}) \Delta \\ &= \underbrace{(a^{(\ell)} b^{(\ell)})}_{= c^{(\ell)}} - c^{(\ell)} \Delta^2 + v_a^{(\ell)} v_b^{(\ell)} \\ &\quad + (a^{(\ell)} v_b^{(\ell)} + b^{(\ell)} v_a^{(\ell)} - v_c^{(\ell)}) \Delta \\ &\stackrel{?}{=} v_a^{(\ell)} v_b^{(\ell)} + (a^{(\ell)} v_b^{(\ell)} + b^{(\ell)} v_a^{(\ell)} - v_c^{(\ell)}) \Delta \end{aligned} \quad (4)$$

When the gate is satisfied, the Δ^2 terms cancel. To batch all M gates, both parties sample a random challenge $\chi \xleftarrow{\$} \mathbb{F}_p$ (via Fiat-Shamir) and check the combined equation

$$V = \sum_\ell \chi^\ell B_\ell = A_0 + A_1 \cdot \Delta \stackrel{?}{=} 0,$$

where $A_0 = \sum_\ell \chi^\ell v_a^{(\ell)} v_b^{(\ell)}$ and $A_1 = \sum_\ell \chi^\ell (v_c^{(\ell)} - b^{(\ell)} v_a^{(\ell)} - a^{(\ell)} v_b^{(\ell)})$. Since the prover knows (v_j, w_j) but not Δ , it computes A_0 and A_1 and masks them with a fresh dedicated check-wire VOLE pair (u_0, v_0) : it sends $\tilde{A}_0 = A_0 + v_0$ and $\tilde{A}_1 = A_1 + u_0$. The verifier then checks $V + q_0 \stackrel{?}{=} \tilde{A}_0 + \tilde{A}_1 \cdot \Delta$, where $q_0 = u_0 \cdot \Delta + v_0$ is the check-wire VOLE tag.

From single repetition to the full protocol. Protocols 15 and 16 describe a single repetition with soundness error $1/\tau$. For κ -bit soundness, we run $R = \lceil \kappa / \log_2 \tau \rceil$ repetitions with the following adaptations:

- (1) **BAVC tree [7].** Naively, R independent GGM trees would cost $R \cdot \log_2 \tau$ co-path hash values in communication. The *Batch All-But-One Vector Commitment* (BAVC) technique replaces them with a single GGM tree of $R \cdot \tau$ leaves: each repetition $\rho \in [R]$ corresponds to leaves $\{(\rho-1)\tau+1, \dots, \rho\tau\}$, yielding an independent VOLE correlation $(u_j^{(\rho)}, v_j^{(\rho)})$. In our two-level variant, the BAVC applies to the outer Merkle tree over all $R \cdot \tau$ sub-tree roots h_i ; the sub-Merkle-tree structure within each leaf remains unchanged.
- (2) **Masked witness and challenges.** The prover computes one masked witness $\tilde{\mathbf{w}}^{(\rho)}$ per repetition using the corresponding VOLE masks: $\tilde{w}_j^{(\rho)} = w_j + u_j^{(\rho)}$. The Fiat-Shamir transformation derives R independent challenges (χ_ρ, Δ_ρ) from $H(\delta, \text{rt}, \tilde{\mathbf{w}}^{(1)}, \dots, \tilde{\mathbf{w}}^{(R)})$.
- (3) **Per-repetition co-paths and checks.** For each repetition ρ , the prover opens $\text{cp}(\Delta_\rho)$ and computes its own check values $(\tilde{A}_0^{(\rho)}, \tilde{A}_1^{(\rho)})$. Each verifier independently performs the GGM reconstruction, input binding, and QuickSilver check for

every repetition; the proof is accepted only if all R checks pass.

Protocol 15 $\Pi_{\text{VithH, Prove}} - \text{VOLEithH Proof Generation}$

Players: Prover $\mathcal{P}$ (dealer S_i).

Input: Extended witness $\mathbf{w} \in \mathbb{F}_p^L$; a collision resistant hash function H , public statement $\delta \in \mathbb{F}_p$.

- ```
// GGM tree expansion
1: Generate a GGM tree from sd and obtain τ leaf seeds $\{sd_i\}_{i \in [\tau]}$,
 where $sd \xleftarrow{\$} \{0, 1\}^\kappa$.
2: $\forall i \in [\tau]$, derive shares $\{s_j^{(i)}\}_{j=0}^L$ from sd_i ($j=0$: check wire);
 compute $u_j = \sum_i s_j^{(i)}$, $v_j = -\sum_i i \cdot s_j^{(i)}$.
// Extended witness masking
3: Compute the masked witness $\tilde{\mathbf{w}} \in \mathbb{F}_p^L$ with $\tilde{w}_j = w_j + u_j$ for all
 $j \in [L]$.
// Two-level GGM commitment
4: $\forall i \in [\tau]$, build a sub-Merkle-tree over the share vector
 $(s_0^{(i)}, s_1^{(i)}, \dots, s_L^{(i)})$; let h_i be its computed root commitment.
5: Build the outer Merkle tree over $\{h_i\}_{i \in [\tau]}$ and derive root
 commitment rt .
// Fiat-Shamir challenge and opening
6: Derive χ, Δ from $H(\delta, \text{rt}, \tilde{\mathbf{w}})$ via Fiat-Shamir.
7: Open $\text{cp}(\Delta)$ (outer tree).
// Batched multiplication check (QuickSilver)
8: Compute A_0, A_1 from the gate-by-gate VOLE check (4), batched
 with challenge χ .
9: Mask with check-wire VOLE: $\tilde{A}_0 = A_0 + v_0$,
10: $\tilde{A}_1 = A_1 + u_0$.
```

**Broadcast:**  $\pi_{\text{bc}} = (\text{rt}, \tilde{\mathbf{w}}, \tilde{A}_0, \tilde{A}_1, \text{cp}(\Delta))$ .

**$\mathcal{P}$  sends to each  $S_j$ :** sub-tree authentication paths in leaf  $\Delta$  for every input-wire position that  $S_j$  holds locally.

---

*Proof size and complexity.* For  $R = \lceil \kappa / \log_2 \tau \rceil$  repetitions over  $\mathbb{F}_p$  with  $L = 2N + M$  wires ( $M = N-1$  multiplication gates) and  $d = \lceil \log_2 \tau \rceil$ :

- **Broadcast proof size** (per proof): root commitment  $\text{rt}$  ( $2\kappa$  bits via BAVC); masked witnesses  $\tilde{\mathbf{w}}^{(1)}, \dots, \tilde{\mathbf{w}}^{(R)}$  ( $RL$  field elements); check values  $\tilde{A}_0, \tilde{A}_1$  ( $2R$  field elements); co-paths ( $Rd$  GGM sibling seeds and  $Rd$  Merkle sibling hashes).
- **P2P proof size** (per verifier  $S_j$ ):  $R$  sub-tree authentication paths for input-wire shares, each of depth  $\lceil \log_2 (L+1) \rceil$  hashes.
- **Prover:**  $O(R\tau L)$  hash evaluations,  $O(R\tau)$  PRG evaluations,  $O(RM)$  field operations.
- **Verifier:**  $O(R\tau)$  PRG evaluations,  $O(R(L+d))$  hash evaluations,  $O(RM)$  field operations.

In summary, substituting  $R = O(\kappa)$ ,  $L = O(N)$ ,  $M = N-1$ , and  $d = O(1)$ , a single invocation of  $\Pi_{\text{VithH}}$  has broadcast cost  $O(\kappa N)$  field elements plus  $O(\kappa^2)$  bits, P2P cost  $O(\kappa \log N)$  hashes per verifier, prover computation  $O(\kappa N)$  hashes, and verifier computation  $O(\kappa N)$  hashes plus  $O(\kappa N)$  field operations. When invoked  $t+1$  times in  $\Pi_{\text{ZKPGen}}$ , the total broadcast is  $O(t\kappa N)$  and the total P2P per verifier is  $O(t\kappa \log N)$  hashes—all from efficient symmetric-key primitives.

**Protocol 16**  $\Pi_{\text{Vith.Verify}} - \text{VOLEitH Proof Verification}$ **Players:** Verifier  $S_j, j \in [n] \setminus \{i\}$ .**Input:** Broadcast proof  $\pi_{bc}$  from  $\mathcal{P}$ ; P2P proof  $\pi_j$  for each  $S_j$ ; public  $\delta, \mathbb{F}, H$ . Each  $S_j$  holds its local shares  $\{(a_T, m_T)\}$  for every subset  $T$  it belongs to.

// Fiat-Shamir re-derivation

1: Re-derive  $\chi, \Delta$  from  $H(\delta, \text{rt}, \tilde{w})$  via Fiat-Shamir.

// GGM reconstruction

2: Reconstruct  $\{sd_i\}_{i \neq \Delta}$  from the GGM sibling seeds  $(sd_1^*, \dots, sd_d^*)$  in  $\text{cp}(\Delta)$ .3:  $\forall i \neq \Delta$ : expand  $sd_i$  to obtain shares  $\{s_j^{(i)}\}_{j=0}^L$ .// Input binding and commitment check for hidden leaf  $\Delta$ 4: **for each**  $S_j$  **do**5:     **for each input wire**  $k$  **that**  $S_j$  **holds locally do**6:         Compute  $u_k = \tilde{w}_k - w_k^{\text{local}}$  (VOLE mask from masked witness and local share).7:         Recover hidden-leaf share:  $s_k^{(\Delta)} = u_k - \sum_{i \neq \Delta} s_k^{(i)}$ .8:         Verify  $s_k^{(\Delta)}$  against  $h_\Delta$  using the sub-tree authentication path from  $\pi_j$ .9:     **end**10: **end**11: Verify  $h_\Delta$  against  $\text{rt}$  using the Merkle sibling hashes $(h_1^*, \dots, h_d^*)$  in  $\text{cp}(\Delta)$ .

// VOLEitH multiplication check (QuickSilver)

12: Re-derive VOLE tags:  $q_j = \sum_{i \neq \Delta} s_j^{(i)} (\Delta - i)$  for all wires  $j$ .13: Compute wire commitments  $\text{Commit}(w_j) = \tilde{w}_j \cdot \Delta - q_j$ .14: Evaluate the batched gate check (4):  $V = \sum_{\ell} \chi^\ell B_\ell$ .15: Compute check-wire tag  $q_0 = \sum_{i \neq \Delta} s_0^{(i)} (\Delta - i)$ ; check  $V + q_0 \stackrel{?}{=} \tilde{A}_0 + \tilde{A}_1 \cdot \Delta$ .**Output:** Accept iff all checks pass.

### J.3 Full Ligero Protocol

We now give the protocol descriptions for  $\Pi_{\text{Lig}}$  (cf. Section 4.2.3).

*Soundness and parameters.* If any constraint in any instance is violated, the error polynomial  $E(x) = Z_W(x) \cdot \hat{C}(x) - N(x)$  is nonzero with  $\deg(E) \leq 2N-1$  (independent of  $B$ ). The zero-knowledge constraint limits the sampling budget: each sampled pair opens 2 columns, so we require  $2|Q| \leq N-1$ , giving  $|Q| \leq \lfloor (N-1)/2 \rfloor$ . Setting  $(\frac{2N-1}{n_c-N})^{|Q|} \leq 2^{-\kappa}$  yields:

$$n_c \geq N + (2N-1) \cdot 2^{\kappa/|Q|}.$$

The Ligero blowup  $n_c/N$  depends on  $N$ : it approaches the standard Ligero  $3\times$  for large  $N$  but grows exponentially in  $\kappa/N$  when  $N$  is small. At statistical security  $\kappa = 40$ ,  $\Pi_{\text{Lig}}$  is practical for  $N \gtrsim 20$  (blowup  $\lesssim 42\times$ ) and very efficient for  $N \geq 128$  (blowup  $\approx 3\times$ ); for smaller  $N$ ,  $\Pi_{\text{Vith}}$  is preferred since its proof size scales with the number of multiplication gates rather than the code blowup. For large  $N$  ( $N \gg \kappa$ ),  $|Q| \approx N/2$  is large, so  $2^{\kappa/|Q|} \approx 1$  and the blowup  $n_c/N \rightarrow 3$  (the standard Ligero regime). However, for small  $N$  the sampling budget  $|Q| \approx N/2$  is limited, causing  $2^{\kappa/|Q|}$  to grow exponentially in  $\kappa/N$  and making the blowup impractical (e.g.,  $\sim 487\times$  for  $N = 10$  at  $\kappa = 40$ ).

*Witness encoding and commitment.* For each instance  $b \in [B]$ , the prover defines base cases  $p_0^{(b)} := 1, s_0^{(b)} := 0$  and, for  $k = 1, \dots, N$ , forms the running products  $p_k^{(b)} = \prod_{\ell=1}^k m_\ell^{(b)}$  and running sums  $s_k^{(b)} = \sum_{\ell=1}^k a_\ell^{(b)}$ . All  $B$  instances are stacked into a single  $4B \times N$  witness matrix:

$$W = \begin{pmatrix} M_1^{(1)} & \dots & M_N^{(1)} \\ a_1^{(1)} & \dots & a_N^{(1)} \\ p_1^{(1)} & \dots & p_N^{(1)} \\ s_1^{(1)} & \dots & s_N^{(1)} \\ \vdots & \ddots & \vdots \\ M_1^{(B)} & \dots & M_N^{(B)} \\ a_1^{(B)} & \dots & a_N^{(B)} \\ p_1^{(B)} & \dots & p_N^{(B)} \\ s_1^{(B)} & \dots & s_N^{(B)} \end{pmatrix} \in \mathbb{F}^{4B \times N}.$$

For each instance  $b$ , rows  $4(b-1)+1$  and  $4(b-1)+2$  are *share rows* (verifiable by servers holding the corresponding shares), while rows  $4(b-1)+3$  and  $4(b-1)+4$  are *computation rows* (known only to the prover). To achieve statistical zero knowledge, the prover appends  $\lambda = 2$  random blinding rows, yielding a  $(4B+2) \times N$  matrix. Each row is then independently encoded by an  $[n_c, N]$  *Reed-Solomon* (RS) code over  $\mathbb{F}$  with evaluation domain

$$D = \{1, \omega, \omega^2, \dots, \omega^{n_c-1}\},$$

where  $\omega$  is a primitive  $n_c$ -th root of unity, producing the extended matrix  $\hat{W} \in \mathbb{F}^{(4B+2) \times n_c}$ . The code rate is  $< 1/3$  (i.e.,  $n_c > 3N-1$ ), necessary for soundness. The columns of  $\hat{W}$  are committed via a Merkle tree, where witness-position columns ( $j = 1, \dots, N$ ) use a two-level hash separating share rows from private rows:

$$\text{leaf}_j = H(\hat{W}_{\text{share},j} \parallel H(\hat{W}_{\text{priv},j})),$$

and extension-position columns ( $j = N+1, \dots, n_c$ ) are hashed as a single block.

*Constraint verification via quotient polynomials.* After RS encoding, each witness row becomes a polynomial of degree at most  $N-1$  evaluated on the multiplicative domain  $D = \{1, \omega, \dots, \omega^{n_c-1}\}$ . For each instance  $b \in [B]$ , denote the row polynomials by  $f_M^{(b)}, f_a^{(b)}, f_p^{(b)}, f_s^{(b)}$ . The dual-share consistency relation requires five constraints per instance:

- *Transition constraints* (for each  $k \in \{2, \dots, N\}$ ):  $p_k^{(b)} = p_{k-1}^{(b)} \cdot M_k^{(b)}, s_k^{(b)} = s_{k-1}^{(b)} + a_k^{(b)}$ .
- *Boundary constraints*:  $p_1^{(b)} = M_1^{(b)}, s_1^{(b)} = a_1^{(b)}, p_N^{(b)} - s_N^{(b)} = \delta^{(b)}$ .

These translate into five numerator polynomials per instance:

- (1)  $h_1^{(b)}(x) = f_p^{(b)}(\omega x) - f_p^{(b)}(x) \cdot f_M^{(b)}(\omega x)$  (mult. transition, on  $T$ )
- (2)  $h_2^{(b)}(x) = f_s^{(b)}(\omega x) - f_s^{(b)}(x) - f_a^{(b)}(\omega x)$  (add. transition, on  $T$ )
- (3)  $h_3^{(b)}(x) = f_p^{(b)}(x) - f_M^{(b)}(x)$  (boundary at  $x = 1$ )
- (4)  $h_4^{(b)}(x) = f_s^{(b)}(x) - f_a^{(b)}(x)$  (boundary at  $x = 1$ )
- (5)  $h_5^{(b)}(x) = f_p^{(b)}(x) - f_s^{(b)}(x) - \delta^{(b)}$  (boundary at  $x = \omega^{N-1}$ )

where  $T = \{1, \omega, \dots, \omega^{N-2}\}$  denotes the  $N-1$  transition points. The per-instance batched numerator, promoted to the common

denominator  $Z_W(x) = \prod_{k=0}^{N-1} (x - \omega^k)$ :

$$\begin{aligned} \mathcal{N}^{(b)}(x) &= (x - \omega^{N-1}) [h_1^{(b)}(x) + \alpha h_2^{(b)}(x)] \\ &\quad + Z_T^*(x)(x - \omega^{N-1}) [\alpha^2 h_3^{(b)}(x) + \alpha^3 h_4^{(b)}(x)] \\ &\quad + \alpha^4 Z_T(x) h_5^{(b)}(x), \end{aligned}$$

and the global batched numerator:  $\mathcal{N}(x) = \sum_{b=1}^B \alpha^{5(b-1)} \mathcal{N}^{(b)}(x)$ . The composition polynomial  $C(x) = \mathcal{N}(x)/Z_W(x)$  has degree  $\leq N-1$  when all constraints hold.

The prove and verify phases are given in Protocols 17 and 18, respectively.

*Complexity of  $\Pi_{\text{Lig}}$ .*

- **Communication (single invocation,  $B$  instances).**
  - Broadcast proof  $\pi_{bc}$ :  $O(\kappa \log N)$  hashes +  $O(B\kappa + N)$  field elements. The amortised per-instance broadcast cost approaches  $O(\kappa)$  for large  $B$ .
  - P2P proof  $\pi_j$  per verifier:  $O(|\tilde{T}_j| \log n_c)$  hashes +  $O(B|\tilde{T}_j|)$  field elements.
- **Communication ( $t+1$  invocations in  $\Pi_{\text{ZKPGen}}$ ).** Total broadcast:  $O((t+1)\kappa \log N)$  hashes +  $O((t+1)(B\kappa + N))$  field elements. Total P2P per verifier:  $O((t+1)|\tilde{T}_j| \log n_c)$  hashes +  $O((t+1)B|\tilde{T}_j|)$  field elements.
- **Prover cost.** RS encoding of  $4B+3$  rows:  $O(BN \log N)$  field operations, plus  $O(n_c)$  hashes for the Merkle trees.
- **Verifier cost.**  $O(\kappa \log N)$  hash evaluations plus  $O(B\kappa)$  field operations.

## K Security Proof of Our dVOPRF Protocol

Before proving Theorem 5.1, we introduce two core lemmas that we will use.

### K.1 Properties of Our VIP Protocol

LEMMA K.1. *In the  $\mathcal{F}_{\text{Coin}}$ -hybrid model,  $\Pi_{\text{VIP}}$  (Protocol 4) satisfies the following properties.*

- (1) **Correctness.** *If the sharing server  $S_i$  is honest, then the protocol outputs consistent secret sharings  $\llbracket c_i \rrbracket, \llbracket d_i^{(y)} \rrbracket, \llbracket a_i^{(1)} \rrbracket, \llbracket b_i^{(1)} \rrbracket, \llbracket \sigma_i \rrbracket$  of values satisfying  $c_i = \sum_{\ell=1}^L a_i^{(\ell)} b_i^{(\ell)}, d_i^{(y)} = a_i^{(1)} b_i^{(1)}$ , and  $\sigma_i = 0$ .*
- (2) **Soundness.** *If the sharing server  $S_i$  is corrupted, then except with probability at most  $\epsilon_{\text{VIP}} = (2\lceil \log L \rceil + 1)/(|\mathbb{F}_p| - 3)$ , either the protocol aborts, or the protocol outputs consistent secret sharings  $\llbracket c_i \rrbracket, \llbracket d_i^{(y)} \rrbracket, \llbracket a_i^{(1)} \rrbracket, \llbracket b_i^{(1)} \rrbracket, \llbracket \sigma_i \rrbracket$  such that, if the reconstructed values satisfy  $d_i^{(y)} = a_i^{(1)} b_i^{(1)}$  and  $\sigma_i = 0$ , then necessarily  $c_i = \sum_{\ell=1}^L a_i^{(\ell)} b_i^{(\ell)}$ .*

PROOF. Fix an instance indexed by  $i \in [n]$ , and let  $\gamma := \lceil \log L \rceil$ . Throughout the proof, dummy zero pairs appended when the current length is odd are ignored in the notation, since they do not change any inner product.

*Correctness.* Assume first that the sharing server  $S_i$  is honest. Every value explicitly distributed by  $S_i$  is shared using VSS, and every subsequent shared value is obtained from already shared values by local linear operations and Lagrange interpolation. Hence, unless the protocol aborts, all sharings generated by the protocol are consistent. For each round  $k \in [\gamma]$ , let  $D^{(k-1)}$  denote the inner

---

### Protocol 17 $\Pi_{\text{Lig, Prove}}$ – Ligerio Proof Generation ( $B$ instances)

---

**Players:** Prover  $\mathcal{P}$  (dealer  $S_i$ ).

**Input:** Private:  $B$  share vectors  $\{(m_k^{(b)}, a_k^{(b)})\}_{k=1}^N$  for  $b = 1, \dots, B$ .  
Public:  $\{\delta^{(b)}\}_{b=1}^B, \mathbb{F}, [n_c, N]$  RS code with evaluation domain  $D, H$ .

// Witness encoding and commitment

- 1: **for**  $b = 1, \dots, B$  **do**
- 2:     Compute running products  $p_k^{(b)} = \prod_{\ell=1}^k M_\ell^{(b)}$  and running sums  $s_k^{(b)} = \sum_{\ell=1}^k a_\ell^{(b)}$  for  $k = 1, \dots, N$ .
- 3: **end**
- 4: Construct  $W \in \mathbb{F}^{4B \times N}$  with rows  $(M_k^{(b)})_k, (a_k^{(b)})_k, (p_k^{(b)})_k, (s_k^{(b)})_k$  for each  $b$ ; append 2 random blinding rows.
- 5: RS-encode each row  $\rightarrow \hat{W} \in \mathbb{F}^{(4B+2) \times n_c}$ .
- 6: Merkle-commit columns of  $\hat{W}$ : two-level hash for witness positions ( $j \leq N$ ), single hash for extension positions ( $j > N$ )  $\rightarrow \text{rt}_{\hat{W}}$ .
- // Composition polynomial
- 7: Derive batching challenge  $\alpha \in \mathbb{F}$  from  $(\text{rt}_{\hat{W}}, \text{salt}_1)$  via Fiat-Shamir, where  $\text{salt}_1$  is a fixed domain-separation tag.
- 8: For each  $b \in [B]$ : compute numerator polynomials  $h_1^{(b)}, \dots, h_5^{(b)}$  from row polynomials  $f_M^{(b)}, f_a^{(b)}, f_p^{(b)}, f_s^{(b)}$  and form per-instance  $\mathcal{N}^{(b)}(x)$ .
- 9: Form the global batched numerator  $\mathcal{N}(x) = \sum_{b=1}^B \alpha^{5(b-1)} \mathcal{N}^{(b)}(x)$ .
- 10: Compute  $C(x) = \mathcal{N}(x)/Z_W(x)$  (degree  $\leq N-1$ ), RS-encode  $\rightarrow \hat{C} \in \mathbb{F}^{n_c}$ .
- 11: Merkle-commit  $\hat{C}$  with leaves  $\text{leaf}_j = H(\hat{C}_j)$  for  $j = 1, \dots, n_c \rightarrow \text{rt}_{\hat{C}}$ .
- // Interleaved consistency (commit before column sampling)
- 12: Form  $\hat{W}' = (\hat{W}; \hat{C}) \in \mathbb{F}^{(4B+3) \times n_c}$ . Derive  $\mathbf{v}' \xleftarrow{\$} \mathbb{F}^{4B+3}$  from  $(\text{rt}_{\hat{W}}, \text{rt}_{\hat{C}}, \text{salt}_2)$ .
- 13: Compute  $\mathbf{u} = (\mathbf{v}')^\top \hat{W}' \in \mathbb{F}^{n_c}$  and commit  $\text{rt}_{\mathbf{u}} = H(\mathbf{u})$ .
- // Column sampling and opening
- 14: Partition the  $n_c - N$  extension positions into  $\lfloor (n_c - N)/2 \rfloor$  disjoint pairs  $\{j, \omega j\}$ .
- 15: Sample  $|Q| = \lfloor (N-1)/2 \rfloor$  pairs from  $(\text{rt}_{\hat{W}}, \text{rt}_{\hat{C}}, \text{rt}_{\mathbf{u}}, \text{salt}_3)$  via Fiat-Shamir; let  $Q$  denote the  $2|Q|$  opened positions.
- 16: Open  $\{\hat{W}_{*,j}\}_{j \in Q}$  (full columns of  $4B+2$  field elements each, with Merkle paths from  $\text{rt}_{\hat{W}}$ ) and  $\{\hat{C}_j\}_{j \in Q}$  (with Merkle paths from  $\text{rt}_{\hat{C}}$ ).
- 17: Broadcast  $\mathbf{u}$ .
- // P2P share openings (per-verifier)
- 18: For each verifier  $S_j$ : send partial openings of witness-position columns  $\{k : j \in \tilde{T}_k\}$ , each consisting of the share-row entries  $\hat{W}_{\text{share},k} = (M_k^{(1)}, a_k^{(1)}, \dots, M_k^{(B)}, a_k^{(B)})$ , the inner hash  $H(\hat{W}_{\text{priv},k})$ , and the Merkle authentication path  $\text{auth}_k$ .

**Broadcast output:**  $\pi_{bc} = (\text{rt}_{\hat{W}}, \text{rt}_{\hat{C}}, \text{rt}_{\mathbf{u}}, \{\hat{W}_{*,j}\}_{j \in Q}, \{\hat{C}_j\}_{j \in Q}, \mathbf{u})$  with Merkle paths.

**P2P output to  $S_j$ :**  $\pi_j = \{(\hat{W}_{\text{share},k}, H(\hat{W}_{\text{priv},k}), \text{auth}_k)\}_{k: j \in \tilde{T}_k}$ .

---

product of the current folded vectors at the beginning of round  $k$ .

**Protocol 18**  $\Pi_{\text{Lig.Verify}}$  – Liger Proof Verification ( $B$  instances)**Players:** Verifiers  $\{S_j\}_{j \in [n] \setminus \{i\}}$ .**Broadcast input:**  $\pi_{bc} = (\text{rt}_{\hat{W}}, \text{rt}_C, \text{rt}_u, \{\hat{W}_{*,j}\}_{j \in Q}, \{\hat{C}_j\}_{j \in Q}, \mathbf{u})$  with Merkle paths.**P2P input to  $S_j$ :**  $\pi_j = \{(\hat{W}_{\text{share},k}, H(\hat{W}_{\text{priv},k}), \text{auth}_k)\}_{k: j \in T_k}$ .**Public:**  $\{\delta^{(b)}\}_{b=1}^B, \mathbb{F}, [n_c, N]$  RS code with domain  $D$ ,  $H$ . Each  $S_j$  holds  $\{(m_k^{(b)}, a_k^{(b)})\}_{k: j \in T_k}$  for all  $b \in [B]$ .// Local share verification (per-verifier, using P2P input  $\pi_j$ )

- 1: **for each**  $S_j$  **do**
- 2:   For each  $(\hat{W}_{\text{share},k}, H(\hat{W}_{\text{priv},k}), \text{auth}_k) \in \pi_j$ :
  - Check share rows: for each  $b \in [B]$ , verify  $(M_k^{(b)}, a_k^{(b)})$  matches the corresponding entries in  $\hat{W}_{\text{share},k}$ .
  - Recompute  $\text{leaf}_k = H(\hat{W}_{\text{share},k} \| H(\hat{W}_{\text{priv},k}))$  and verify  $\text{auth}_k$  against  $\text{rt}_{\hat{W}}$ .
- 3: **end**
- // Interleaved consistency check (before column sampling)
- 4: Re-derive  $\mathbf{v}' \in \mathbb{F}^{4B+3}$  from  $(\text{rt}_{\hat{W}}, \text{rt}_C, \text{salt}_2)$ .
- 5: Check: (i)  $\text{rt}_u = H(\mathbf{u})$  (prover is bound to  $\mathbf{u}$  before  $Q$  was derived);
- 6: (ii)  $\mathbf{u}$  is a valid RS codeword of degree  $\leq N-1$ .
- // Constraint verification via column sampling (extension-position columns)
- 7: Re-derive  $\alpha$  from  $(\text{rt}_{\hat{W}}, \text{salt}_1)$  and the  $|Q|$  sampled pairs from  $(\text{rt}_{\hat{W}}, \text{rt}_u, \text{salt}_3)$  via Fiat–Shamir; let  $Q$  be the  $2|Q|$  opened positions.
- 8: Verify Merkle paths of all opened columns against  $\text{rt}_{\hat{W}}$  and  $\text{rt}_C$ .
- 9: Check column consistency:  $\mathbf{u}|_Q = (\mathbf{v}')^\top \hat{W}'_{*,Q}$  (opened columns match the committed  $\mathbf{u}$ ).
- 10: For each sampled pair  $\{j, \omega j\} \subset Q$ , compute  $\mathcal{N}(j)$  from opened row values at positions  $j$  and  $\omega j$  across all  $B$  instances:
  - For each  $b \in [B]$ , evaluate  $\mathcal{N}^{(b)}(j)$  using the per-instance row values at the opened positions (transition, boundary, and output terms as in the text).
  - Combine:  $\mathcal{N}(j) = \sum_{b=1}^B \alpha^{5(b-1)} \mathcal{N}^{(b)}(j)$ .
  - Check:  $Z_W(j) \cdot \hat{C}_j = \mathcal{N}(j)$ .

**Output:** Accept or reject the claims  $\delta^{(b)} = \prod_{k=1}^N M_k^{(b)} - \sum_{k=1}^N a_k^{(b)}$  for all  $b \in [B]$ .

In particular,  $D^{(0)} = \sum_{\ell=1}^L a_i^{(\ell)} b_i^{(\ell)}$ . Let  $q^{(k)}$  be the quadratic polynomial honestly defined in round  $k$  from the current folded vectors:  $q^{(k)}(x) = \sum_{\ell=1}^{L/2} f_{2\ell-1}(x) f_{2\ell}(x)$ , where  $L$  is the current folded length in that round. By construction of the linear polynomials, the two evaluation points 1 and 2 correspond to the two halves of the current folded vectors. Therefore,  $q^{(k)}(1) + q^{(k)}(2) = D^{(k-1)}$ . In the first round, the protocol defines  $\llbracket c_i \rrbracket \leftarrow \llbracket q^{(1)}(1) \rrbracket + \llbracket q^{(1)}(2) \rrbracket$ . Hence,  $c_i = q^{(1)}(1) + q^{(1)}(2) = D^{(0)} = \sum_{\ell=1}^L a_i^{(\ell)} b_i^{(\ell)}$ . For each round  $k$ , after the challenge  $r_k \xleftarrow{\$} \mathbb{F}_p \setminus \{1, 2, 3\}$  is sampled, the protocol sets  $d_i^{(k)} = q^{(k)}(r_k)$  and updates the folded vectors to  $a_i^{(\ell)} \leftarrow f_{2\ell-1}(r_k)$  and  $b_i^{(\ell)} \leftarrow f_{2\ell}(r_k)$ . Thus the next folded inner product is exactly  $D^{(k)} = q^{(k)}(r_k) = d_i^{(k)}$ . By induction over  $k$ , after the final round the folded vectors have length one and  $d_i^{(Y)} = D^{(Y)} = a_i^{(1)} b_i^{(1)}$ . It

remains to show that  $\sigma_i = 0$ . For each  $k \geq 2$ , the protocol checks the residual  $d_i^{(k-1)} - q^{(k)}(1) - q^{(k)}(2)$ . In the honest execution,  $d_i^{(k-1)} = D^{(k-1)}$  and  $q^{(k)}(1) + q^{(k)}(2) = D^{(k-1)}$ , so every such residual is zero. Hence  $\sigma_i = \sum_{k=2}^Y \epsilon_k (d_i^{(k-1)} - q^{(k)}(1) - q^{(k)}(2)) = 0$ . Therefore the output sharings reconstruct to  $c_i = \sum_{\ell=1}^L a_i^{(\ell)} b_i^{(\ell)}$ ,  $d_i^{(Y)} = a_i^{(1)} b_i^{(1)}$ , and  $\sigma_i = 0$ . This proves correctness.

**Soundness.** Assume now that the sharing server  $S_i$  is corrupted, and condition on the event that the protocol does not abort. By VSS, each value shared by  $S_i$  is fixed consistently before the corresponding challenge  $r_k$  is sampled. Thus, for each round  $k$ , the three shared values determine a unique quadratic polynomial. We denote this actually shared polynomial by  $\tilde{q}^{(k)}$ . We also define an ideal honest folding process. Let  $q_*^{(k)}$  denote the quadratic polynomial that would be obtained in round  $k$  from the honestly folded vectors determined by the original inputs and the previously sampled challenges  $r_1, \dots, r_{k-1}$ . Let  $D^{(0)} := \sum_{\ell=1}^L a_i^{(\ell)} b_i^{(\ell)}$  be the true initial inner product, and for  $k \in [Y]$  define  $D^{(k)} := q_*^{(k)}(r_k)$ . Thus  $D^{(k)}$  is the honest folded inner product after round  $k$ . On the other hand, the actual values produced by the possibly corrupted dealer are  $c_i = \tilde{q}^{(1)}(1) + \tilde{q}^{(1)}(2)$ , and, for  $k \in [Y]$ ,  $d^{(k)} := \tilde{q}^{(k)}(r_k)$ . For notational convenience, define  $d^{(0)} := c_i$ . For each  $k \in [Y]$ , define the polynomial error  $e_k(x) := \tilde{q}^{(k)}(x) - q_*^{(k)}(x)$ , and define the folded error  $\delta^{(k)} := d^{(k)} - D^{(k)}$  for  $k \in [Y]$ . For the initial claim, define  $\delta^{(0)} := d^{(0)} - D^{(0)} = c_i - \sum_{\ell=1}^L a_i^{(\ell)} b_i^{(\ell)}$ . Since  $c_i = \tilde{q}^{(1)}(1) + \tilde{q}^{(1)}(2)$  and  $D^{(0)} = q_*^{(1)}(1) + q_*^{(1)}(2)$ , we have  $\delta^{(0)} = e_1(1) + e_1(2)$ . Moreover, for every  $k \in [Y]$ ,  $\delta^{(k)} = d^{(k)} - D^{(k)} = \tilde{q}^{(k)}(r_k) - q_*^{(k)}(r_k) = e_k(r_k)$ . For each  $k \in \{2, \dots, Y\}$ , define the actual residual  $\tilde{s}^{(k)} := d^{(k-1)} - \tilde{q}^{(k)}(1) - \tilde{q}^{(k)}(2)$ . Since the honest polynomial satisfies  $q_*^{(k)}(1) + q_*^{(k)}(2) = D^{(k-1)}$ , we obtain  $\tilde{s}^{(k)} = \delta^{(k-1)} - (e_k(1) + e_k(2))$ . The value computed by the protocol is  $\sigma_i = \sum_{k=2}^Y \epsilon_k \tilde{s}^{(k)}$ , where the coefficients  $\epsilon_2, \dots, \epsilon_Y$  are sampled only after all folding rounds are completed. We now prove that, except with the claimed probability, it is impossible to have an incorrect initial claim  $c_i$  while also satisfying the two later checks  $d_i^{(Y)} = a_i^{(1)} b_i^{(1)}$  and  $\sigma_i = 0$ . Let Bad be the event that  $\delta^{(0)} \neq 0$ ,  $\sigma_i = 0$ , and  $d_i^{(Y)} = a_i^{(1)} b_i^{(1)}$ . Since  $\delta^{(0)} = c_i - \sum_{\ell=1}^L a_i^{(\ell)} b_i^{(\ell)}$ , ruling out Bad is exactly the desired soundness statement. We distinguish two cases.

*Case 1: Some residual from rounds  $2, \dots, Y$  is nonzero.* Suppose there exists  $k \in \{2, \dots, Y\}$  such that  $\tilde{s}^{(k)} \neq 0$ . Let  $j = \max\{k \in \{2, \dots, Y\} : \tilde{s}^{(k)} \neq 0\}$ . All values  $\tilde{s}^{(2)}, \dots, \tilde{s}^{(j)}$  are fixed before  $\epsilon_2, \dots, \epsilon_j$  are sampled. Conditioned on all coefficients except  $\epsilon_j$ , the value  $\sigma_i = \sum_{k=2}^Y \epsilon_k \tilde{s}^{(k)}$  is a nonconstant affine function of the uniform variable  $\epsilon_j$ . Therefore,  $\Pr[\sigma_i = 0 \mid \exists k \geq 2 : \tilde{s}^{(k)} \neq 0] \leq 1/|\mathbb{F}_p|$ . Consequently,  $\Pr[\text{Bad} \wedge (\exists k \in \{2, \dots, Y\} : \tilde{s}^{(k)} \neq 0)] \leq 1/|\mathbb{F}_p|$ .

*Case 2: All residuals from rounds  $2, \dots, Y$  are zero.* Assume now that  $\tilde{s}^{(k)} = 0$  for all  $k \in \{2, \dots, Y\}$ . We show that, on Bad, some nonzero error polynomial must vanish at one of the random folding challenges. First, not all  $e_k$  can be the zero polynomial. Indeed, if  $e_k \equiv 0$  for all  $k \in [Y]$ , then in particular  $e_1 \equiv 0$ , and hence  $\delta^{(0)} = e_1(1) + e_1(2) = 0$ , contradicting Bad. Thus, on Bad, there exists at least one round  $k$  with  $e_k \neq 0$ . Let  $\tau = \max\{k \in [Y] : e_k \neq 0\}$ . Then  $e_\tau$  is a nonzero polynomial of degree at most 2.

We claim that Bad and the condition  $\tilde{s}^{(k)} = 0$  for all  $k \geq 2$  imply  $\delta^{(\tau)} = 0$ . If  $\tau = \gamma$ , the condition  $d_i^{(\gamma)} = a_i^{(1)}b_i^{(1)}$  implies  $d^{(\gamma)} = D^{(\gamma)}$ , because  $D^{(\gamma)}$  is exactly the honest final folded inner product, namely  $D^{(\gamma)} = a_i^{(1)}b_i^{(1)}$ . Therefore  $\delta^{(\gamma)} = 0$ . If instead  $\tau < \gamma$ , then by maximality of  $\tau$  we have  $e_{\tau+1} \equiv 0$ . Since all residuals from round 2 onward are zero, in particular  $\tilde{s}^{(\tau+1)} = 0$ . Using  $\tilde{s}^{(\tau+1)} = \delta^{(\tau)} - (e_{\tau+1}(1) + e_{\tau+1}(2))$ , we get  $0 = \delta^{(\tau)}$ . Hence in all cases,  $\delta^{(\tau)} = 0$ . But  $\delta^{(\tau)} = e_\tau(r_\tau)$ , so on this event one must have  $e_\tau(r_\tau) = 0$  for a nonzero polynomial  $e_\tau$  of degree at most 2.

For each  $j \in [\gamma]$ , let  $E_j$  be the event that Bad holds, all residuals  $\tilde{s}^{(k)}$  for  $k \in \{2, \dots, \gamma\}$  are zero, and  $j = \max\{k \in [\gamma] : e_k \neq 0\}$ . The events  $E_1, \dots, E_\gamma$  are pairwise disjoint. Fix  $j \in [\gamma]$ . Condition on any transcript up to, but not including, the sampling of  $r_j$ . At that point, the dealer has already VSS-shared  $\tilde{q}^{(j)}(1), \tilde{q}^{(j)}(2), \tilde{q}^{(j)}(3)$ , and therefore  $e_j$  is already fixed. On  $E_j$ , this polynomial is nonzero and has degree at most 2, and the event  $E_j$  implies  $e_j(r_j) = 0$ . Since  $r_j \xleftarrow{\$} \mathbb{F}_p \setminus \{1, 2, 3\}$ , the Schwartz–Zippel lemma gives  $\Pr[E_j] \leq 2/(\|\mathbb{F}_p\| - 3)$ . Summing over all  $j \in [\gamma]$  yields  $\Pr[\text{Bad} \wedge (\forall k \in \{2, \dots, \gamma\}, \tilde{s}^{(k)} = 0)] \leq 2\gamma/(\|\mathbb{F}_p\| - 3)$ . Combining the two cases,  $\Pr[\text{Bad}] \leq 1/\|\mathbb{F}_p\| + 2\gamma/(\|\mathbb{F}_p\| - 3) \leq (2\gamma + 1)/(\|\mathbb{F}_p\| - 3)$ . Since  $\gamma = \lceil \log L \rceil$ , this is exactly  $\epsilon_{\text{VIP}} = (2\lceil \log L \rceil + 1)/(\|\mathbb{F}_p\| - 3)$ .

Therefore, except with probability at most  $\epsilon_{\text{VIP}}$ , either the protocol aborts, or the output sharings are such that if the reconstructed values satisfy  $d_i^{(\gamma)} = a_i^{(1)}b_i^{(1)}$  and  $\sigma_i = 0$ , then necessarily  $c_i = \sum_{t=1}^L a_i^{(t)}b_i^{(t)}$ . This proves soundness, and hence the lemma.  $\square$

## K.2 Properties of Our Batched VIP Protocol

LEMMA K.2. *In the  $(\mathcal{F}_{\text{Coin}}, \mathcal{F}_{\text{Rand}}, \mathcal{F}_{\text{RSS.Mul}})$ -hybrid model,  $\Pi_{\text{VIP}}^{\text{PrI}}$  (Protocol 5) satisfies the following properties. Let  $\epsilon_{\text{VIP}} := (2\lceil \log L \rceil + 1)/(\|\mathbb{F}_p\| - 3)$ ,  $C_i^* := \sum_{t=1}^L a_i^{(t)}b_i^{(t)}$  and  $C^* := \sum_{i=1}^n C_i^*$ . We say that an output tuple  $(c, \Sigma, W(\rho), U(\rho), V(\rho))$  is accepting if it satisfies  $\Sigma = 0$  and  $W(\rho) = U(\rho)V(\rho)$ .*

- (1) **Correctness.** *If all underlying  $\Pi_{\text{VIP}}$  instances are executed honestly, then the protocol outputs consistent secret sharings of values satisfying  $c = C^*$ , and the corresponding output tuple is accepting.*
- (2) **Designated-verifier privacy.** *Let  $\text{View}_{\text{corr}}^{\text{VIP}}$  denote the joint view of the corrupted sharing servers in the underlying  $\Pi_{\text{VIP}}$  instances. The opened tuple  $(U(\rho), V(\rho), W(\rho), \Sigma, c)$  is statistically simulatable from  $c$ , the public coins, and  $\text{View}_{\text{corr}}^{\text{VIP}}$ . In particular, if the output tuple is accepting, then the opened tuple is simulatable from  $c$  and the fact that the tuple is accepting.*
- (3) **Soundness.** *For any adversary corrupting at most  $t < n/2$  servers, except with probability at most  $\epsilon_{\text{VIP}}^{\text{PrI}} = n\epsilon_{\text{VIP}} + 2n/(\|\mathbb{F}_p\| - 2n - 1) + 1/\|\mathbb{F}_p\|$ , either the protocol aborts, or the output sharings are consistent and satisfy the following implication: if the output tuple is accepting, then  $c = C^*$ .*

PROOF. We prove correctness, designated-verifier privacy, and soundness separately. For readability, write  $d_i$  for  $d_i^{(\gamma)}$ ,  $a_i$  for  $a_i^{(1)}$ , and  $b_i$  for  $b_i^{(1)}$  in the proof.

**Correctness.** Assume that all underlying  $\Pi_{\text{VIP}}$  instances are executed honestly. By Lemma K.1, for every  $i \in [n]$ , the  $i$ -th instance outputs consistent secret sharings of values satisfying  $c_i = C_i^*$ ,

$d_i = a_i b_i$ , and  $\sigma_i = 0$ . Protocol 5 computes  $\llbracket c \rrbracket \leftarrow \sum_{i=1}^n \llbracket c_i \rrbracket$  by local linear operations. Hence the reconstructed value satisfies  $c = \sum_{i=1}^n c_i = \sum_{i=1}^n C_i^* = C^*$ . Moreover, Protocol 5 computes  $\llbracket \Sigma \rrbracket \leftarrow \sum_{i=1}^n \epsilon'_i \llbracket \sigma_i \rrbracket$ . Since  $\sigma_i = 0$  for every  $i$ , we have  $\Sigma = 0$ . It remains to prove that  $W(\rho) = U(\rho)V(\rho)$ . By construction,  $U(x)$  and  $V(x)$  are degree at most  $n$  polynomials such that  $U(i) = a_i$  and  $V(i) = b_i$  for every  $i \in [n]$ . For each  $i \in [n]$ , the polynomial  $W(x)$  is interpolated so that  $W(i) = d_i$ . Hence, by correctness of each VIP instance,  $W(i) = d_i = a_i b_i = U(i)V(i)$  for every  $i \in [n]$ . For each  $i \in \{0, n+1, \dots, 2n\}$ , Protocol 5 obtains  $\llbracket W(i) \rrbracket = \llbracket U(i)V(i) \rrbracket$  from  $\mathcal{F}_{\text{RSS.Mul}}$ , and therefore  $W(i) = U(i)V(i)$  at these points as well. Thus the two degree at most  $2n$  polynomials  $W(x)$  and  $U(x)V(x)$  agree on the  $2n+1$  distinct points  $0, 1, \dots, 2n$ . Therefore they are identical polynomials, and in particular  $W(\rho) = U(\rho)V(\rho)$ . Hence the output tuple is accepting. All output sharings are consistent because they are obtained from consistent outputs of the VIP instances,  $\mathcal{F}_{\text{Rand}}$ , and  $\mathcal{F}_{\text{RSS.Mul}}$  by local operations and interpolation. This proves correctness.

**Designated-verifier privacy.** Recall that  $\text{View}_{\text{corr}}^{\text{VIP}}$  denotes the joint view of the corrupted sharing servers in the underlying  $\Pi_{\text{VIP}}$  instances, including their internal views and local outputs. Thus, for each corrupted sharing server  $S_i$ , the values  $\sigma_i$  and  $\Delta_i := d_i - a_i b_i$  are determined by  $\text{View}_{\text{corr}}^{\text{VIP}}$  and the public coins. We show that the opened tuple  $(U(\rho), V(\rho), W(\rho), \Sigma, c)$  can be simulated from  $c$ , the public coins, and  $\text{View}_{\text{corr}}^{\text{VIP}}$ . Let  $\mathcal{I}_{\text{corr}} \subseteq [n]$  with  $|\mathcal{I}_{\text{corr}}| \leq t$  denote the set of corrupted sharing servers. For each  $i \in [n]$ , define the local multiplication residual  $\Delta_i := d_i - a_i b_i$ . For every honest sharing server  $S_i$ , Lemma K.1 gives  $d_i = a_i b_i$  and  $\sigma_i = 0$ , and hence  $\Delta_i = 0$  and  $\sigma_i = 0$ . For every corrupted sharing server  $S_i$ , the values  $\Delta_i$  and  $\sigma_i$  are determined by  $\text{View}_{\text{corr}}^{\text{VIP}}$  and the public coins. Let  $E(x) := W(x) - U(x)V(x)$ . Then  $E(x)$  has degree at most  $2n$ . By the construction of  $W(x)$ , for every  $i \in [n]$  we have  $E(i) = d_i - a_i b_i = \Delta_i$ . For every  $i \in \{0, n+1, \dots, 2n\}$ ,  $W(i)$  is obtained from  $\mathcal{F}_{\text{RSS.Mul}}$  as  $W(i) = U(i)V(i)$ , so  $E(i) = 0$ . Therefore  $E(x)$  is completely determined by the values  $\{\Delta_i\}_{i \in [n]}$  and the public interpolation points. Since  $\Delta_i = 0$  for every honest  $i$ , the value  $E(\rho)$  is determined by the public coin  $\rho$  and the corrupted instances' residuals  $\{\Delta_i\}_{i \in \mathcal{I}_{\text{corr}}}$ . Similarly, since  $\Sigma = \sum_{i=1}^n \epsilon'_i \sigma_i$  and  $\sigma_i = 0$  for every honest  $i$ , the value  $\Sigma$  is determined by the public coins  $\{\epsilon'_i\}_{i \in [n]}$  and the corrupted instances' values  $\{\sigma_i\}_{i \in \mathcal{I}_{\text{corr}}}$ . It remains to argue that  $U(\rho)$  and  $V(\rho)$  are fresh uniform masks, even conditioned on  $c$ , the public coins, and  $\text{View}_{\text{corr}}^{\text{VIP}}$ . Let  $\lambda_0(\rho)$  be the Lagrange coefficient of the interpolation point 0 in the degree- $n$  interpolation of  $U(\rho)$  from the points  $0, 1, \dots, n$ . Since  $\rho \notin \{0, 1, \dots, 2n\}$ , in particular  $\rho \notin \{1, \dots, n\}$ , and hence  $\lambda_0(\rho) \neq 0$ . Thus  $U(\rho)$  can be written as  $U(\rho) = \lambda_0(\rho)a_0 + \Delta_U$ , where  $\Delta_U$  is determined by  $\rho$  and the values  $a_1, \dots, a_n$ . The value  $a_0$  is sampled by  $\mathcal{F}_{\text{Rand}}$  independently of all VIP outputs and of  $c$ . Moreover, by privacy of the RSS sharing produced by  $\mathcal{F}_{\text{Rand}}$ ,  $a_0$  remains uniformly distributed over  $\mathbb{F}_p$  conditioned on the corrupted parties' view. Since  $\lambda_0(\rho) \neq 0$ , it follows that  $U(\rho)$  is uniformly distributed over  $\mathbb{F}_p$  conditioned on  $c$ , the public coins, and  $\text{View}_{\text{corr}}^{\text{VIP}}$ . The same argument applies to  $V(\rho)$ , using the independently sampled random value  $b_0$ . Therefore  $U(\rho)$  and  $V(\rho)$  are independent uniform field elements conditioned on  $c$ , the public coins, and  $\text{View}_{\text{corr}}^{\text{VIP}}$ .

The simulator works as follows. Given  $c$ , the public coins, and  $\text{View}_{\text{corr}}^{\text{VIP}}$ , it computes  $\Delta_\Sigma := \sum_{i \in I_{\text{corr}}} \epsilon'_i \sigma_i$  and computes  $\Delta_W := E(\rho)$  from the residuals  $\{\Delta_i = d_i - a_i b_i\}_{i \in I_{\text{corr}}}$  and the public interpolation coefficients. It then samples  $u, v \xleftarrow{\$} \mathbb{F}_p$  independently and outputs  $(u, v, uv + \Delta_W, \Delta_\Sigma, c)$ . By the discussion above, in the real execution we have  $U(\rho)$  and  $V(\rho)$  distributed as independent uniform values  $u, v$ , while  $W(\rho) = U(\rho)V(\rho) + E(\rho)$  and  $\Sigma = \Delta_\Sigma$ . Hence the simulator's output distribution is identical to the real distribution of  $(U(\rho), V(\rho), W(\rho), \Sigma, c)$ . This proves the designated-verifier privacy statement.

Finally, suppose the output tuple is accepting. Then  $\Sigma = 0$  and  $W(\rho) = U(\rho)V(\rho)$ , i.e.,  $\Delta_\Sigma = 0$  and  $\Delta_W = 0$ . In this case the simulator does not need  $\text{View}_{\text{corr}}^{\text{VIP}}$ ; given only  $c$  and the fact that the tuple is accepting, it samples  $u, v \xleftarrow{\$} \mathbb{F}_p$  and outputs  $(u, v, uv, 0, c)$ . This is identically distributed to the real opened tuple conditioned on acceptance. Thus, in accepting executions, the opened tuple is simulatable from  $c$  and the fact that the tuple is accepting.

**Soundness.** Let  $\text{Bad}$  be the event that the protocol does not abort, the output tuple is accepting, and yet  $c \neq C^*$ . We bound the probability of  $\text{Bad}$ . For each  $i \in [n]$ , let  $\text{Good}_i$  denote the following condition: either the  $i$ -th VIP instance aborts, or it outputs consistent sharings such that if  $d_i = a_i b_i$  and  $\sigma_i = 0$ , then  $c_i = C_i^*$ . If  $S_i$  is honest, then  $\text{Good}_i$  holds by Lemma K.1 correctness. If  $S_i$  is corrupted, then by Lemma K.1 soundness,  $\text{Good}_i$  fails with probability at most  $\epsilon_{\text{VIP}}$ . By a union bound over all  $n$  instances, the event  $\text{Good} := \bigwedge_{i=1}^n \text{Good}_i$  fails with probability at most  $n\epsilon_{\text{VIP}}$ . We now condition on  $\text{Good}$  and on the event that the protocol does not abort. All output sharings are then consistent: the VIP outputs are consistent by  $\text{Good}$ ,  $\mathcal{F}_{\text{RSS}, \text{Mul}}$  outputs consistent sharings, and all remaining values are obtained by local linear operations and interpolation. Assume toward  $\text{Bad}$  that the output tuple is accepting but  $c \neq C^*$ . Since Protocol 5 computes  $c = \sum_{i=1}^n c_i$  and  $C^* = \sum_{i=1}^n C_i^*$ , the inequality  $c \neq C^*$  implies that there exists at least one index  $j \in [n]$  such that  $c_j \neq C_j^*$ . By  $\text{Good}_j$ , this implies that the local acceptance condition of the  $j$ -th VIP instance fails, namely  $d_j \neq a_j b_j$  or  $\sigma_j \neq 0$ . We distinguish two cases. First, suppose that  $\sigma_i \neq 0$  for at least one  $i \in [n]$ . The values  $\sigma_1, \dots, \sigma_n$  are fixed before the coefficients  $\epsilon'_1, \dots, \epsilon'_n$  are sampled by  $\mathcal{F}_{\text{Coin}}$ . Since the vector  $(\sigma_1, \dots, \sigma_n)$  is nonzero, the value  $\Sigma = \sum_{i=1}^n \epsilon'_i \sigma_i$  is a nonconstant affine function of at least one uniform coefficient  $\epsilon'_i$ . Therefore  $\Pr[\Sigma = 0] \leq 1/|\mathbb{F}_p|$ . Since an accepting output tuple requires  $\Sigma = 0$ , the probability of  $\text{Bad}$  in this case is at most  $1/|\mathbb{F}_p|$ . Second, suppose that  $\sigma_i = 0$  for every  $i \in [n]$ . Since  $c \neq C^*$  and  $\text{Good}$  holds, there must exist an index  $j$  such that  $d_j \neq a_j b_j$ . Define  $E(x) := W(x) - U(x)V(x)$ . As above,  $E(x)$  has degree at most  $2n$ , satisfies  $E(i) = d_i - a_i b_i$  for every  $i \in [n]$ , and satisfies  $E(i) = 0$  for every  $i \in \{0, n+1, \dots, 2n\}$ . Because  $E(j) = d_j - a_j b_j \neq 0$ , the polynomial  $E(x)$  is nonzero. The polynomial  $E(x)$  is fixed before  $\rho$  is sampled: the values defining  $U, V, W$  are fixed before the final call to  $\mathcal{F}_{\text{Coin}}$  that samples  $\rho$ . The challenge  $\rho$  is sampled uniformly from  $\mathbb{F}_p \setminus \{0, 1, \dots, 2n\}$ . Since  $E(x)$  is a nonzero polynomial of degree at most  $2n$ , the Schwartz–Zippel lemma gives  $\Pr[E(\rho) = 0] \leq 2n/(|\mathbb{F}_p| - 2n - 1)$ . But  $E(\rho) = 0$  is exactly the condition  $W(\rho) = U(\rho)V(\rho)$ . Thus, in this second case, the probability that the output tuple is accepting is at most  $2n/(|\mathbb{F}_p| - 2n - 1)$ .

Combining the failure probability of  $\text{Good}$  with the two cases above, we obtain  $\Pr[\text{Bad}] \leq n\epsilon_{\text{VIP}} + 2n/(|\mathbb{F}_p| - 2n - 1) + 1/|\mathbb{F}_p|$ . Therefore, except with probability at most  $\epsilon_{\text{VIP}}^{\text{PrI}} = n\epsilon_{\text{VIP}} + 2n/(|\mathbb{F}_p| - 2n - 1) + 1/|\mathbb{F}_p|$ , either the protocol aborts, or the output sharings are consistent and satisfy the implication that if the output tuple is accepting, then  $c = C^*$ . This proves soundness and completes the proof.  $\square$

### K.3 Proof of Theorem 5.1

Let  $I \subseteq [n]$  be the set of corrupted servers, with  $|I| \leq t$ . We write  $\mathcal{H} = [n] \setminus I$  for the honest servers (reserving  $H$  for the hash function). For each client input  $x^{(j)}$ , define  $a^{(j)} := x^{(j)} + k$  and  $b^{(j)} := (a^{(j)})^e$ . Thus the desired opened value before the final exponentiation is  $v^{(j)} = a^{(j)}b^{(j)} = (x^{(j)} + k)(a^{(j)})^e$ .

We first prove correctness. Assume that all parties are honest and that no ideal functionality aborts. By construction of the balanced assignment  $\{\Lambda_i\}_{i \in [n]}$ , for every  $j \in [m]$ , the product  $a^{(j)}b^{(j)}$  is decomposed as  $a^{(j)}b^{(j)} = \sum_{i=1}^n \sum_{\ell=1}^L a_i^{(j,\ell)} b_i^{(j,\ell)}$ . Protocol 6 samples additive zero shares  $[z^{(j)}]_1, \dots, [z^{(j)}]_n$  satisfying  $\sum_{i=1}^n [z^{(j)}]_i = 0$  and each server computes  $v_i^{(j)} = \sum_{\ell=1}^L a_i^{(j,\ell)} b_i^{(j,\ell)} + [z^{(j)}]_i$ . Therefore,  $\sum_{i=1}^n v_i^{(j)} = a^{(j)}b^{(j)}$ . The batching coefficients  $\epsilon_1, \dots, \epsilon_m$  are derived from the hashes of the values  $v_i^{(j)}$ . Protocol 6 then invokes  $\Pi_{\text{VIP}}^{\text{PrI}}$  on the flattened vectors  $\{\hat{a}_i^{(j,\ell)}, \hat{b}_i^{(j,\ell)}\}_{j \in [m], \ell \in [L]}$ , where  $\hat{a}_i^{(j,\ell)} = \epsilon_j a_i^{(j,\ell)}$  and  $\hat{b}_i^{(j,\ell)} = b_i^{(j,\ell)}$ . Thus, for each  $i \in [n]$ , the target inner product in the  $i$ -th underlying VIP instance is  $C_i^* = \sum_{j=1}^m \epsilon_j \sum_{\ell=1}^L a_i^{(j,\ell)} b_i^{(j,\ell)}$ . The effective input length of each VIP instance is  $mL$ . By Lemma K.2 correctness, the output tuple of  $\Pi_{\text{VIP}}^{\text{PrI}}$  is accepting and its value  $c$  satisfies  $c = \sum_{i=1}^n C_i^* = \sum_{j=1}^m \epsilon_j \sum_{i=1}^n \sum_{\ell=1}^L a_i^{(j,\ell)} b_i^{(j,\ell)}$ . Since the zero masks sum to zero for every  $j$ , we also have  $\sum_{i=1}^n v_i^{(j)} = \sum_{i=1}^n \sum_{\ell=1}^L a_i^{(j,\ell)} b_i^{(j,\ell)}$ . Hence  $c = \sum_{j=1}^m \epsilon_j \sum_{i=1}^n v_i^{(j)}$ . Therefore the client accepts all checks. Finally, since  $p = eg + 1$ , we have  $eg = p - 1$ . For each  $j$ ,  $((a^{(j)})^e)^g = (a^{(j)})^{eg} = 1$  because  $a^{(j)} \in \mathbb{F}_p^*$ . Thus the client outputs  $(v^{(j)})^g = ((x^{(j)} + k)(a^{(j)})^e)^g = (x^{(j)} + k)^g = F_k(x^{(j)})$ . This proves correctness.

We construct a simulator  $\mathcal{S}$  for any static adversary  $\mathcal{A}$  corrupting at most  $t$  servers and, possibly, the client  $C$ . The simulator internally emulates the hybrid functionalities toward  $\mathcal{A}$  and interacts with  $\mathcal{F}_{\text{dVOPRF}}^{F_k}$ .

**Extracting the client's input.** Whether or not  $C$  is corrupted,  $\mathcal{S}$  must hand  $\mathcal{F}_{\text{dVOPRF}}^{F_k}$  a well-defined input vector. Playing the honest servers,  $\mathcal{S}$  knows every mask  $r^{(j)}$ : the honest servers hold  $\llbracket r^{(j)} \rrbracket_i$  for  $n - t \geq t + 1$  indices  $i$ , and these cover all  $N$  additive components. At Step 2 of Protocol 6 it observes the values  $u^{(j)}$  that  $C$  sends to the honest servers. If the deferred echo checked at Step 6 fails—that is, if  $C$  sent different  $u^{(j)}$  to two honest servers—or if  $C$  aborts or sends nothing, then  $\mathcal{S}$  sends  $\perp$  to  $\mathcal{F}_{\text{dVOPRF}}^{F_k}$  and halts, matching the  $\perp$  branch of Fig. 2. Otherwise all honest servers hold a common  $u^{(j)}$  and  $\mathcal{S}$  submits  $x^{(j)} := u^{(j)} - r^{(j)}$ . By Lemma E.1 (consistency and extractability), this is exactly the value that the honest servers' sharings  $\llbracket x^{(j)} \rrbracket$  encode, so the ideal outputs  $y^{(j)} = F_k(x^{(j)})$  agree with the real execution. When  $C$  is honest, Lemma E.1 (input integrity) guarantees in addition that  $x^{(j)}$  is the value  $C$  actually supplied: a

corrupted server that lies during the mask opening is caught by the hash check, and  $C$  aborts.

We first treat the case in which  $\mathcal{A}$  corrupts  $C$ , and argue that the corrupted client's view is simulatable from the extracted input and the ideal output. Observe that nothing  $C$  sends influences what it receives: within  $\Pi_{\text{dVOPRF}}^m$  the client is a pure *receiver*. It cannot bias the batching coefficients  $\epsilon_j$ , which the servers derive by hashing their own  $h_i$ , nor the VIP challenge  $\rho$ , which  $\mathcal{F}_{\text{Coin}}$  produces among the servers; and the additive zero shares from  $\mathcal{F}_{\text{Zero}}$  that mask the  $v_i^{(j)}$  are independent of  $C$ 's messages. The client's only influence is its choice of  $u^{(j)}$ , which the extraction step above has already translated into an ideal input. The argument below therefore never uses semi-honesty of  $C$ . For each  $j \in [m]$ , the simulator receives the ideal output  $y^{(j)} = F_k(x^{(j)})$  from  $\mathcal{F}_{\text{dVOPRF}}^{F_k}$ . It samples  $v^{(j)}$  uniformly from the fiber  $\{v \in \mathbb{F}_p^* : v^g = y^{(j)}\}$ . This matches the real distribution: in the real protocol,  $v^{(j)} = (x^{(j)} + k)(\alpha^{(j)})^e$ , and since  $\alpha^{(j)}$  is uniform over  $\mathbb{F}_p^*$ ,  $(\alpha^{(j)})^e$  is uniform over the kernel of the map  $z \mapsto z^g$ . Hence  $v^{(j)}$  is uniform over the same fiber of  $y^{(j)}$ . The simulator then samples the values  $\{v_i^{(j)}\}_{i \in [n], j \in [m]}$  subject to the aggregate constraints  $\sum_{i=1}^n v_i^{(j)} = v^{(j)}$  for all  $j$ . If some servers are corrupted, their values are fixed consistently with the adversary's view, and the simulator samples the honest servers' values uniformly subject to the same aggregate constraints. This is identical to the real distribution because the additive zero shares mask the individual local products and reveal only their sum. The simulator computes the hashes  $h_i$ , the random-oracle-derived coefficients  $\epsilon_j$ , and sets  $c := \sum_{j=1}^m \epsilon_j v^{(j)}$ . It then simulates the proof-opening tuple  $(U(\rho), V(\rho), W(\rho), \Sigma, c)$  using Lemma K.2 designated-verifier privacy. Namely, the opened tuple is statistically simulatable from  $c$ , the public coins, and the joint view of the corrupted VIP instances. In accepting executions, it is simulatable from  $c$  and the fact that the tuple is accepting alone. Therefore, the client's entire view consisting of the opened values  $v_i^{(j)}$  and the proof tuple  $(U(\rho), V(\rho), W(\rho), \Sigma, c)$  is statistically identical to the real execution.

If the client is honest and only servers are corrupted, the simulator uses the honest client's inputs in the ideal functionality and simulates the corrupted servers' shares and messages using the privacy of the underlying RSS sharings and the ideal hybrid functionalities. In particular, at most  $t$  corrupted servers see only forbidden sets of RSS shares, and the values generated by  $\mathcal{F}_{\text{RandExp}}$ ,  $\mathcal{F}_{\text{Zero}}$ ,  $\mathcal{F}_{\text{Coin}}$ , and  $\mathcal{F}_{\text{Rand}}$  are simulated exactly according to their ideal distributions. The proof values produced by  $\Pi_{\text{VIP}}^{\text{PrI}}$  are simulated using Lemma K.2 privacy. Thus the corrupted servers' view is statistically identical to the real view, except for the bad events analyzed below, which account for the statistical simulation error.

It remains to bound the bad event in which a malicious adversary makes an honest client accept an incorrect output. This event is the only remaining way in which the real execution can differ from the ideal simulation. Let  $L_{\text{VIP}} := mL$  be the effective input length of each underlying VIP instance in Protocol 6. By Lemma K.2 soundness applied with input length  $L_{\text{VIP}}$ , except with probability at most  $\epsilon_{\text{VIP}}^{\text{PrI}}(mL) = n \cdot (2\lceil \log(mL) \rceil + 1) / (|\mathbb{F}_p| - 3) + 2n / (|\mathbb{F}_p| - 2n - 1) + 1 / |\mathbb{F}_p|$ , either  $\Pi_{\text{VIP}}^{\text{PrI}}$  aborts, or whenever its output tuple is accepting, the reconstructed value  $c$  satisfies  $c = \sum_{i=1}^n \sum_{j=1}^m \epsilon_j \sum_{\ell=1}^L a_i^{(j,\ell)} b_i^{(j,\ell)}$ .

Equivalently, letting  $y^{(j)} := \sum_{i=1}^n \sum_{\ell=1}^L a_i^{(j,\ell)} b_i^{(j,\ell)}$ , we have  $c = \sum_{j=1}^m \epsilon_j y^{(j)}$ . Now consider the values  $\tilde{v}^{(j)}$  reconstructed by the client from the server-provided values  $v_i^{(j)}$ , namely  $\tilde{v}^{(j)} = \sum_{i=1}^n v_i^{(j)}$ . The client accepts only if  $c = \sum_{j=1}^m \epsilon_j \tilde{v}^{(j)}$ . Combining this with the conclusion of Lemma K.2, an accepting execution with some incorrect opened value  $\tilde{v}^{(j)} \neq y^{(j)}$  must satisfy  $\sum_{j=1}^m \epsilon_j (\tilde{v}^{(j)} - y^{(j)}) = 0$ . If the error vector  $(\tilde{v}^{(1)} - y^{(1)}, \dots, \tilde{v}^{(m)} - y^{(m)})$  is nonzero, then this is a nontrivial linear equation in the random-oracle-derived batching coefficients  $\epsilon_1, \dots, \epsilon_m$ . Because these coefficients are derived after the values are committed by the hashes  $h_i$ , the probability that this equation holds is at most  $1/|\mathbb{F}_p|$ . Here  $a^{(j)} = x^{(j)} + k$  holds by Lemma E.1: on non-abort the honest servers' shares are consistent components of a sharing of a single well-defined  $x^{(j)}$ , and when  $C$  is honest that value is the one it supplied. This is what licenses the substitution below, and is precisely the step a non-verifiable input sharing would leave unjustified. Therefore, except with probability at most  $\epsilon_{\text{VIP}}^{\text{PrI}}(mL) + 1/|\mathbb{F}_p|$ , if the honest client accepts, then  $\tilde{v}^{(j)} = y^{(j)} = a^{(j)} b^{(j)} = (x^{(j)} + k)(\alpha^{(j)})^e$  for every  $j \in [m]$ . Consequently, the client's final outputs satisfy  $(\tilde{v}^{(j)})^g = ((x^{(j)} + k)(\alpha^{(j)})^e)^g = (x^{(j)} + k)^g = F_k(x^{(j)})$  for every  $j \in [m]$ . Combining the two error sources gives  $\epsilon_{\text{dVOPRF}} = \epsilon_{\text{VIP}}^{\text{PrI}}(mL) + 1/|\mathbb{F}_p|$ , namely  $\epsilon_{\text{dVOPRF}} = n \cdot (2\lceil \log(mL) \rceil + 1) / (|\mathbb{F}_p| - 3) + 2n / (|\mathbb{F}_p| - 2n - 1) + 2 / |\mathbb{F}_p|$ . Finally, we record what a corrupted client can learn about the key. Fig. 2 evaluates  $F_k$  on  $m$  points of  $\mathcal{A}$ 's choosing and returns nothing else, so a malicious client's view across all sessions is exactly that of an oracle for  $F_k$ . Key privacy against such a client therefore reduces to the pseudorandomness of Gold PRF, i.e. to the DSPRS assumption (Appendix G). No further leakage arises from the protocol itself: by Lemma K.2 (designated-verifier privacy), everything the client receives is statistically simulatable from the ideal outputs alone.

This completes the simulation proof.  $\square$

## L Fiat–Shamir Variants of Our Protocols

We can apply the Fiat–Shamir transform [19] to reduce the number of rounds in our VIP protocol  $\Pi_{\text{VIP}}$  (Protocol 4) as in [11], though our VIP protocol is not (public-coin) zero-knowledge proofs. Suppose that  $\{\llbracket s_1^{(k)} \rrbracket, \llbracket s_2^{(k)} \rrbracket, \llbracket s_3^{(k)} \rrbracket\}_{k \in [\lceil \log T \rceil]}$  are generated during the offline phase, and  $S_i$  knows all  $s_1^{(k)}, s_2^{(k)}, s_3^{(k)}$ . This can be done via random sharing and secure opening protocols. Let  $H$  be a random oracle. We give the formal description of the FS-applied VIP protocol  $\hat{\Pi}_{\text{VIP}}$  in Protocol 19. The Fiat–Shamir transform allows  $S_i$  to share  $(q^{(k)}(1), q^{(k)}(2), q^{(k)}(3))_{k \in [Y]}$  once, and therefore, the resulting protocol achieves constant rounds. Note that the Fiat–Shamir transform is not applicable to the batched VIP protocol (Protocol 5) and our dVOPRF protocol (Protocol 6).

## M Detailed Efficiency Analysis for Online Phases

We analyze the communication costs and the number of rounds of our dVOPRF protocol (Protocol 6) and compare it with the naive dVOPRF construction built from the ideal functionalities for secure multiplication  $\mathcal{F}_{\text{RSS,Mul}}$  and correctness verification of multiplications  $\mathcal{F}_{\text{Vrfy}}$  [11]. We assume that in both protocols, ideal functionalities independent of the client's input, i.e.,  $\mathcal{F}_{\text{RandExp}}$ ,  $\mathcal{F}_{\text{Coin}}$ ,  $\mathcal{F}_{\text{Rand}}$ , and  $\mathcal{F}_{\text{Zero}}$ , are called in the offline phases if the protocols allow.

---

**Protocol 19**  $\widehat{\Pi}_{VIP}$  – Fiat–Shamir Variant of Verifiable Computation for Inner Products over RSS in the  $\mathcal{F}_{Coin}$ -Hybrid Model

---

**Players:** Servers  $S_1, \dots, S_n$ , where  $S_i$  is a dealer.

**Input:** Servers hold  $\{\llbracket a_i^{(\ell)} \rrbracket, \llbracket b_i^{(\ell)} \rrbracket\}_{\ell \in [L]}$  and  $\{\llbracket s_1^{(k)} \rrbracket, \llbracket s_2^{(k)} \rrbracket, \llbracket s_3^{(k)} \rrbracket\}_{k \in [\lceil \log L \rceil]}$ , where  $S_i$  also knows  $\{a_i^{(\ell)}, b_i^{(\ell)}\}_{\ell \in [L]}$  and  $\{s_1^{(k)}, s_2^{(k)}, s_3^{(k)}\}_{k \in [\lceil \log L \rceil]}$ .

- 1: Let  $\gamma \leftarrow \lceil \log L \rceil$ .
- 2: **for**  $k = 1, \dots, \gamma$  **do**
- 3:   Let  $L \leftarrow L + 1$  if  $L$  is odd; let  $L \leftarrow L$  otherwise.
- 4:   Servers define (or update) polynomials  $f_1, f_2, \dots, f_L$ , and  $q^{(k)}$  as defined in Eqs. (1) and (2).
- 5:    $S_i$  computes  $\eta_j^{(k)} = q^{(k)}(j) - s_j^{(k)}$  for  $j = 1, 2, 3$ .
- 6:    $S_i$  computes  $r_k = H(\eta_1^{(1)} || \eta_2^{(1)} || \eta_3^{(1)} || \dots || \eta_1^{(k)} || \eta_2^{(k)} || \eta_3^{(k)})$ .
- 7:    $S_i$  sets  $\{a_i^{(\ell)}, b_i^{(\ell)}\} \leftarrow \{f_{2\ell-1}(r_k), f_{2\ell}(r_k)\}$  for all  $\ell \in [L/2]$ , and  $d_i \leftarrow q^{(k)}(r_k)$ , and sets  $L \leftarrow L/2$ .
- 8: **end**
- 9:  $S_i$  shares  $(q^{(k)}(1), q^{(k)}(2), q^{(k)}(3))_{k \in [\gamma]}$  to all servers using VSS and sends  $(\eta_1^{(k)}, \eta_2^{(k)}, \eta_3^{(k)})_{k \in [\gamma]}$ .
- 10: Servers locally compute  $\llbracket c_i \rrbracket \leftarrow \llbracket q_1(1) \rrbracket + \llbracket q_1(2) \rrbracket$  and  $\llbracket d_i \rrbracket \leftarrow \llbracket c_i \rrbracket$ .
- 11: Servers initialize  $L$ .
- 12: **for**  $k = 1, \dots, \gamma$  **do**
- 13:   Let  $L \leftarrow L + 1$  if  $L$  is odd; let  $L \leftarrow L$  otherwise.
- 14:   Servers compute  $r_k$  as above.
- 15:   Servers use Lagrange interpolation with  $\{\llbracket a_i^{(j)} \rrbracket, \llbracket b_i^{(j)} \rrbracket\}_{j \in [L]}$  to locally compute  $\{\llbracket a_i^{(\ell)} \rrbracket, \llbracket b_i^{(\ell)} \rrbracket\} \leftarrow \{\llbracket f_{2\ell-1}(r_k) \rrbracket, \llbracket f_{2\ell}(r_k) \rrbracket\}$  for all  $\ell \in [L/2]$ .
- 16:   Servers use Lagrange interpolation with  $(q^{(k)}(1), q^{(k)}(2), q^{(k)}(3))$  to locally compute  $\llbracket d_i \rrbracket \leftarrow \llbracket q^{(k)}(r_k) \rrbracket$ .
- 17:   Servers set  $L \leftarrow L/2$ .
- 18: **end**
- 19: Servers call  $\mathcal{F}_{Coin}$  to obtain  $\epsilon_2, \dots, \epsilon_\gamma \in \mathbb{F}_p$ , and locally updates  $\llbracket \sigma_i \rrbracket \leftarrow \sum_{k=2}^\gamma \epsilon_k (\llbracket d_i \rrbracket - \llbracket q^{(k)}(1) \rrbracket - \llbracket q^{(k)}(2) \rrbracket)$ .

**Output:** Servers record  $(\llbracket d_i \rrbracket, \llbracket a_i^{(1)} \rrbracket, \llbracket b_i^{(1)} \rrbracket, \llbracket \sigma_i \rrbracket, \llbracket c_i \rrbracket)$ .

---

Note that some of them must remain fixed at those points to ensure security; for example,  $\mathcal{F}_{Coin}$  at the last step in  $\Pi_{VIP}$  (Protocol 4) must be called.

Let  $N = \binom{n}{t}$ ,  $N' = \binom{n-1}{t}$ ,  $N'' = \binom{n-1}{t-1}$ , and  $L = N^2/n$ . We abstract the communication costs per server and the number of rounds for several protocols as follows:

- Let  $c_{Share}$  and  $r_{Share}$  denote the communication cost per server and the number of rounds for  $(t, n)$ -RSS sharing by a server dealer (as inside  $\Pi_{VIP}$  and  $\mathcal{F}_{vrfy}$ ); the client's input is handled separately by  $c_{Input}$  below. In this analysis, we consider two instantiations of  $\mathcal{F}_{RSS, Share}$ . The first one is the traditional RSS sharing protocol; a dealer generates RSS shares of the input  $x$  and distributes them to  $n$  parties. The communication cost is  $c_{Share} = N'$  field elements per party and it requires  $r_{Share} = 1$  round. The second one is Protocol 7. The communication cost for the online phase is at most

**Players:** Servers  $S_1, \dots, S_n$ , adversary  $\mathcal{A}$  corrupting up to  $t$  servers, and  $\mathcal{F}_{vrfy}$ .

**Input:** Honest servers  $\{S_i\}_{i \in \mathcal{H}}$  input their shares  $\{(\llbracket a^{(j)} \rrbracket_{\mathcal{H}}, \llbracket b^{(j)} \rrbracket_{\mathcal{H}}, \llbracket c^{(j)} \rrbracket_{\mathcal{H}})\}_{j=1}^m$ .

**Computation:**  $\mathcal{F}_{vrfy}$  computes  $\{(a^{(j)}, b^{(j)}, c^{(j)})\}$  and corrupted servers' shares  $\{(\llbracket a^{(j)} \rrbracket_{[n] \setminus \mathcal{H}}, \llbracket b^{(j)} \rrbracket_{[n] \setminus \mathcal{H}}, \llbracket c^{(j)} \rrbracket_{[n] \setminus \mathcal{H}})\}_{j=1}^m$  which are sent to  $\mathcal{A}$ . Then,  $\mathcal{F}_{vrfy}$  checks  $c^{(j)} = a^{(j)} \cdot b^{(j)}$  holds for all  $j \in [m]$ .

**Output:** If the above holds,  $\mathcal{F}_{vrfy}$  sends accept to  $\mathcal{A}$ . In this case,  $\mathcal{F}_{vrfy}$  waits for  $\mathcal{A}$  to send out  $i \in \{\text{accept}, \text{reject}\}$  which is then handed to honest server  $S_i$ . Otherwise,  $\mathcal{F}_{vrfy}$  sends reject to  $\mathcal{A}$  and the honest servers. In addition,  $\mathcal{F}_{vrfy}$  sends  $d^{(j)} = c^{(j)} - a^{(j)} \cdot b^{(j)}$  for each  $j \in [m]$  for which  $d^{(j)} \neq 0$  to  $\mathcal{A}$ .

**Figure 8:**  $\mathcal{F}_{vrfy}$  – Verifying Correctness of Multiplications

---

**Protocol 20**  $\Pi_{naive-dVOPRF}^m$  – Naive dVOPRF Protocol in the  $(\mathcal{F}_{RandExp}, \mathcal{F}_{Rand}, \mathcal{F}_{RSS, Mul}, \mathcal{F}_{vrfy}, \mathcal{F}_{RSS, Open})$ -Hybrid Model

---

**Players:** Client  $C$  and Servers  $S_1, \dots, S_n$ .

**Input:** Client  $C$  inputs  $(x^{(1)}, \dots, x^{(m)})$ , and servers inputs  $\llbracket k \rrbracket$ .

- 1: Servers call  $\mathcal{F}_{RandExp}(e)$  and  $\mathcal{F}_{Rand}$   $m$  times each to obtain  $(\llbracket (\alpha^{(j)})^e \rrbracket)_{j \in [m]}$  and the input masks  $(\llbracket r^{(j)} \rrbracket)_{j \in [m]}$ .
- 2: Client  $C$  and the servers run  $\Pi_{Input}$  (Protocol 9) to obtain  $(\llbracket x^{(j)} \rrbracket)_{j \in [m]}$ .
- 3: For every  $j \in [m]$ , let  $\llbracket a^{(j)} \rrbracket \leftarrow \llbracket x^{(j)} + k \rrbracket (= \llbracket x^{(j)} \rrbracket + \llbracket k \rrbracket)$  and  $\llbracket b^{(j)} \rrbracket \leftarrow \llbracket (\alpha^{(j)})^e \rrbracket$ .
- 4: For every  $j \in [m]$ , servers call  $\mathcal{F}_{RSS, Mul}$  with  $(\llbracket a^{(j)} \rrbracket, \llbracket b^{(j)} \rrbracket)$  to obtain  $\llbracket c^{(j)} \rrbracket$ .
- 5: Servers call  $\mathcal{F}_{vrfy}$  with  $\{(\llbracket a^{(j)} \rrbracket, \llbracket b^{(j)} \rrbracket, \llbracket c^{(j)} \rrbracket)\}_{j=1}^m$ .
- 6: If its output is reject, servers abort the protocol.
- 7: For every  $j \in [m]$ , client  $C$  and servers call  $\mathcal{F}_{RSS, Open}$  with  $\llbracket c^{(j)} \rrbracket$  to robustly reconstruct  $c^{(j)}$ . If at least one opening fails,  $C$  aborts the protocol.

**Output:** Client  $C$  outputs  $((c^{(1)})^g, \dots, (c^{(m)})^g)$ .

---

$c_{Share} \approx 1.5$  elements per party, and it requires  $r_{Share} = 2$  rounds.

- Let  $c_{Input}$  and  $r_{Input}$  denote the communication cost per server and the number of rounds of  $\Pi_{Input}$  (Protocol 9) for all  $m$  client inputs. Each additive component of each mask is sent to  $C$  by exactly one designated server and each server sends a single hash covering all  $m$  masks, so the servers send  $mN + n$  field elements to  $C$  in total;  $C$  then sends  $m$  elements to each server. Hence  $c_{Input} = m(N/n + 1) + 1$  elements per party, and  $r_{Input} = 3$  rounds standalone. Note that a direct client sharing would instead cost  $m \cdot c_{Share} = mN'$  elements per party, so  $\Pi_{Input}$  is cheaper whenever  $N/n + 1 < N'$ , which holds for every  $(n, t)$  we report. In  $\Pi_{naive-dVOPRF}^m$  the echo of  $\Pi_{Input}$  is merged into the hash exchange the protocol already performs at Step 6, so it contributes only  $r_{Input} - 1 = 2$  rounds there;  $\Pi_{naive-dVOPRF}^m$  has no such exchange and pays all 3.
- Let  $c_{Open}$  and  $r_{Open}$  denote the communication cost per server and the number of rounds for  $(t, n)$ -RSS Opening. In this

analysis, we consider two instantiations of  $\mathcal{F}_{\text{RSS}, \text{Open}}$ . The first one is the traditional RSS opening protocol; all parties send their shares  $\llbracket x \rrbracket$  to an opening party, and the honest opening party can reconstruct  $x$  by checking the consistency of each additive share in RSS shares, even if there are at most  $t < n/2$  corrupted parties. The communication cost is  $c_{\text{Open}} = N'$  field elements per party and it requires  $r_{\text{Open}} = 1$  round. The second one is Protocol 8. The communication cost for the online phase is at most  $c_{\text{Open}} \approx (N'' + 2t)$  elements per party,<sup>8</sup> and it requires only  $r_{\text{Open}} = 1$  round.

- Let  $c_{\text{Mul}}$  and  $r_{\text{Mul}}$  denote the communication cost per server and the number of rounds during secure multiplication protocols. The communication cost of Protocol 11 (or the DN multiplication protocol [16]) for the online phase is at most  $c_{\text{Mul}} \approx 1.5$  elements per party, and it requires  $r_{\text{Mul}} = 2$  rounds.
- Let  $c_{\text{Coin}}$  and  $r_{\text{Coin}}$  denote the communication cost per server and the number of rounds for  $\mathcal{F}_{\text{Coin}}$ . One of its popular instantiations is that parties call  $\mathcal{F}_{\text{Rand}}$  to get  $\llbracket r \rrbracket$  and opens it [13]. Therefore, its communication cost for the online phase is  $c_{\text{Open}}$  per party, since  $\mathcal{F}_{\text{Rand}}$  can be realized without any interactions if we allow its setup phase [14]. When applying the above RSS opening procedure, the communication cost for the online phase is at most  $c_{\text{Coin}} = N' \cdot (n - 1)$  elements per party, and it requires  $r_{\text{Coin}} = 1$  rounds. When applying Protocol 8, the communication cost for the online phase is at most  $c_{\text{Coin}} \approx N'' + 2t$  elements per party, and it requires  $r_{\text{Coin}} = 1$  rounds.

We give the detailed efficiency comparisons between the naive protocol  $\Pi_{\text{naive-dVOPRF}}^m$  and our protocol  $\Pi_{\text{dVOPRF}}^m$  in Tabs. 3 and 4. We elaborate on how we count them below.

## M.1 Communication Costs

*Naive dVOPRF protocol  $\Pi_{\text{naive-dVOPRF}}^m$ .* At the beginning of the online phase, client  $C$  and the servers run  $\Pi_{\text{Input}}$  to produce  $(\llbracket x^{(j)} \rrbracket)_{j \in [m]}$ , costing  $c_{\text{Input}}$  field elements per server. Servers then call  $m$  instances of  $\mathcal{F}_{\text{RSS}, \text{Mul}}$  in parallel, so each server sends  $m \cdot c_{\text{Mul}}$  field elements to other servers. Servers call  $\mathcal{F}_{\text{vrfy}}$  with  $3m$  RSS shares  $\{(\llbracket a^{(j)} \rrbracket, \llbracket b^{(j)} \rrbracket, \llbracket c^{(j)} \rrbracket)\}_{j=1}^m$ . An instantiation of  $\mathcal{F}_{\text{vrfy}}$  is given in [12, Protocol 4.2], which is the full version of [11]. In this instantiation, (a) servers call  $\mathcal{F}_{\text{Coin}}$   $m$  times, which can be computed in the offline phase, (b) each server sends to the other servers an RSS sharing of its additive share of the input products, (c) servers call  $n$  instances of  $\mathcal{F}_{\text{provDeg2Rel}}^{\text{abort}}$  with  $2m + 1$  RSS shares in parallel, and (d) each server calls  $\mathcal{F}_{\text{RSS}, \text{Open}}$  to reconstruct a certain RSS share  $\llbracket \beta \rrbracket$  and use it to check the validity of the computation. Step (b) requires  $nc_{\text{Share}}$  field elements per server, and Step (d) requires  $n \cdot c_{\text{Open}}$  field elements per server. Regarding Step (c), Boyle et al.'s protocol that securely realizes  $\mathcal{F}_{\text{provDeg2Rel}}^{\text{abort}}$  [11, (FS-applied) Protocol 3.2] is executed as follows.

- (1) A sharing server  $S_i$  repeats  $\lceil \log mL \rceil$  iterations for folding polynomials (with the output-randomization procedure at

the last iteration). Each iteration calls  $\mathcal{F}_{\text{Share}}$  to share 3 polynomial evaluations  $(q^{(k)}(1), q^{(k)}(2), q^{(k)}(3))$  (plus,  $q^{(k)}(0)$  and  $q^{(k)}(4)$  at the last iteration). Separately, the FS transform contributes  $3/n$  additional field elements of communication per server (except for the last iteration, in which  $5/n$  additional field elements are sent).

- (2) Servers call  $\mathcal{F}_{\text{Coin}}$   $\lceil \log mL \rceil$  times to securely update the RSS sharing of  $b = 0$ .
- (3) The servers batch the values  $\llbracket q(r) \rrbracket$ ,  $\llbracket f_1(r) \rrbracket$ ,  $\llbracket f_2(r) \rrbracket$ , and  $\llbracket b \rrbracket$  obtained from all instances into four shared verification values, using  $m$  calls to  $\mathcal{F}_{\text{Coin}}$  and  $n$  invocations of semi-honest secure multiplication. They then call  $\mathcal{F}_{\text{Open}}$  to reconstruct the four batched values and check whether the corresponding batched relations hold.

Note that  $\mathcal{F}_{\text{Coin}}$  after the iterations cannot be moved to the offline phase for security reasons. Therefore, the communication cost of an instance of  $\mathcal{F}_{\text{provDeg2Rel}}^{\text{abort}}$  is  $(3c_{\text{Share}} + 3/n)(\lceil \log mL \rceil - 1) + 5c_{\text{Share}} + 5/n + \lceil \log mL \rceil \cdot c_{\text{Coin}} + 4n \cdot c_{\text{Open}}$ .<sup>9</sup> Therefore,  $\mathcal{F}_{\text{vrfy}}$  for  $m$  secret-shared multiplication triples, instantiated as in [11, Protocol 4.2] requires  $(3nc_{\text{Share}} + 3)(\lceil \log mL \rceil - 1) + 6nc_{\text{Share}} + 5 + (\lceil \log mL \rceil + m) \cdot c_{\text{Coin}} + nc_{\text{Mul}} + 4n \cdot c_{\text{Open}}$  field elements for the communication cost per server. Finally, client  $C$  opens RSS shares,  $(\llbracket c^{(1)} \rrbracket, \dots, \llbracket c^{(m)} \rrbracket)$ , among servers, which requires  $m \cdot c_{\text{Open}}$  field elements for communication cost per server.

Overall, the communication cost per party of the naive dVOPRF protocol  $\Pi_{\text{naive-dVOPRF}}^m$  is

$$c_{\text{Input}} + 6n \cdot c_{\text{Share}} + (m + n) \cdot c_{\text{Mul}} + (\lceil \log mL \rceil + m) \cdot c_{\text{Coin}} + (3nc_{\text{Share}} + 3)(\lceil \log mL \rceil - 1) + (4n + m) \cdot c_{\text{Open}} + 5$$

field elements. When we instantiate  $\mathcal{F}_{\text{RSS}, \text{Share}}$  and  $\mathcal{F}_{\text{RSS}, \text{Open}}$  as the naive protocols, and  $\mathcal{F}_{\text{Coin}}$ , and  $\mathcal{F}_{\text{RSS}, \text{Mul}}$  as stated at the beginning of Appendix M, it turns to

$$N'((4n - 1)\lceil \log mL \rceil + (n + 1)m + 7n) + 3\lceil \log mL \rceil + 1.5(m + n) + 2$$

field elements.

On the other hand, if we instantiate  $\mathcal{F}_{\text{RSS}, \text{Share}}$  and  $\mathcal{F}_{\text{RSS}, \text{Open}}$  by Protocols 7 and 8, then we have

$$(N'' + 2t)(\lceil \log mL \rceil + 2m + 4n) + (4.5n + 3)\lceil \log mL \rceil + 3m + 6n + 2$$

field elements.

*Our dVOPRF protocol  $\Pi_{\text{dVOPRF}}^m$ .* We start with the sub-protocols.

**The VIP Protocol  $\Pi_{\text{VIP}}$ .** For each iteration  $k \in [\lceil \log L \rceil]$ , server  $S_i$  shares 3 field elements  $(q^{(k)}(1), q^{(k)}(2), q^{(k)}(3))$  with 3 more field elements  $(\eta_1^{(k)}, \eta_2^{(k)}, \eta_3^{(k)})$ , and then servers call  $\mathcal{F}_{\text{Coin}}$  to generate  $\epsilon_2, \dots, \epsilon_{\lceil \log L \rceil}$ . Hence, the communication cost is  $(3c_{\text{Share}} + 3/n) \cdot \lceil \log L \rceil + c_{\text{Coin}} \cdot (\lceil \log L \rceil - 1)$  field elements per server.

**The Batched VIP Protocol  $\Pi_{\text{VIP}}^{\text{Prf}}$ .** Servers invoke  $n$  instances of  $\Pi_{\text{VIP}}$  in parallel. For every  $k \in [\lceil \log L \rceil]$ , the challenge  $r_k$  is generated synchronously across all instances (by FS transform), and the common randomness  $\epsilon_k$  is used in all instances. We suppose that  $(\epsilon_k)_{k \in [\lceil \log L \rceil]}$  are shared among all servers by calling  $\mathcal{F}_{\text{Coin}}$  after the iteration steps of all the instances of the single VIP protocol

<sup>8</sup>We here regard a hash value as a single field element for simplicity, though it could be smaller, say, 256 bits.

<sup>9</sup>Although we may apply the batching technique [11, Section 3] to reduce the communication cost, we do not consider it here as it requires  $\mathcal{F}_{\text{RSS}, \text{Mul}}$  that increases the number of rounds.

**Table 3: Efficiency comparison for online phases when instantiating  $\mathcal{F}_{\text{RSS.Share}}$  and  $\mathcal{F}_{\text{RSS.Open}}$  by simple ones (i.e., generating/receiving shares and distributing/reconstructing them).**

| $(n, t)$ | Protocol (online)             | $m = 1$       |          | $m = 35$      |          | $m = 100$     |          |
|----------|-------------------------------|---------------|----------|---------------|----------|---------------|----------|
|          |                               | Comm.         | Rnds     | Comm.         | Rnds     | Comm.         | Rnds     |
| (3, 1)   | $\Pi_{\text{naive-dVOPRF}}^m$ | 109           | 13       | 557           | 13       | 1 224.5       | 13       |
| (3, 1)   | Our $\Pi_{\text{dVOPRF}}^m$   | <b>72</b>     | <b>9</b> | <b>299</b>    | <b>9</b> | <b>544</b>    | <b>9</b> |
| (5, 2)   | $\Pi_{\text{naive-dVOPRF}}^m$ | 840           | 13       | 2 598         | 13       | 4 957.5       | 13       |
| (5, 2)   | Our $\Pi_{\text{dVOPRF}}^m$   | <b>633</b>    | <b>9</b> | <b>1 354</b>  | <b>9</b> | <b>1 731</b>  | <b>9</b> |
| (7, 3)   | $\Pi_{\text{naive-dVOPRF}}^m$ | 5 485         | 13       | 13 215        | 13       | 23 888.5      | 13       |
| (7, 3)   | Our $\Pi_{\text{dVOPRF}}^m$   | <b>4 470</b>  | <b>9</b> | <b>7 423</b>  | <b>9</b> | <b>8 964</b>  | <b>9</b> |
| (9, 4)   | $\Pi_{\text{naive-dVOPRF}}^m$ | 32 056        | 13       | 66 302        | 13       | 113 230.5     | 13       |
| (9, 4)   | Our $\Pi_{\text{dVOPRF}}^m$   | <b>27 373</b> | <b>9</b> | <b>40 182</b> | <b>9</b> | <b>46 128</b> | <b>9</b> |

$\Pi_{\text{VIP}}$ . Therefore, those  $n$  instances of  $\Pi_{\text{VIP}}$  requires  $(3nc_{\text{Share}} + 3) \cdot \lceil \log L \rceil + c_{\text{Coin}} \cdot (\lceil \log L \rceil - 1)$  field elements for the communication cost per server. Servers also call  $n+1$  instances of  $\mathcal{F}_{\text{RSS.Mul}}$  in parallel, so each server sends  $(n+1)c_{\text{Mul}}$  field elements to other servers. Finally, servers call  $\mathcal{F}_{\text{Coin}}$  to get randomness  $\rho$ , which requires  $c_{\text{Coin}}$  field elements per server for the communication cost. Overall, the communication cost of  $\Pi_{\text{VIP}}^{\text{PrI}}$  is  $(3nc_{\text{Share}} + 3) \cdot \lceil \log L \rceil + c_{\text{Coin}} \cdot \lceil \log L \rceil + (n+1)c_{\text{Mul}}$  field elements per server. An important observation here is that, as can be seen, the communication cost of  $\Pi_{\text{VIP}}^{\text{PrI}}$  is logarithmic in the number of terms  $L$  to be shared.

**Our dVOPRF protocol  $\Pi_{\text{dVOPRF}}^m$ .** Now, we are ready to analyze the communication cost of  $\Pi_{\text{dVOPRF}}^m$ . Client  $C$  and the servers run  $\Pi_{\text{Input}}$  to produce  $(\llbracket x^{(j)} \rrbracket)_{j \in [m]}$ , costing  $c_{\text{Input}}$  field elements per server. Each server  $S_i$  sends a hash value  $h_i$  to the other servers, incurring  $n-1$  communication costs per server.<sup>10</sup> Servers invoke  $\Pi_{\text{VIP}}^{\text{PrI}}$  for  $mL$  terms, which requires  $(3nc_{\text{Share}} + 3) \cdot \lceil \log mL \rceil + c_{\text{Coin}} \cdot \lceil \log mL \rceil + (n+1)c_{\text{Mul}}$  field elements for communications. Finally, client  $C$  opens RSS shares,  $(\llbracket W(\rho) \rrbracket, \llbracket U(\rho) \rrbracket, \llbracket V(\rho) \rrbracket, \llbracket \Sigma \rrbracket, \llbracket c \rrbracket)$ , among servers, and receives additive shares  $(v_i^{(1)}, \dots, v_i^{(m)})_{i=1}^n$  from all servers. Therefore, each server sends  $(5 \cdot c_{\text{Open}} + m)$  field elements to  $C$ .

Overall, the communication cost of our dVOPRF protocol  $\Pi_{\text{dVOPRF}}^m$  is

$$c_{\text{Input}} + (3nc_{\text{Share}} + c_{\text{Coin}} + 3) \cdot \lceil \log mL \rceil + (n+1)c_{\text{Mul}} + 5 \cdot c_{\text{Open}} + m + n - 1$$

field elements. When we instantiate  $\mathcal{F}_{\text{RSS.Share}}$  and  $\mathcal{F}_{\text{RSS.Open}}$  as the naive protocols, and  $\mathcal{F}_{\text{Coin}}$ , and  $\mathcal{F}_{\text{RSS.Mul}}$  as stated at the beginning of Appendix M, it turns to

$$N'((4n-1)\lceil \log mL \rceil + m + 5) + 3\lceil \log mL \rceil + m + 2.5n + 0.5$$

field elements.

On the other hand, if we instantiate  $\mathcal{F}_{\text{RSS.Share}}$  and  $\mathcal{F}_{\text{RSS.Open}}$  by Protocols 7 and 8, then we have

$$(N'' + 2t)(\lceil \log mL \rceil + 5) + (4.5n + 3)\lceil \log mL \rceil + 2.5(m + n) + 0.5$$

field elements.

<sup>10</sup>We here regard a hash value as a single field element for simplicity, though it could be smaller, say, 256 bits.

**Table 4: Efficiency comparison for online phases when instantiating  $\mathcal{F}_{\text{RSS.Share}}$  and  $\mathcal{F}_{\text{RSS.Open}}$  by Protocols 7 and 8.**

| $(n, t)$ | Protocol (online)             | $m = 1$        |           | $m = 35$       |           | $m = 100$      |           |
|----------|-------------------------------|----------------|-----------|----------------|-----------|----------------|-----------|
|          |                               | Comm.          | Rnds      | Comm.          | Rnds      | Comm.          | Rnds      |
| (3, 1)   | $\Pi_{\text{naive-dVOPRF}}^m$ | 105.5          | 15        | 526            | 15        | 1 182.5        | 15        |
| (3, 1)   | Our $\Pi_{\text{dVOPRF}}^m$   | <b>66</b>      | <b>10</b> | <b>265.5</b>   | <b>10</b> | <b>499.5</b>   | <b>10</b> |
| (5, 2)   | $\Pi_{\text{naive-dVOPRF}}^m$ | 381            | 15        | 1 245.5        | 15        | 2 611.5        | 15        |
| (5, 2)   | Our $\Pi_{\text{dVOPRF}}^m$   | <b>225.5</b>   | <b>10</b> | <b>529</b>     | <b>10</b> | <b>822.5</b>   | <b>10</b> |
| (7, 3)   | $\Pi_{\text{naive-dVOPRF}}^m$ | 1 126.5        | 15        | 3 087          | 15        | 6 415.5        | 15        |
| (7, 3)   | Our $\Pi_{\text{dVOPRF}}^m$   | <b>575</b>     | <b>10</b> | <b>1 090.5</b> | <b>10</b> | <b>1 656.5</b> | <b>10</b> |
| (9, 4)   | $\Pi_{\text{naive-dVOPRF}}^m$ | 3 688          | 15        | 9 138.5        | 15        | 18 746         | 15        |
| (9, 4)   | Our $\Pi_{\text{dVOPRF}}^m$   | <b>1 542.5</b> | <b>10</b> | <b>2 624</b>   | <b>10</b> | <b>3 879</b>   | <b>10</b> |

## M.2 Round Complexity

We here consider the same instantiations of  $\mathcal{F}_{\text{RSS.Share}}$ ,  $\mathcal{F}_{\text{RSS.Open}}$ ,  $\mathcal{F}_{\text{RSS.Mul}}$ ,  $\mathcal{F}_{\text{vrfy}}$ , and  $\mathcal{F}_{\text{Coin}}$  as in the previous section.

**Naive dVOPRF protocol  $\Pi_{\text{naive-dVOPRF}}^m$ .** For the online phase, it runs  $\Pi_{\text{Input}}$  at Step 2, and calls  $\mathcal{F}_{\text{RSS.Mul}}$  at Step 4,  $\mathcal{F}_{\text{vrfy}}$  at Step 5, and  $\mathcal{F}_{\text{RSS.Open}}$  at the last step. Since this protocol performs no server-to-server hash exchange of its own, the echo of  $\Pi_{\text{Input}}$  cannot be absorbed and costs a round of its own. In particular,  $\mathcal{F}_{\text{vrfy}}$  can be instantiated with invocations of  $\mathcal{F}_{\text{Coin}}$ ,  $\mathcal{F}_{\text{RSS.Share}}$ ,  $\mathcal{F}_{\text{provDeg2Rel}}^{\text{abort}}$ , and  $\mathcal{F}_{\text{RSS.Open}}$ , though  $\mathcal{F}_{\text{Coin}}$  can be moved to the offline phase, so we do not count its rounds. Since the Boyle et al.'s protocol that realizes  $\mathcal{F}_{\text{provDeg2Rel}}^{\text{abort}}$  [11, Protocol 3.2] invokes invocations of  $\mathcal{F}_{\text{RSS.Share}}$ ,  $\mathcal{F}_{\text{Coin}}$ ,  $\mathcal{F}_{\text{Mul}}$ , and  $\mathcal{F}_{\text{RSS.Open}}$ . Therefore, Boyle et al.'s instantiation of  $\mathcal{F}_{\text{vrfy}}$  [11, Protocol 4.2] requires  $(2r_{\text{Share}} + r_{\text{Coin}} + r_{\text{Mul}} + 2r_{\text{Open}})$  rounds. Overall, the number of rounds of the online phase of  $\Pi_{\text{naive-dVOPRF}}^m$  is

$$r_{\text{Input}} + 2r_{\text{Share}} + r_{\text{Coin}} + 2r_{\text{Mul}} + 3r_{\text{Open}},$$

namely 13 rounds with the instantiations we employ. If we instantiate  $\mathcal{F}_{\text{RSS.Share}}$  and  $\mathcal{F}_{\text{RSS.Open}}$  by Protocols 7 and 8, then we have 15 rounds.

**Our dVOPRF protocol  $\Pi_{\text{dVOPRF}}^m$ .** We begin with sub-protocols.

**The VIP protocol  $\Pi_{\text{VIP}}$ .** By applying the FS transform as in Boyle et al. [11], or by moving  $\mathcal{F}_{\text{Coin}}$  in the iterations to the offline phase, the iterations in  $\Pi_{\text{VIP}}$  can be non-interactive, and hence all elements computed during the iterations can be shared among other servers one-shot.  $\Pi_{\text{VIP}}$  then calls  $\mathcal{F}_{\text{Coin}}$ , and hence, it requires  $r_{\text{Share}} + r_{\text{Coin}}$  rounds in total.

**The batched VIP protocol  $\Pi_{\text{VIP}}^{\text{PrI}}$ .** Servers invoke  $n$  instances of  $\Pi_{\text{VIP}}$  in parallel, which can be done in  $(r_{\text{Share}} + r_{\text{Coin}})$  rounds.  $\mathcal{F}_{\text{Coin}}$  to generate  $\epsilon'_1, \dots, \epsilon'_n$  can be simultaneously called with  $\mathcal{F}_{\text{Coin}}$  at the last step of the single VIP protocol  $\Pi_{\text{VIP}}$ , which shares the same randomness  $\{\epsilon_k\}_{k \in [\lceil \log L \rceil]}$  across all the instances. Servers also call  $n+1$  instances of  $\mathcal{F}_{\text{RSS.Mul}}$  in parallel, which can also be done in  $r_{\text{Mul}}$  rounds. Finally, servers call  $\mathcal{F}_{\text{Coin}}$  to get  $\rho$ , requiring  $r_{\text{Coin}}$  rounds. Therefore, the total number of rounds in  $\Pi_{\text{VIP}}^{\text{PrI}}$  is  $r_{\text{Share}} + 2r_{\text{Coin}} + r_{\text{Mul}}$ . Note that we here assume that  $\mathcal{F}_{\text{Rand}}$  is called in the offline phase.

**Our dVOPRF protocol  $\Pi_{\text{dVOPRF}}^m$ .** For the online phase, it runs  $\Pi_{\text{Input}}$  at Step 2, which requires  $r_{\text{Input}} - 1 = 2$  rounds here because

its echo is absorbed into Step 6. Step 6 requires 1 round to exchange hash values, and carries that echo at no extra cost.  $\Pi_{\text{dVOPRF}}^m$  invokes  $\Pi_{\text{VIP}}^{\text{PrI}}$  at Step 10, and calls  $\mathcal{F}_{\text{RSS}, \text{Open}}$  at Step 11. Overall, the number of rounds of the online phase of  $\Pi_{\text{dVOPRF}}^m$  is

$$(r_{\text{Input}} - 1) + r_{\text{Share}} + 2r_{\text{Coin}} + r_{\text{Mul}} + r_{\text{Open}} + 1,$$

namely 9 rounds with the instantiations we employ. If we instantiate  $\mathcal{F}_{\text{RSS}, \text{Share}}$  and  $\mathcal{F}_{\text{RSS}, \text{Open}}$  by Protocols 7 and 8, then we have 10 rounds.

## N Analytical Evaluation Results

Tab. 5, Tab. 6, and Tab. 7 report the detailed numerical results of our experimental runs. Each table reports computation and WAN time in milliseconds and communication in KB. Comp denotes the local computation time and WAN the total wall-clock time over the network. In Tab. 5, P2P refers to data sent to a specific receiver and Bcast to data sent to all  $n - 1$  remaining receivers. In Tab. 6, S $\leftrightarrow$ S refers to communication between servers and S $\leftrightarrow$ C to client input/output. For all tables, Rnds denotes the required rounds and Tot the total communication.

[Implementation rework pending: these numbers predate  $\Pi_{\text{Input}}$  (Protocol 9) and must be re-measured once it is implemented. Expected per-server communication delta at the simple instantiation: +0.05 KB at (3, 1); -0.09/-14.02 KB at (5, 2); -0.61/-65.58 KB at (7, 3); -2.53/-257.77 KB at (9, 4), for  $m = 1/m = 100$ . Rounds: ours 8  $\rightarrow$  9, baseline 11  $\rightarrow$  13, i.e. +100/+200 ms of WAN time at RTT = 100 ms before any change in  $T_{\text{Comp}}$ .]

## O Detailed Discussions of Further Improvements

### O.1 Saving Terms by Utilizing Local Share Conversion

In our dVOPRF protocol  $\Pi_{\text{dVOPRF}}^m$ ,  $L (= N^2/n)$  terms of the desired product  $c = ab = ([a]_1 + \dots + [a]_N)([b]_1 + \dots + [b]_N) = \sum_{u=1}^N \sum_{v=1}^N [a]_u \cdot [b]_v$  are assigned to each server. In this section, we present a technique to reduce the number of terms each server actually computes under the VIP protocol by employing the local share conversion. Specifically, each server  $S_i$  can use the local share conversion to non-interactively share  $N/n$  terms of  $c_i$ , which consists of  $L$  terms, among other servers. Therefore, it is sufficient for  $S_i$  to execute the VIP protocol  $\Pi_{\text{VIP}}$  with  $L - N/n (= (N(N-1)/n))$  terms.

*Stratified Balanced Assignments of Shares.* First, we revisit the share-assignment algorithm proposed in §5.1 to appropriately utilize the above technique. Recall the set of eligible servers  $\mathcal{E}_{u,v} := [n] \setminus (T_u \cup T_v)$  for term  $(u, v)$  for all  $T_u, T_v \in \mathcal{T}$ . In other words,  $\#\mathcal{E}_{u,v}$  servers share the same term  $[a]_u \cdot [b]_v$ . Since we have  $1 \leq \#\mathcal{E}_{u,v} \leq t + 1$  in the case of  $n = 2t + 1$ , the local share conversion can be applied to any term  $[a]_u \cdot [b]_v$  such that the corresponding set  $\mathcal{E}_{u,v}$  includes  $t + 1$  eligible servers.

For every  $k \in [t + 1]$ , let  $\mathcal{P}_k := \{(u, v) \in [N]^2 \mid \#\mathcal{E}_{u,v} = k\}$  be a  $k$ -th stratum, which represents the set of terms  $([a]_u, [b]_v)$  that exactly  $k$  eligible servers share. Clearly,  $N^2 = \sum_{k=1}^{t+1} \#\mathcal{P}_k$  holds.

“ $\mathcal{E}_{u,v}$  includes exactly  $k$  servers” means  $\#(T_u \cup T_v) = n - k$ , and therefore, we have  $\#(T_u \cap T_v) = \#T_u + \#T_v - \#(T_u \cup T_v) =$

$t + t - (n - k) = k - 1$ . Hence, to derive  $\#\mathcal{P}_k$  for every  $k \in [t + 1]$ , it is sufficient to count the number of pairs  $(T_u, T_v) \in \mathcal{T}^2$  such that  $\#(T_u \cap T_v) = k - 1$ . There are  $\binom{n}{k-1}$  intersections  $I := T_u \cap T_v$ ; there are  $\binom{n-(k-1)}{t-(k-1)}$  combinations of  $T_u$  for every intersection  $I$ ; and there are  $\binom{t+1}{t-(k-1)}$  combinations of  $T_v$  such that  $T_v \setminus I$  is disjoint from  $T_u \setminus I$  since after choosing  $T_u$ , remaining elements outside  $I \setminus (T \setminus I)$  are  $n - t = t + 1$ . Therefore, we have

$$\#\mathcal{P}_k = \binom{n}{k-1} \cdot \binom{2t-k}{t-k+1} \cdot \binom{t+1}{t-k+1}.$$

In particular, the number of terms to which we can apply the local share conversion is

$$\#\mathcal{P}_{t+1} = \binom{n}{t} \cdot \binom{t-1}{0} \cdot \binom{t+1}{0} = N$$

For every  $k \in [t + 1]$ ,  $\#\mathcal{P}_k$  is divisible by  $n$  since it contains  $\binom{n}{k-1}$  terms. Therefore, we can realize the *stratified balanced share-assignment algorithm*, which assigns terms  $[a]_u \cdot [b]_v$  so that the same number of terms per stratum is assigned to servers while every server can compute the terms only from its own shares. The construction is essentially the same as our balanced share-assignment algorithm in §5.1; we just apply it to every  $\mathcal{P}_k$ , instead of  $\mathcal{T}^2$ . Consequently,  $\sum_{k=1}^{t+1} \frac{\#\mathcal{P}_k}{n} (= L)$  terms are assigned to each server.

*The Most Effective Case:  $n = 3$ .* Let us consider the smallest case, i.e.,  $t = 1$  and  $n = 3$ , with RSS. Suppose that for a secret  $\gamma$ , three random elements  $\gamma_1, \gamma_2, \gamma_3 \in \mathbb{F}_p$  under the constraints that  $\gamma = \gamma_1 + \gamma_2 + \gamma_3$  (we here denote like this rather than  $\gamma = [\gamma]_1 + [\gamma]_2 + [\gamma]_3$  for simple notation).  $S_1, S_2$ , and  $S_3$  have their own shares  $[\gamma]_1 = (\gamma_2, \gamma_3)$ ,  $[\gamma]_2 = (\gamma_3, \gamma_1)$ , and  $[\gamma]_3 = (\gamma_1, \gamma_2)$ . For simplicity, we abuse the notation  $[\gamma]_i = (\gamma_{i+1}, \gamma_{i-1})$ .<sup>11</sup>

Now,  $S_i$  wants to compute  $c_i = a_{i+1}b_{i+1} + a_{i+1}b_{i-1} + a_{i-1}b_{i+1}$  and distribute it as RSS shares to  $S_{i-1}$  and  $S_{i+1}$ , as  $ab = (a_1 + a_2 + a_3)(b_1 + b_2 + b_3) = (a_2b_2 + a_3b_2 + a_2b_3) + (a_3b_3 + a_3b_1 + a_1b_3) + (a_1b_1 + a_2b_3 + a_3b_2) = c_1 + c_2 + c_3$ . A naive approach is to apply Protocol 4 or its Fiat-Shamir version (Protocol 19), which requires two iterations since the number of terms in  $c_i$  is three, i.e.,  $\lceil \log 3 \rceil = 2$ .

The key observation here is that every pair of  $(S_{i-1}, S_{i+1})$  has both  $a_i$  and  $b_i$  for  $i = 1, 2, 3$ . Therefore, we can employ the local share conversion technique for  $a_i \cdot b_i$ ; namely, servers can locally compute  $\llbracket a_i \cdot b_i \rrbracket = \{[a_i \cdot b_i]_{i-1}, [a_i \cdot b_i]_i, [a_i \cdot b_i]_{i+1}\} = \{(a_i \cdot b_i, 0), (0, 0), (0, a_i \cdot b_i)\}$ . Thus, it suffices for  $S_i$  to compute and distribute  $c_i = a_{i-1}b_{i+1} + a_{i+1}b_{i-1}$ , thereby reducing the number of iterations from two to one and saving 50% of the communication cost.

*General Case.* In a similar way to the above,  $S_i$ 's partial sum  $c_i = \sum_{(u,v) \in \hat{\Lambda}_i} [a]_u \cdot [b]_v$  can be divided into  $\hat{c}_i = \sum_{(u,v) \in \hat{\Lambda}_i \setminus \mathcal{P}_{t+1}} [a]_u \cdot [b]_v$  and  $\tilde{c}_i = \sum_{(u,v) \in \mathcal{P}_{t+1}} [a]_u \cdot [b]_v$ , where  $\hat{\Lambda}_i$  is a stratified balanced share assignment for  $S_i$ . Then, each  $S_i$  runs the VIP protocol  $\Pi_{\text{VIP}}$  with  $\hat{c}_i$  and shares  $\tilde{c}_i$  via using the local share conversion technique.<sup>12</sup> Servers then locally compute  $\llbracket c_i \rrbracket := \llbracket \hat{c}_i \rrbracket + \llbracket \tilde{c}_i \rrbracket$ .

For any  $t > 1$  and  $n > 3$ , unfortunately, the number of iterations remains the same, as  $\lceil \log L \rceil = \lceil \log(L - N/n) \rceil$  holds. However, this

<sup>11</sup>Here,  $\gamma_{i-1} = \gamma_3$  if  $i = 1$ , and  $\gamma_{i+1} = \gamma_1$  if  $i = 3$ .

<sup>12</sup>To be precise,  $S_i$  shares every term  $[a]_u \cdot [b]_v$  for  $(u, v) \in \mathcal{P}_{t+1}$  and locally computes  $\llbracket \tilde{c}_i \rrbracket := \sum_{(u,v) \in \mathcal{P}_{t+1}} \llbracket [a]_u \cdot [b]_v \rrbracket$ .

**Table 5: Offline phase benchmarks for  $m \in \{1, 100\}$ . Comm. in KB.  $\Pi_{\text{ZKPGen}}^{\text{Lig}}$  for  $N < 20$  and  $\Pi_{\text{AlyGen}}$  at  $(9, 4), m=100$  are omitted.**

| $(n, t)$ | Protocol                                | $m = 1$      |           |                  |               |                  |                | $m = 100$                                |           |                  |               |                  |                 |
|----------|-----------------------------------------|--------------|-----------|------------------|---------------|------------------|----------------|------------------------------------------|-----------|------------------|---------------|------------------|-----------------|
|          |                                         | Comp (ms)    | Rnds      | P2P (KB)         | Bcast (KB)    | Total (KB)       | WAN (ms)       | Comp (ms)                                | Rnds      | P2P (KB)         | Bcast (KB)    | Total (KB)       | WAN (ms)        |
| (3, 1)   | $\Pi_{\text{Aly}} [4, 30]$              | 16.1         | 21        | 19.64            | <b>2.25</b>   | 21.89            | 2 117.9        | 129.5                                    | 21        | 527.99           | <b>2.25</b>   | 530.24           | 2 272.9         |
| (3, 1)   | Our $\Pi_{\text{DegGen}}$               | <b>10.2</b>  | 11        | <b>8.06</b>      | <b>2.25</b>   | <b>10.31</b>     | 1 111.0        | <b>26.2</b>                              | 11        | <b>38.44</b>     | <b>2.25</b>   | <b>40.69</b>     | <b>1 129.5</b>  |
| (3, 1)   | Our $\Pi_{\text{ZKPGen}}^{\text{Vith}}$ | 12.2         | <b>10</b> | 13.02            | 11.16         | 24.17            | <b>1 014.2</b> | 207.7                                    | <b>10</b> | 533.75           | 892.81        | 1 426.56         | 1 324.6         |
| (5, 2)   | $\Pi_{\text{Aly}} [4, 30]$              | 56.8         | 25        | 249.09           | <b>14.06</b>  | 263.16           | 2 578.3        | 1 105.1                                  | 25        | 4 202.00         | <b>14.06</b>  | 4 216.04         | 3 950.5         |
| (5, 2)   | Our $\Pi_{\text{DegGen}}$               | <b>35.1</b>  | 25        | <b>144.38</b>    | <b>14.06</b>  | <b>158.44</b>    | 2 548.0        | <b>125.9</b>                             | 25        | <b>412.68</b>    | <b>14.06</b>  | <b>426.74</b>    | 2 660.9         |
| (5, 2)   | Our $\Pi_{\text{ZKPGen}}^{\text{Vith}}$ | 42.2         | <b>12</b> | 158.25           | 44.30         | 202.55           | <b>1 258.8</b> | 844.6                                    | <b>12</b> | 1 816.66         | 3 037.27      | 4 854.32         | <b>2 442.2</b>  |
| (7, 3)   | $\Pi_{\text{Aly}} [4, 30]$              | 529.1        | 29        | 2 300.48         | <b>78.75</b>  | 2 379.23         | 3 624.0        | 34 450.6                                 | 29        | 27 608.78        | <b>78.75</b>  | 27 687.53        | 39 618.7        |
| (7, 3)   | Our $\Pi_{\text{DegGen}}$               | <b>111.3</b> | 75        | <b>1 352.81</b>  | <b>78.75</b>  | <b>1 431.56</b>  | 7 728.6        | <b>1 304.0</b>                           | 75        | 3 126.30         | <b>78.75</b>  | 3 205.05         | 9 066.6         |
| (7, 3)   | Our $\Pi_{\text{ZKPGen}}^{\text{Vith}}$ | 149.1        | <b>12</b> | 1 492.64         | 197.80        | 1 690.44         | <b>1 487.5</b> | 4 213.2                                  | <b>12</b> | 5 296.68         | 11 984.01     | 17 280.69        | 6 828.8         |
| (7, 3)   | Our $\Pi_{\text{ZKPGen}}^{\text{Lig}}$  | 165.6        | <b>12</b> | 1 467.70         | 157.41        | 1 625.12         | 1 498.7        | 5 170.7                                  | <b>12</b> | <b>2 301.74</b>  | 1 482.66      | <b>3 784.40</b>  | <b>6 680.8</b>  |
| (9, 4)   | $\Pi_{\text{Aly}} [4, 30]$              | 11 660.7     | 31        | 16 548.17        | <b>413.44</b> | 16 961.61        | 16 150.2       | (skipped: witness store > memory budget) |           |                  |               |                  |                 |
| (9, 4)   | Our $\Pi_{\text{DegGen}}$               | <b>565.4</b> | 257       | 10 922.25        | <b>413.44</b> | <b>11 335.68</b> | 27 194.1       | <b>15 293.2</b>                          | 257       | 19 815.96        | <b>413.44</b> | <b>20 228.42</b> | 42 650.3        |
| (9, 4)   | Our $\Pi_{\text{ZKPGen}}^{\text{Vith}}$ | 677.4        | <b>14</b> | 10 927.63        | 913.79        | 11 841.42        | 3 047.5        | 30 451.2                                 | <b>14</b> | 21 061.59        | 50 448.49     | 71 510.07        | 37 709.3        |
| (9, 4)   | Our $\Pi_{\text{ZKPGen}}^{\text{Lig}}$  | 629.8        | <b>14</b> | <b>10 895.63</b> | 526.08        | 11 421.70        | <b>2 965.5</b> | 24 628.4                                 | <b>14</b> | <b>16 079.59</b> | 4 650.08      | 20 729.67        | <b>27 726.6</b> |

**Table 6: Online phase: our batched VIP (client-verified) vs. batched DZKP (server-verified) at  $m \in \{1, 100\}$ . Communication split into server $\leftrightarrow$ server (p2p + broadcast) and server $\leftrightarrow$ client, in KB. [Implementation rework pending: predates  $\Pi_{\text{Input}}$ ; re-measure before submission.]**

| $(n, t)$ | Protocol                          | $m = 1$      |          |                            |                            |               |                | $m = 100$      |          |                            |                            |               |                |
|----------|-----------------------------------|--------------|----------|----------------------------|----------------------------|---------------|----------------|----------------|----------|----------------------------|----------------------------|---------------|----------------|
|          |                                   | Comp (ms)    | Rnds     | S $\leftrightarrow$ S (KB) | S $\leftrightarrow$ C (KB) | Total (KB)    | WAN (ms)       | Comp (ms)      | Rnds     | S $\leftrightarrow$ S (KB) | S $\leftrightarrow$ C (KB) | Total (KB)    | WAN (ms)       |
| (3, 1)   | $\Pi_{\text{naive-dVOPRF}}^m [9]$ | 10.3         | 11       | 10.27                      | <b>0.52</b>                | 10.78         | 1 111.2        | 20.9           | 11       | 36.00                      | 51.57                      | 87.56         | 1 128.1        |
| (3, 1)   | Our $\Pi_{\text{dVOPRF}}^m$       | <b>1.2</b>   | <b>8</b> | <b>3.75</b>                | 2.25                       | <b>6.00</b>   | <b>801.7</b>   | <b>9.2</b>     | <b>8</b> | <b>13.59</b>               | <b>39.38</b>               | <b>52.97</b>  | <b>813.5</b>   |
| (5, 2)   | $\Pi_{\text{naive-dVOPRF}}^m [9]$ | 31.7         | 11       | 140.16                     | <b>1.79</b>                | 141.94        | 1 143.4        | 91.3           | 11       | 286.13                     | 178.13                     | 464.25        | 1 229.4        |
| (5, 2)   | Our $\Pi_{\text{dVOPRF}}^m$       | <b>6.1</b>   | <b>8</b> | <b>26.59</b>               | 8.42                       | <b>35.02</b>  | <b>808.9</b>   | <b>85.4</b>    | <b>8</b> | <b>58.09</b>               | <b>68.75</b>               | <b>126.84</b> | <b>895.8</b>   |
| (7, 3)   | $\Pi_{\text{naive-dVOPRF}}^m [9]$ | 98.7         | 11       | 1 304.30                   | <b>7.08</b>                | 1 311.38      | 1 306.1        | <b>586.6</b>   | 11       | 2 172.94                   | 707.82                     | 2 880.76      | 1 922.6        |
| (7, 3)   | Our $\Pi_{\text{dVOPRF}}^m$       | <b>35.4</b>  | <b>8</b> | <b>90.13</b>               | 34.75                      | <b>124.88</b> | <b>845.6</b>   | 993.5          | <b>8</b> | <b>155.10</b>              | <b>118.28</b>              | <b>273.38</b> | <b>1 815.9</b> |
| (9, 4)   | $\Pi_{\text{naive-dVOPRF}}^m [9]$ | 409.7        | 11       | 9 871.88                   | <b>30.20</b>               | 9 902.07      | 2 320.9        | <b>5 355.6</b> | 11       | 14 888.81                  | 3 018.76                   | 17 907.57     | <b>7 922.6</b> |
| (9, 4)   | Our $\Pi_{\text{dVOPRF}}^m$       | <b>284.0</b> | <b>8</b> | <b>237.09</b>              | 150.14                     | <b>387.23</b> | <b>1 115.7</b> | 11 505.3       | <b>8</b> | <b>347.34</b>              | <b>256.88</b>              | <b>604.22</b> | 12 354.8       |

technique still reduces the number of polynomials to be computed to  $f_1, \dots, f_{L-N/n}$ , which contributes to the efficiency improvement.

## O.2 Faster Online Protocol with Key-Dependent Offline Preprocessing

Protocol 21 gives an online-optimized variant of Protocol 6 by moving the expensive verifiable multiplication step to the preprocessing phase. In Protocol 6, the servers precompute only the input-independent randomness  $\llbracket (\alpha^{(j)})^e \rrbracket$ , and the verifiable multiplication  $\llbracket x^{(j)} + k \rrbracket \cdot \llbracket (\alpha^{(j)})^e \rrbracket$  is performed after the client input  $x^{(j)}$  is fixed. In contrast, Protocol 21 performs this multiplication in advance by replacing the unknown client input with a random mask  $r^{(j)}$ .

This optimization should be interpreted as *key-dependent* preprocessing. The preprocessed material depends on the shared PRF key  $\llbracket k \rrbracket$ , but remains independent of the client input. More precisely, once  $\llbracket k \rrbracket$  has been fixed, the servers sample, for each future query  $j \in [m]$ , a random mask  $\llbracket r^{(j)} \rrbracket$  and a random power  $\llbracket (\alpha^{(j)})^e \rrbracket$ , and then run the same verifiable multiplication as in Protocol 6 to obtain an RSS sharing  $\llbracket c^{(j)} \rrbracket$  of  $c^{(j)} := (r^{(j)} + k)(\alpha^{(j)})^e$ . During

this preprocessing, the servers also prepare the aggregated verification values  $\llbracket \Delta \rrbracket$  and  $\llbracket \Sigma \rrbracket$ . Thus, a preprocessing record consists of  $(\llbracket r^{(j)} \rrbracket, \llbracket (\alpha^{(j)})^e \rrbracket, \llbracket c^{(j)} \rrbracket)$  together with the shared verification values. Each such record is intended to be consumed once.

Once the client input  $x^{(j)}$  is later provided, the online phase requires no secure multiplication and no execution of the VIP protocol. The client first shares  $x^{(j)}$  with the servers, who compute  $\llbracket \delta^{(j)} \rrbracket := \llbracket x^{(j)} \rrbracket - \llbracket r^{(j)} \rrbracket$  and open  $\delta^{(j)}$  among themselves. Since  $r^{(j)}$  is random and unknown before the opening, this masked value does not reveal the client input. The servers then locally correct the preprocessed value by computing  $\llbracket v^{(j)} \rrbracket := \llbracket c^{(j)} \rrbracket + \delta^{(j)} \cdot \llbracket (\alpha^{(j)})^e \rrbracket$ . By linearity, the reconstructed value satisfies  $v^{(j)} = (r^{(j)} + k)(\alpha^{(j)})^e + (x^{(j)} - r^{(j)})(\alpha^{(j)})^e = (x^{(j)} + k)(\alpha^{(j)})^e$ . Therefore, the client can compute  $(v^{(j)})^g = (x^{(j)} + k)^g$ , as in the basic protocol.

This variant is particularly useful in deployments where the PRF key is long-lived, whereas client inputs arrive only later. For example, a threshold OPRF service may first agree on and secret-share a service key  $k$ , and then generate many single-use preprocessing records during idle time. These records can later be consumed by clients whose inputs were not known during preprocessing.

**Table 7: End-to-end benchmarks for  $m \in \{1, 100\}$ , with fresh  $\alpha^e$  regenerated per input  $m$ . Online phase is  $\Pi_{\text{dVOPRF}}^m$  throughout. Total sums p2p, broadcast, and server→client traffic; comm. in KB.  $\Pi_{\text{DZKP}}^{\text{AlyGen}}$  at  $(9, 4)$ ,  $m=100$  and  $\Pi_{\text{ZKPGen}}^{\text{Lig}}$  at  $N < 20$  are omitted. [Implementation rework pending; predates  $\Pi_{\text{Input}}$ ; re-measure before submission.]**

| $(n, t)$ | Protocol (offline + online)               | $m = 1$      |           |                  |                | $m = 100$                                |           |                  |                 |
|----------|-------------------------------------------|--------------|-----------|------------------|----------------|------------------------------------------|-----------|------------------|-----------------|
|          |                                           | Comp (ms)    | Rnds      | Total (KB)       | WAN (ms)       | Comp (ms)                                | Rnds      | Total (KB)       | WAN (ms)        |
| (3, 1)   | $\Pi_{\text{DZKP}}^{\text{Aly}}$          | 25.3         | 32        | 32.68            | 3 228.0        | 151.5                                    | 32        | 617.80           | 3 402.1         |
| (3, 1)   | Our $\Pi_{\text{dVOPRF}}^{\text{DegGen}}$ | <b>11.5</b>  | 19        | <b>16.31</b>     | 1 912.8        | <b>35.9</b>                              | 19        | <b>93.65</b>     | <b>1 943.6</b>  |
| (3, 1)   | Our $\Pi_{\text{dVOPRF}}^{\text{Vith}}$   | 13.2         | <b>18</b> | 30.17            | <b>1 815.7</b> | 219.2                                    | <b>18</b> | 1 479.54         | 2 140.4         |
| (4, 1)   | $\Pi_{\text{L-dOPRF}} [28]$               | 52.2         | <b>3</b>  | 4 425.22         | <b>706</b>     | 3 339.1                                  | <b>3</b>  | 442 522.00       | 39 041          |
| (5, 2)   | $\Pi_{\text{DZKP}}^{\text{Aly}}$          | 89.1         | 36        | 405.10           | 3 722.2        | 1 201.2                                  | 36        | 4 680.30         | 5 184.6         |
| (5, 2)   | Our $\Pi_{\text{dVOPRF}}^{\text{DegGen}}$ | <b>41.1</b>  | 33        | <b>193.45</b>    | 3 357.0        | <b>213.3</b>                             | 33        | <b>553.57</b>    | 3 558.7         |
| (5, 2)   | Our $\Pi_{\text{dVOPRF}}^{\text{Vith}}$   | 47.9         | <b>20</b> | 237.56           | <b>2 067.4</b> | 937.7                                    | <b>20</b> | 4 981.15         | <b>3 345.7</b>  |
| (7, 2)   | $\Pi_{\text{L-dOPRF}} [28]$               | 2 952.2      | <b>3</b>  | 1 761 188.35     | 144 147        | 294 866.9                                | <b>3</b>  | 176 118 835      | 14 384 674      |
| (7, 3)   | $\Pi_{\text{DZKP}}^{\text{Aly}}$          | 631.7        | 40        | 3 690.61         | 4 934.0        | 34 408.5                                 | 40        | 30 568.23        | 40 912.7        |
| (7, 3)   | Our $\Pi_{\text{dVOPRF}}^{\text{DegGen}}$ | <b>147.0</b> | 83        | <b>1 556.44</b>  | 8 574.5        | <b>2 317.9</b>                           | 83        | <b>3 478.45</b>  | 10 902.8        |
| (7, 3)   | Our $\Pi_{\text{dVOPRF}}^{\text{Vith}}$   | 185.9        | <b>20</b> | 1 815.32         | <b>2 334.6</b> | 5 164.0                                  | <b>20</b> | 17 554.13        | 8 602.1         |
| (7, 3)   | Our $\Pi_{\text{dVOPRF}}^{\text{Lig}}$    | 202.4        | <b>20</b> | 1 750.26         | 2 345.8        | 6 119.3                                  | <b>20</b> | 4 072.90         | <b>8 452.9</b>  |
| (9, 4)   | $\Pi_{\text{DZKP}}^{\text{Aly}}$          | 11 819.0     | 42        | 26 863.67        | 18 219.7       | (skipped: witness store > memory budget) |           |                  |                 |
| (9, 4)   | Our $\Pi_{\text{dVOPRF}}^{\text{DegGen}}$ | <b>834.7</b> | 265       | <b>11 722.91</b> | 28 295.0       | <b>26 562.0</b>                          | 265       | <b>20 832.69</b> | 54 768.7        |
| (9, 4)   | Our $\Pi_{\text{dVOPRF}}^{\text{Vith}}$   | 944.2        | <b>22</b> | 12 228.65        | 4 146.0        | 41 450.6                                 | <b>22</b> | 72 114.26        | 49 558.2        |
| (9, 4)   | Our $\Pi_{\text{dVOPRF}}^{\text{Lig}}$    | 907.3        | <b>22</b> | 11 809.23        | <b>4 074.7</b> | 35 516.3                                 | <b>22</b> | 21 371.62        | <b>39 467.1</b> |

The trade-off is that the preprocessing records are tied to the specific shared key  $\llbracket k \rrbracket$ . If the key is refreshed or replaced, the corresponding records cannot be reused and must be generated again. This is the main distinction between Protocol 21 and the fully key-independent preprocessing used to generate only  $\llbracket (\alpha^{(j)})^e \rrbracket$ .

*Efficiency Analysis.* We analyze only the online cost of Protocol 21, namely the cost incurred after the client inputs  $x^{(1)}, \dots, x^{(m)}$  have been fixed. The key-dependent preprocessing cost is excluded from this analysis. We assume that, before the online phase begins, the servers have already generated  $\llbracket r^{(j)} \rrbracket, \llbracket (\alpha^{(j)})^e \rrbracket, \llbracket c^{(j)} \rrbracket_{j \in [m]}$ , where  $\llbracket c^{(j)} \rrbracket$  is an RSS sharing of  $c^{(j)} := (r^{(j)} + k)(\alpha^{(j)})^e$ , and have also prepared the aggregated verification values  $\llbracket \Delta \rrbracket$  and  $\llbracket \Sigma \rrbracket$ .

*Communication costs.* Let  $N := \binom{n}{t}$ ,  $N' := \binom{n-1}{t}$ , and  $N'' := \binom{n-1}{t-1}$ . Let  $c_{\text{Share}}$  and  $r_{\text{Share}}$  denote the communication cost per server and the number of rounds for  $(t, n)$ -RSS sharing and also let  $c_{\text{Open}}$  and  $r_{\text{Open}}$  denote the communication cost per server and the number of rounds for  $(t, n)$ -RSS sharing.

Since all verifiable multiplication steps have already been completed during preprocessing, the online phase contains no invocation of  $\Pi_{\text{VIP}}^{\text{PrL}}$  and no online call to secure multiplication. Its communication consists of three parts. First, the client and the servers run  $\Pi_{\text{Input}}$  to produce  $\llbracket x^{(j)} \rrbracket_{j \in [m]}$ , which costs  $c_{\text{Input}}$  field elements per server. Second, the servers open the  $m$  masked differences  $\delta^{(j)} = x^{(j)} - r^{(j)}$  among themselves, which costs  $mn c_{\text{Open}}$  field elements per server. Third, the servers open the  $m$  output sharings  $\llbracket v^{(1)} \rrbracket, \dots, \llbracket v^{(m)} \rrbracket$  together with  $\llbracket \Delta \rrbracket$  and  $\llbracket \Sigma \rrbracket$  to the client, which costs  $(m+2)c_{\text{Open}}$  field elements per server. Therefore, the total online communication is  $c_{\text{Input}} + m(n+1) + 2c_{\text{Open}}$  field elements per server. When we instantiate  $\mathcal{F}_{\text{RSS.Open}}$  as stated at the beginning of Appendix M, it turns to  $N'(m(n+1)+2) + m(N/n+1) + 1$  field

elements. If we instantiate  $\mathcal{F}_{\text{RSS.Open}}$  by Protocol 8, then we have  $(N'' + 2t)(m(n+1)+2) + m(N/n+1) + 1$  field elements.

The key improvement over Protocol 6 is that the online phase no longer communicates the batched VIP proof. All proof generation, proof aggregation, and secure multiplication are shifted to the key-dependent preprocessing phase. Thus, once the client inputs are known, the online communication is dominated only by input sharing and RSS openings, and grows linearly in  $m$ .

*Round complexity.* We again count only the rounds after the client inputs are fixed. The key-dependent preprocessing rounds are excluded. The online phase consists of the following sequential steps. First, the client and the servers run  $\Pi_{\text{Input}}$ ; this costs  $r_{\text{Input}} - 1 = 2$  rounds, since its echo is attached to the opening of the  $\delta^{(j)}$  below. Second, after receiving the input sharings, the servers compute  $\llbracket \delta^{(j)} \rrbracket := \llbracket x^{(j)} \rrbracket - \llbracket r^{(j)} \rrbracket$  for all  $j \in [m]$ , and open all  $\delta^{(j)}$  values in one batched opening. This costs  $r_{\text{Open}}$  rounds. Third, the servers locally compute  $\llbracket v^{(j)} \rrbracket := \llbracket c^{(j)} \rrbracket + \delta^{(j)} \llbracket (\alpha^{(j)})^e \rrbracket$  for all  $j \in [m]$ , which adds no communication rounds. Finally, the servers open  $\llbracket v^{(1)} \rrbracket, \dots, \llbracket v^{(m)} \rrbracket$  together with  $\llbracket \Delta \rrbracket$  and  $\llbracket \Sigma \rrbracket$  in one batched opening, costing another  $r_{\text{Open}}$  rounds. Therefore, the online round complexity is  $(r_{\text{Input}} - 1) + 2r_{\text{Open}}$ . With either instantiation of  $\mathcal{F}_{\text{RSS.Open}}$  from the beginning of Appendix M, this is 4 rounds.

*Takeaway.* Protocol 21 trades key-independent preprocessing for a substantially lighter online phase. After the client inputs are fixed, the servers only open masked differences, perform local linear corrections, and open the final values together with two aggregated verification checks. Hence, when openings are batched, the number of online rounds is independent of  $m$ . This makes the protocol well-suited to latency-sensitive deployments with stable server-side keys,

---

**Protocol 21** Faster dOPRF Protocol in the  $(\mathcal{F}_{\text{RandExp}}, \mathcal{F}_{\text{Rand}}, \mathcal{F}_{\text{Coin}}, \mathcal{F}_{\text{RSS,Open}})$ -Hybrid Model
 

---

**Players:** Client  $C$  and Servers  $S_1, \dots, S_n$ .

**Input:** Client  $C$  inputs  $(x^{(1)}, \dots, x^{(m)})$ , and servers input  $\llbracket k \rrbracket$ .

- 1: Servers call  $\mathcal{F}_{\text{RandExp}}(e)$   $m$  times and  $\mathcal{F}_{\text{Rand}}$   $2m$  times to obtain  $(\llbracket (\alpha^{(j)})^e \rrbracket)_{j \in [m]}$ , the preprocessing masks  $(\llbracket r^{(j)} \rrbracket)_{j \in [m]}$ , and the independent input masks  $(\llbracket \tilde{r}^{(j)} \rrbracket)_{j \in [m]}$  used by  $\Pi_{\text{Input}}$ .
- 2: For every  $j \in [m]$ , let  $\llbracket a^{(j)} \rrbracket \leftarrow \llbracket r^{(j)} \rrbracket + k \rrbracket (= \llbracket r^{(j)} \rrbracket + \llbracket k \rrbracket)$  and  $\llbracket b^{(j)} \rrbracket \leftarrow \llbracket (\alpha^{(j)})^e \rrbracket$ .
- 3: For every  $i \in [n]$  and  $j \in [m]$ , let  $a_i^{(j,\ell)} \leftarrow [a^{(j)}]_{u_\ell}$  and  $b_i^{(j,\ell)} \leftarrow [b^{(j)}]_{v_\ell}$  for every  $(u_\ell, v_\ell) \in \Lambda_i$ , where  $\llbracket a^{(j)} \rrbracket_i = \{[a^{(j)}]_\ell \mid i \notin T_\ell \in \mathcal{T}\}$ ,  $\llbracket b^{(j)} \rrbracket_i = \{[b^{(j)}]_\ell \mid i \notin T_\ell \in \mathcal{T}\}$ , and  $\{\Lambda_i\}_{i \in [n]}$  is a balanced assignment (defined in §5.1).
- 4: Servers locally obtain  $\llbracket a_i^{(j,\ell)} \rrbracket$  and  $\llbracket b_i^{(j,\ell)} \rrbracket$  via local share conversion for all  $i \in [n]$ ,  $j \in [m]$ , and  $\ell \in [L]$ .
- 5: For every  $j \in [m]$ , servers invoke  $\Pi_{\text{VP}}^{\text{Prd}}$  with  $\{\llbracket a_i^{(j,\ell)} \rrbracket, \llbracket b_i^{(j,\ell)} \rrbracket\}_{\ell \in [L]}$  to obtain proof  $(\llbracket W^{(j)} \rrbracket, \llbracket U^{(j)} \rrbracket, \llbracket V^{(j)} \rrbracket, \llbracket \Sigma^{(j)} \rrbracket, \llbracket c^{(j)} \rrbracket)$ .
- 6: For every  $j \in [m]$ , servers invoke  $\mathcal{F}_{\text{RSS,Mul}}$  with  $\llbracket U^{(j)} \rrbracket$  and  $\llbracket V^{(j)} \rrbracket$  to obtain  $\llbracket P^{(j)} \rrbracket = \llbracket U^{(j)} \cdot V^{(j)} \rrbracket$ .
- 7: Servers call  $\mathcal{F}_{\text{Coin}}$  to obtain  $\epsilon_1, \dots, \epsilon_m \xleftarrow{\$} \mathbb{F}_p$  and locally compute  $\llbracket \Sigma \rrbracket = \sum_{j=1}^m \epsilon_j \llbracket \Sigma^{(j)} \rrbracket$  and  $\llbracket \Delta \rrbracket = \sum_{j=1}^m \epsilon_j (\llbracket W^{(j)} \rrbracket - \llbracket P^{(j)} \rrbracket)$ .
- 8: Client  $C$  and the servers run  $\Pi_{\text{Input}}$  (Protocol 9) on  $(x^{(j)})_{j \in [m]}$  with the masks  $(\llbracket \tilde{r}^{(j)} \rrbracket)_{j \in [m]}$ , attaching its echo to the opening of Step 7; the servers obtain  $(\llbracket x^{(j)} \rrbracket)_{j \in [m]}$ .
- 9: For every  $j \in [m]$ , servers locally compute  $\llbracket \delta^{(j)} \rrbracket = \llbracket x^{(j)} \rrbracket - \llbracket r^{(j)} \rrbracket$ , and then call  $\mathcal{F}_{\text{RSS,Open}}$  with  $\llbracket \delta^{(j)} \rrbracket$  to obtain  $\delta^{(j)}$ . If at least one opening fails, the servers abort the protocol.
- 10: Servers locally compute  $\llbracket v^{(j)} \rrbracket = \llbracket c^{(j)} \rrbracket + \delta^{(j)} \cdot \llbracket (\alpha^{(j)})^e \rrbracket$ .
- 11: Client  $C$  calls  $\mathcal{F}_{\text{RSS,Open}}$  to robustly open  $\{\llbracket v^{(j)} \rrbracket\}_{j \in [m]}$ ,  $\llbracket \Delta \rrbracket$ , and  $\llbracket \Sigma \rrbracket$ . If at least one opening fails,  $C$  aborts the protocol.
- 12: Client  $C$  aborts if  $\Delta \neq 0$  or  $\Sigma \neq 0$ .

**Output:** Client  $C$  outputs  $((v^{(1)})^g, \dots, (v^{(m)})^g)$ .
 

---

while being less attractive when keys are frequently refreshed or when preprocessing must remain reusable across different keys.